

Agent4System: 从工具调用到自主改进复杂系统

模型、接口、Harness 与反馈如何共同塑造 Agent 能力

Coding · Systems · Science · AI4AI

秦海岩 · 无问芯穹
2026.08.20

PPT内容为Agent协助生成



Agent4System: 模型能力如何变成系统能力

从 Harness、行动接口，到 2023—2026 的真实案例与能力来源

01 从模型到 Agent

为什么“会推理”
还不等于“能完成任务”？



02 从代码到系统

L1-L5 行动接口
决定 Agent 能做什么



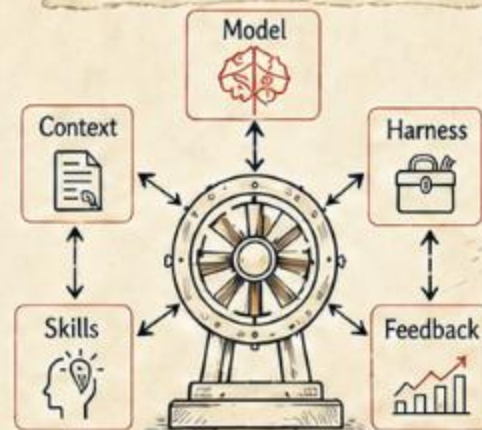
03 案例演化: 2023-2026

ChatEDA · A-Lab · SWE-agent ·
KernelBench · STRATUS · AISHPerf ·
PithTrain · VibeServe · Prime



04 能力来源与未来

Model · Context · Harness ·
Skills · Feedback



从模型到 Agent

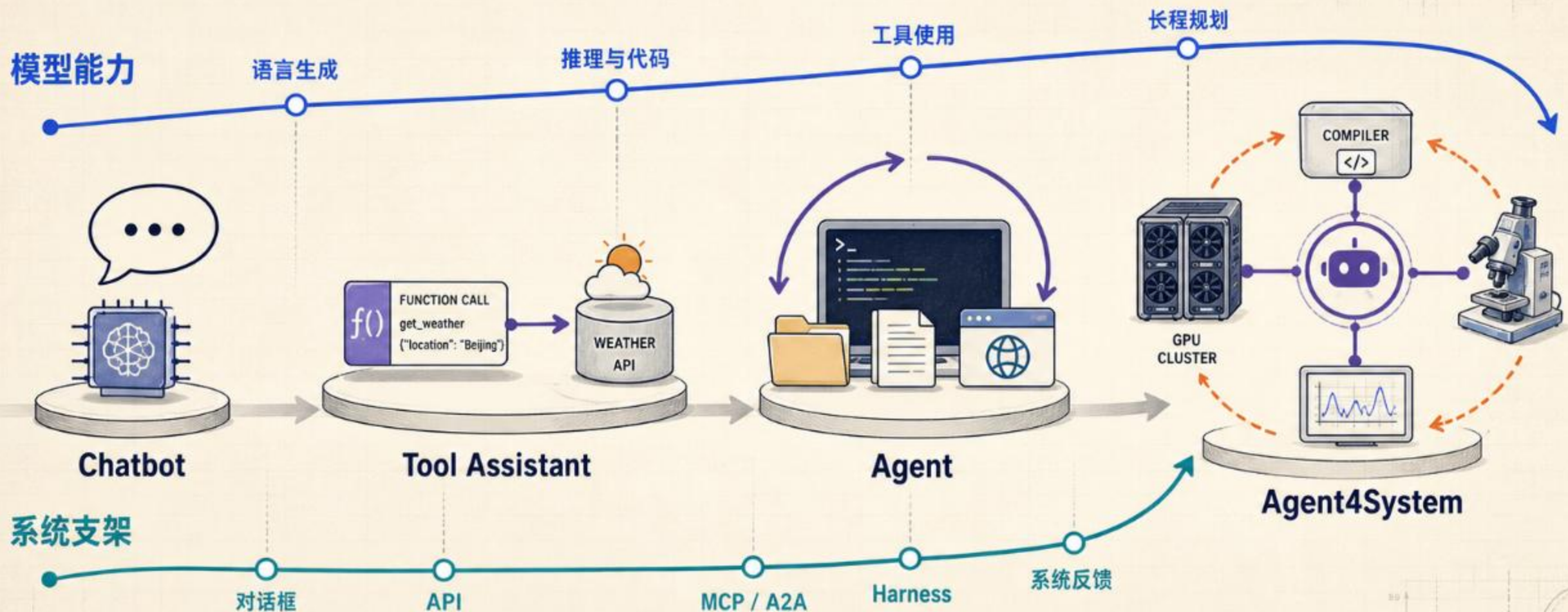
行动接口

案例演化

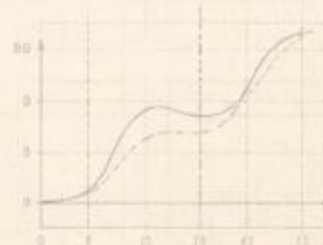
能力来源与未来

核心问题: Agent 的能力, 究竟来自模型, 还是模型与系统的共同设计?

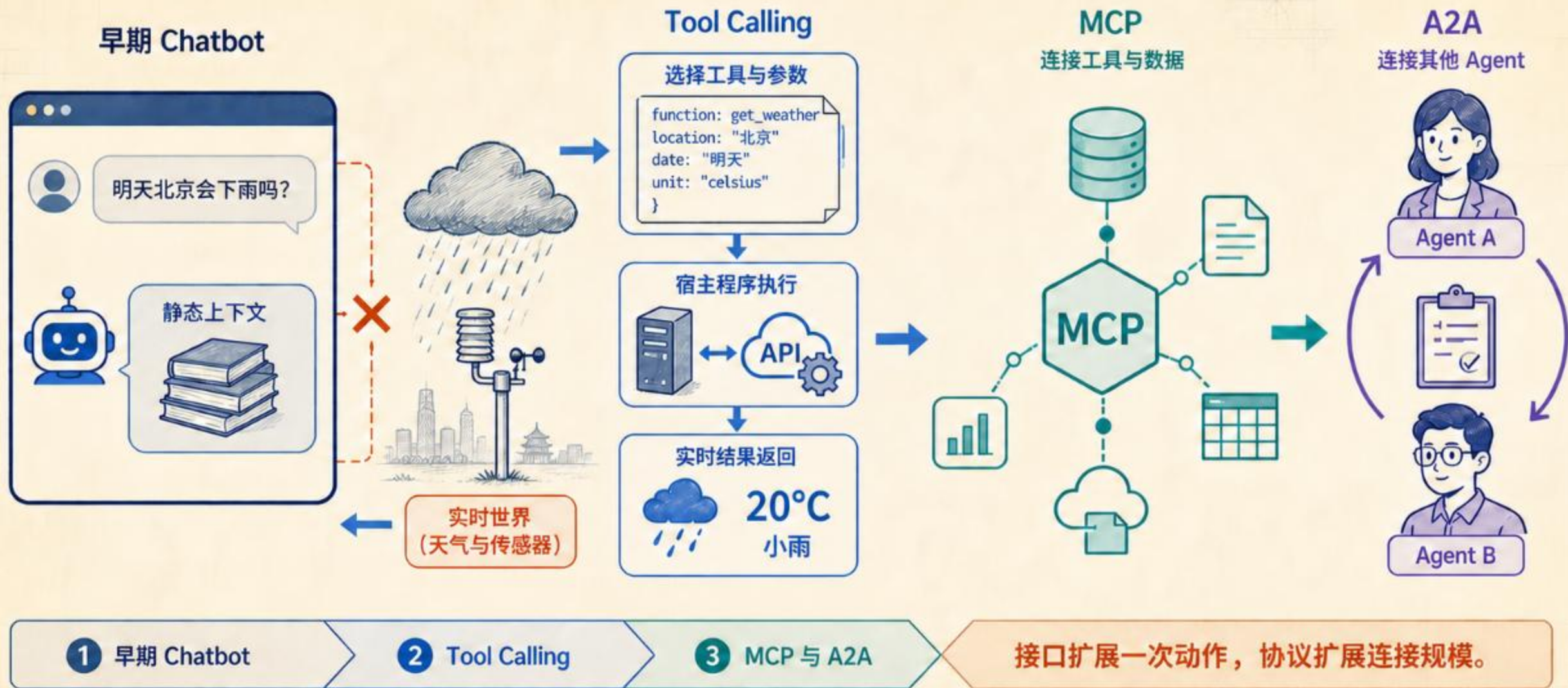
Agent 的能力是怎样“长出来”的？



模型决定能想到什么，**Harness** 决定能连续做什么，**系统反馈** 决定是否真正做成。



模型会推理，不等于它看得见世界



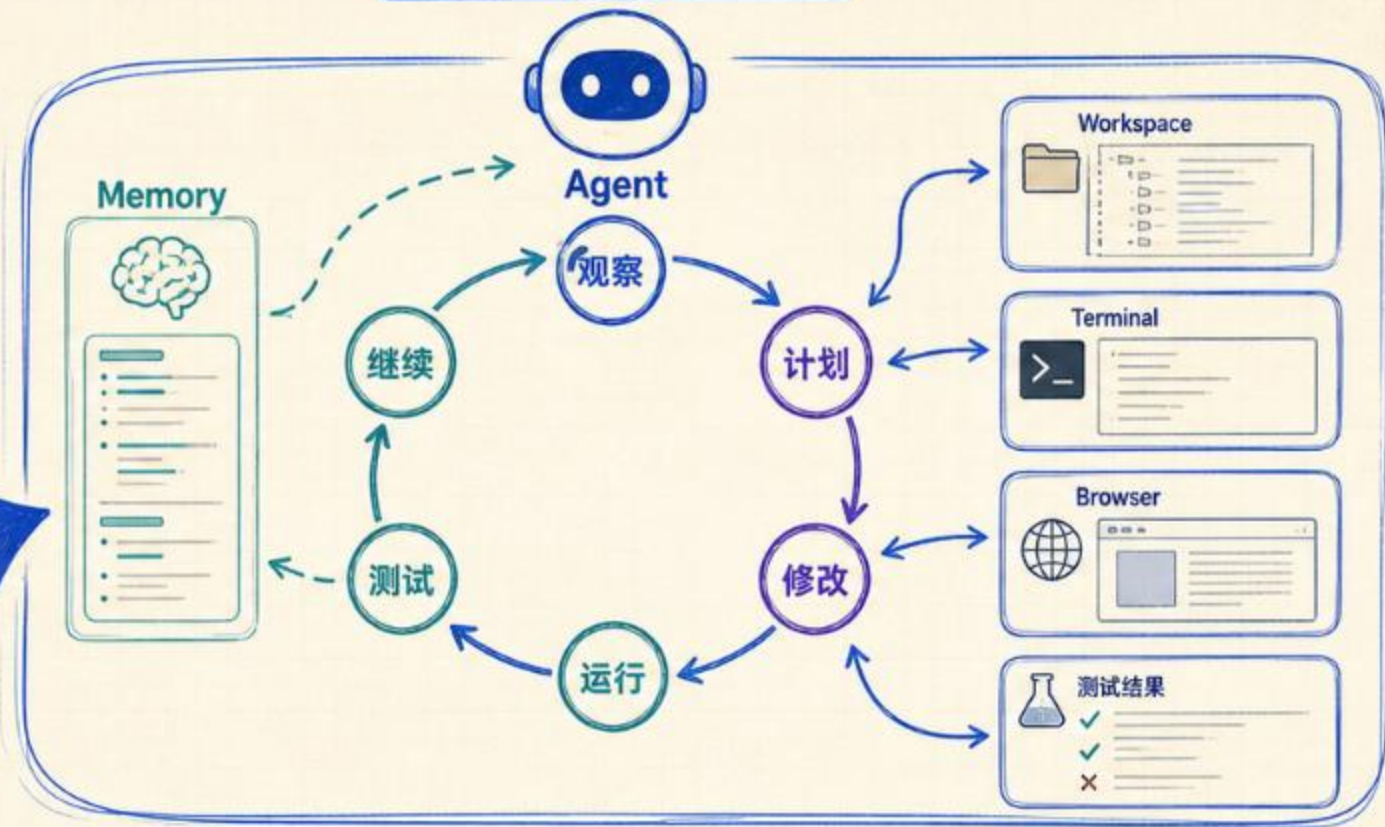
真正的跃迁：Harness 把一次调用变成持续工作

Chatbot 工作方式

人负责复制、粘贴、运行、反馈



Agent 工作方式



Cursor

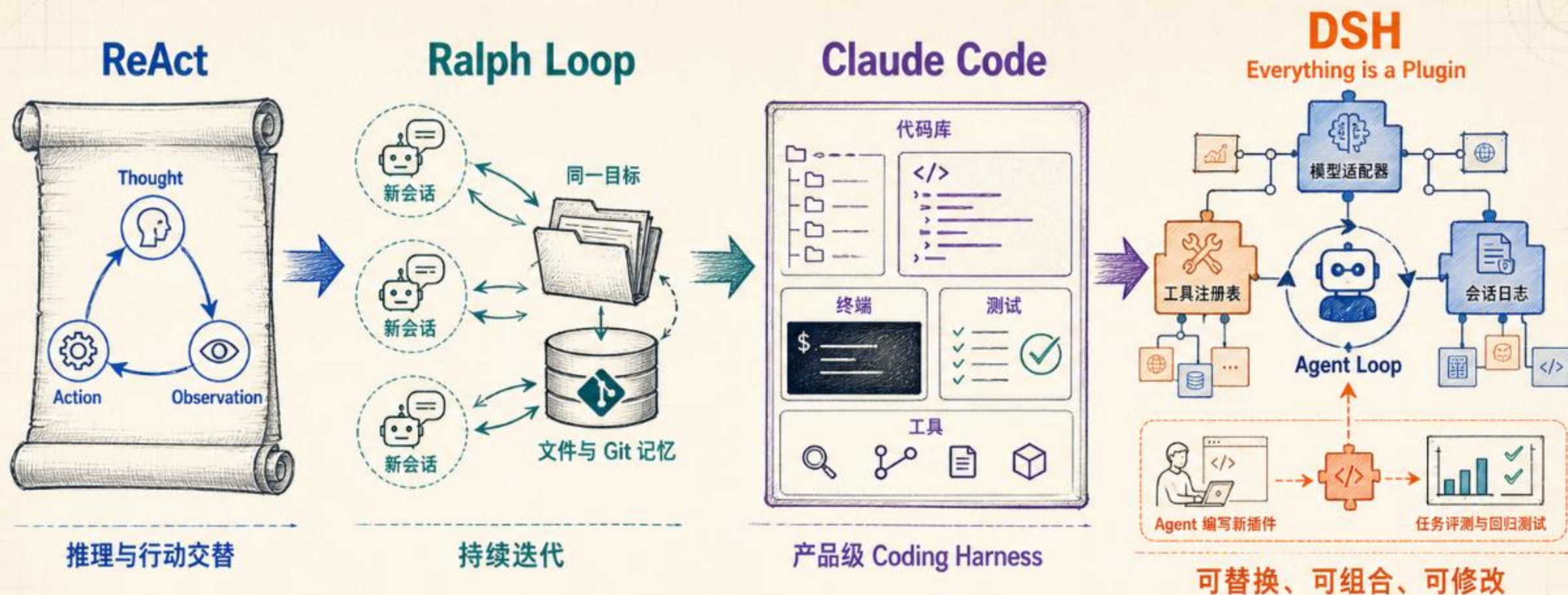
Claude Code

Manus

OpenClaw / Hermes

Model + Harness, 才是用户真正感受到的 Agent 能力。

Harness 不是一个东西：从循环模式到可重构运行系统

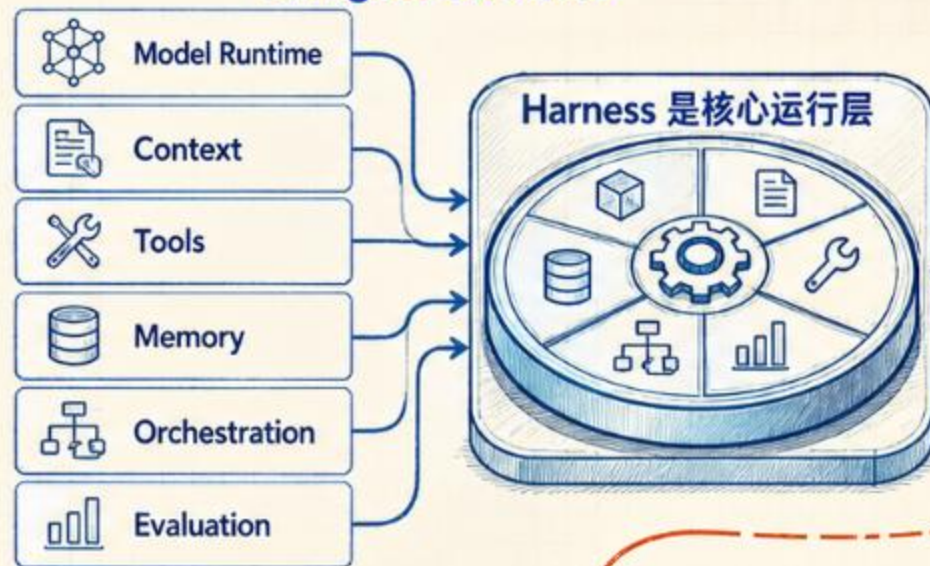


循环 → 长程 → 产品化 → Harness 成为可修改对象

只有通过任务评测与回归测试，Harness 改动才成为能力提升。

System4Agent 与 Agent4System: 方向相反, 最终形成闭环

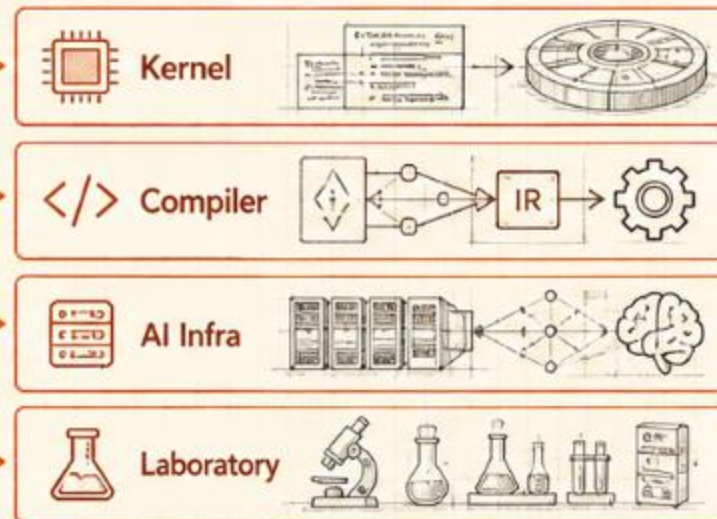
System4Agent 为 Agent 构建系统



Agent



Agent4System 用 Agent 改造系统



当 Harness 成为改造对象



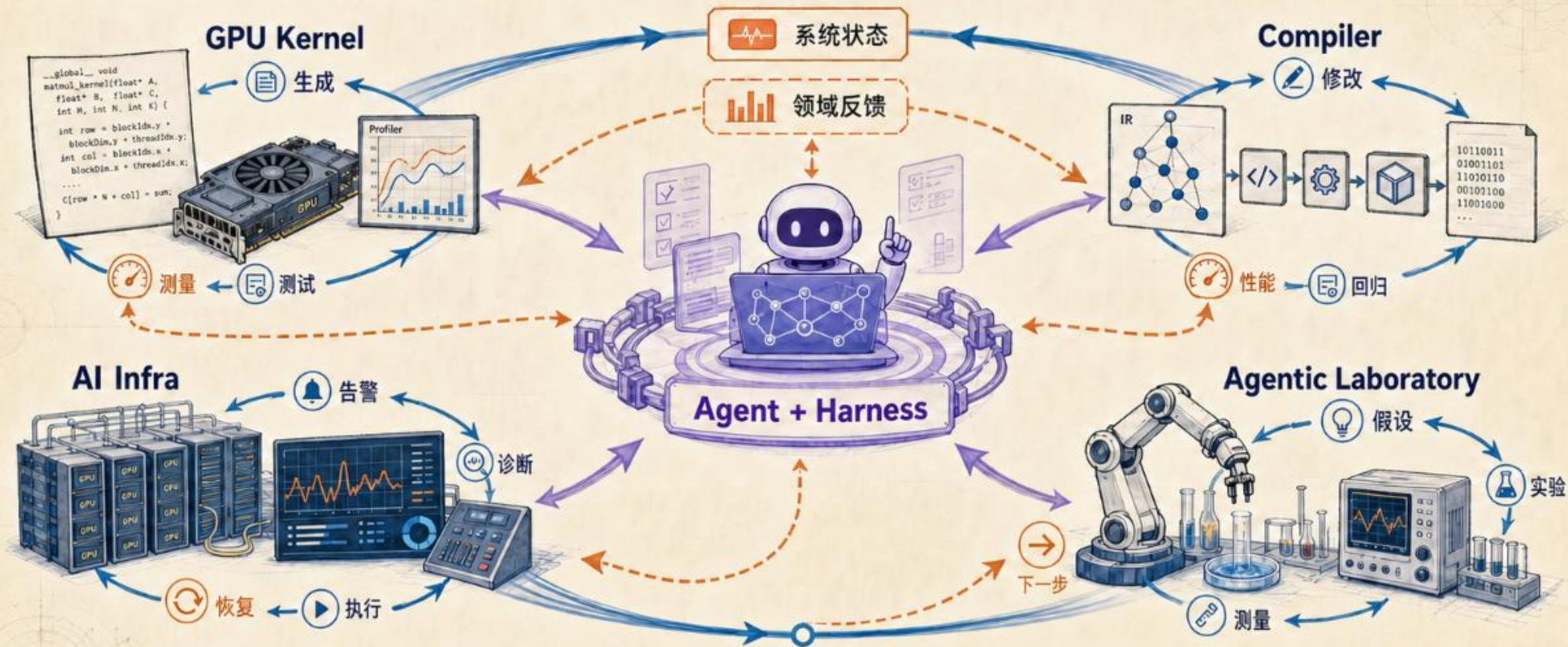
DSH / Self-Harness

System4Agent → Agent → Agent4System → 改进 System4Agent

先改运行系统, 再用任务结果判断 Agent 是否真的更强。

当系统成为被改造的对象，Agent4System 才真正出现

分界线不是题目多难，而是系统是否进入反馈闭环。



系统不再只是工具，而是 Agent 要理解、修改和验证的对象。

Agent4System 已经进入八类真实系统

目标系统不同，但都必须接入真实状态、系统动作与可测反馈。

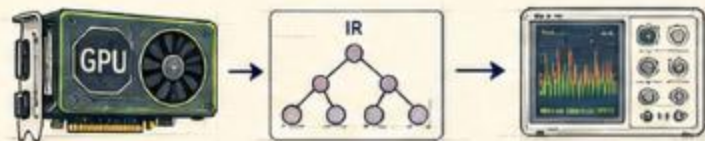
01 软件工程

Repository · Patch · Tests



02 Kernel / Compiler

CUDA · IR · Hardware Feedback



03 分布式训练

Mesh · Sharding · Throughput



04 推理 / Serving

Scheduler · KV Cache · TTFT / TPOT



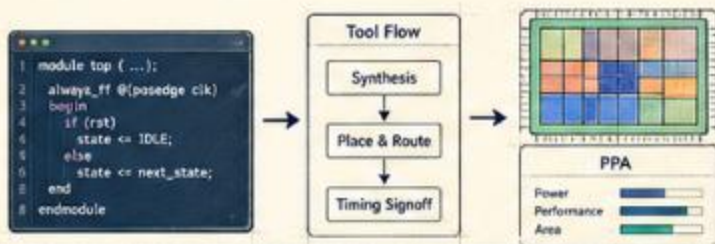
05 Cloud Operations / SRE

Telemetry · Diagnosis · Mitigation



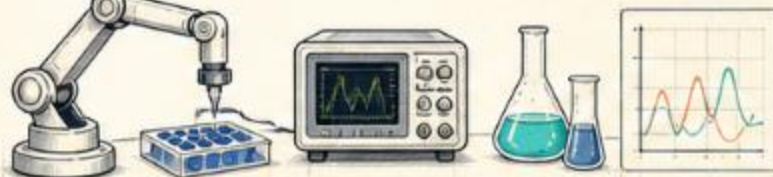
06 EDA / Chip Design

RTL · Tool Flow · PPA



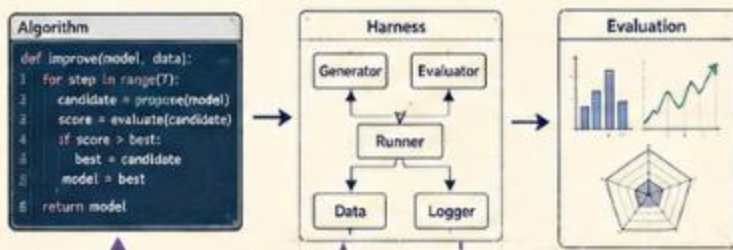
07 Agentic Laboratory

Experiment · Instrument · Measurement



08 AI4AI / RSI

Algorithm · Model · Harness · Evaluation



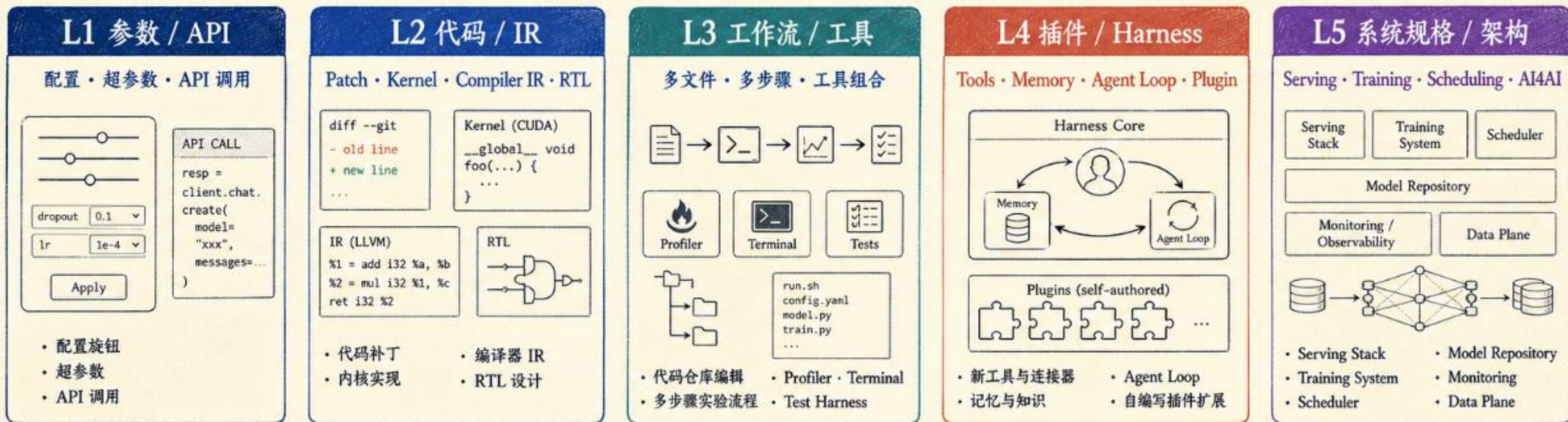
自我改进

对象不同，闭环相同。

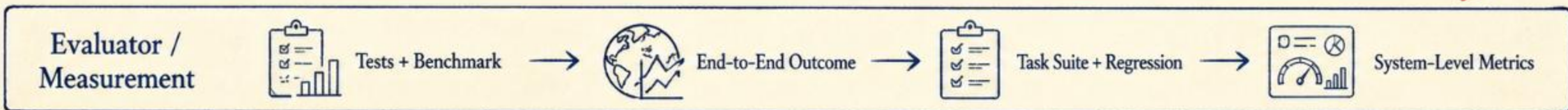
Agent 能改什么，取决于系统给它什么行动接口

L1-L5 是本课程的行动接口深度，不是通用智能等级

行动范围与潜在影响增加 →



反馈延迟、系统耦合与评测成本增加 →



模型相同，行动接口不同，Agent 能够改变的系统空间就不同。

能力前沿如何移动：时间 × 行动接口

2023



ChatEDA |
L3 工作流 / 工具



A-Lab |
L3 工作流 / 工具

2024



SWE-agent |
L2 代码 / IR

2025



KernelBench |
L2 代码 / IR



STRATUS |
L3 工作流 / 工具

2026



PithTrain |
L5 系统规格 / 架构



VibeServe |
L5 系统规格 / 架构



Prime nanoGPT |
L3-4 研究工作流 / Harness



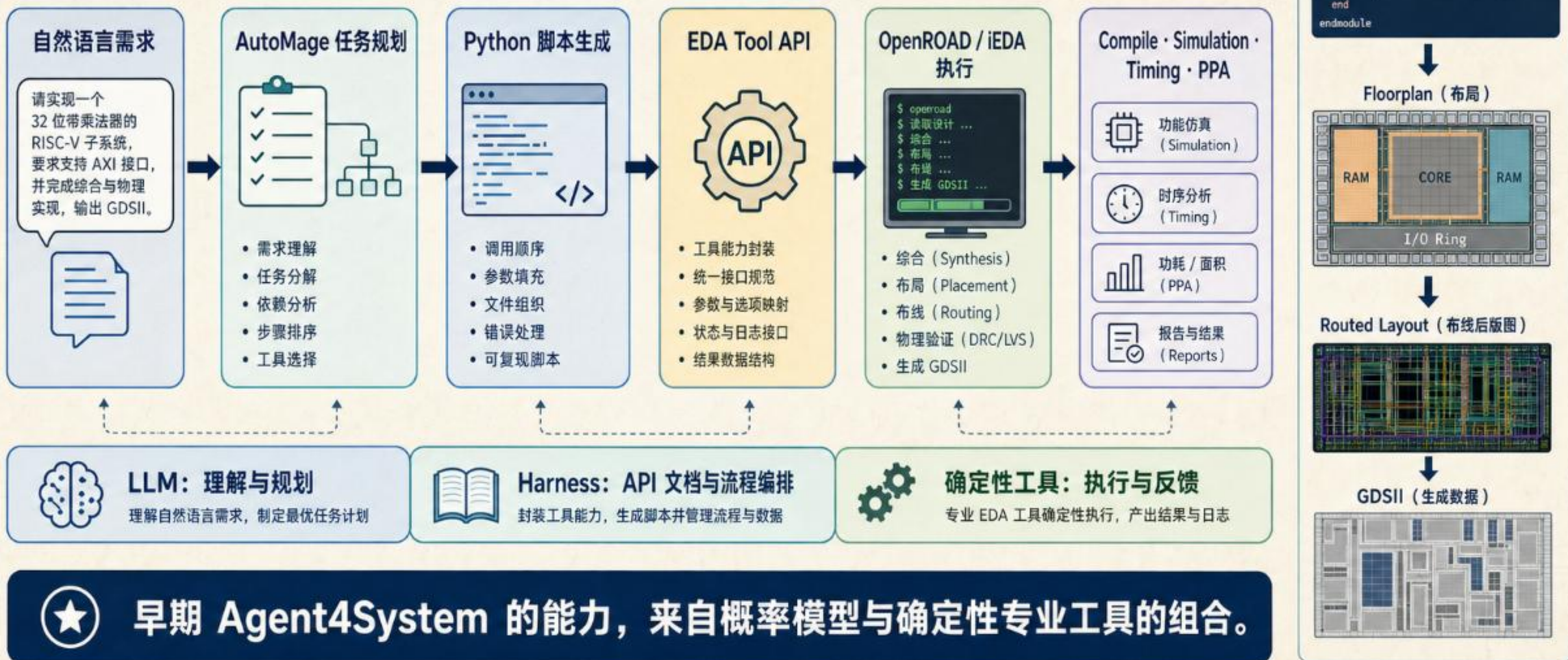
前沿扩大的是系统层级、任务跨度和自我修改范围；早期案例也可能很复杂，但通常更封闭、更专用。

2023 | ChatEDA: LLM 开始编排专业 EDA 工具链

EDA / Chip Design

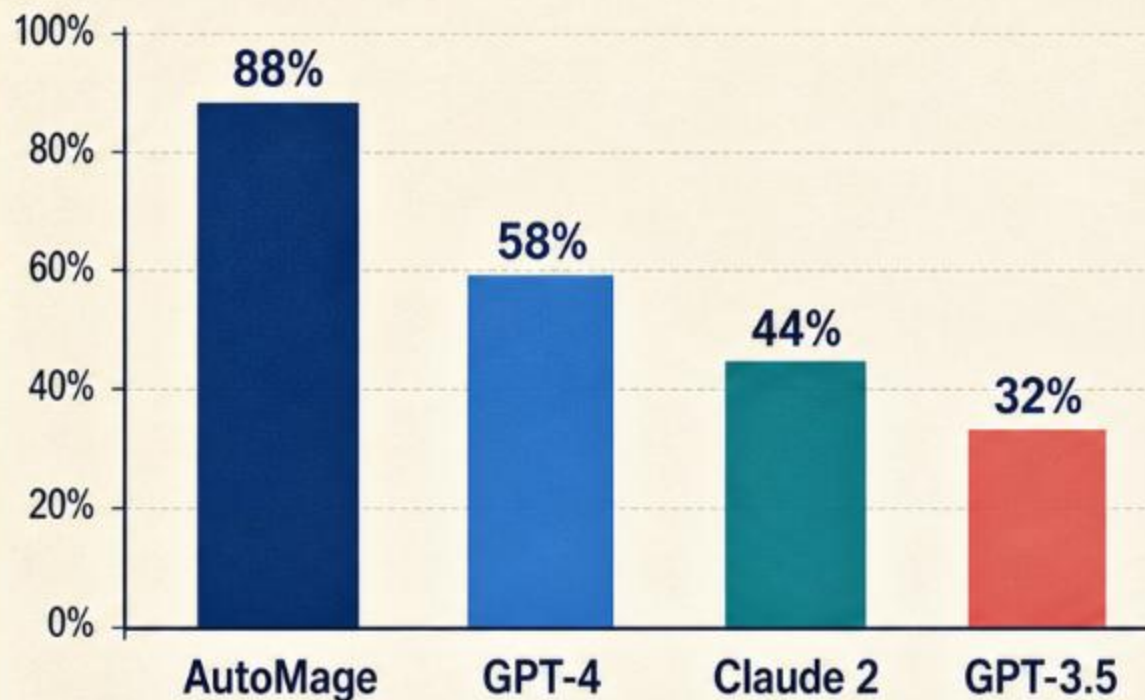
L3 工作流 / 工具

目标: RTL → GDSII



ChatEDA 的效果：同一工具链，专用 Agent 显著优于通用模型

50 个测试用例 | Grade A



端到端任务



任务规划



脚本生成

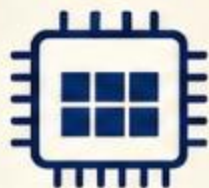


RTL → GDSII 执行

参数调优案例 | 平均 Power × Area (缩放, 越低越好)

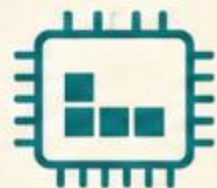
默认设置

1.31



AutoMage

1.00



关键不只是模型更懂 EDA，而是它能把计划变成可执行脚本，再由真实工具给出机器反馈。

2023 | A-Lab: 科学家、AI 与机器人进入同一个发现闭环

Agentic Laboratory

L3 工作流 / 工具

无机材料合成

计算与知识中枢

知识库

- 材料数据库
- 文献知识图谱
- 实验历史数据
- 合成与表征记录

XRD 仪器

实验状态 · 历史结果 · 不确定性 · 下一步

自动化合成平台

固体试剂库

坩埚与反应器库

高温炉

结果看板与分析



AI: 提出配方与选择下一实验



机器人: 执行真实实验



仪器 + ML: 把物理结果变成反馈

复杂自主能力并非来自单一大模型，而是来自一个被精心工程化的科学闭环。

A-Lab 的效果：17 天里持续运行一个真实材料发现闭环

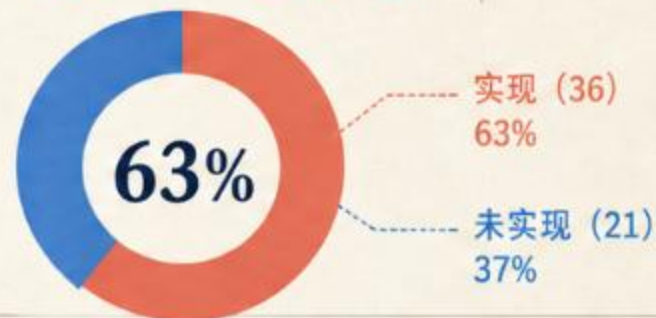
17 天
连续运行



353 次
真实合成实验



36 / 57 目标材料实现



闭环流程（持续迭代）



复杂度很高

- 计算
- 文献
- 机器人
- 表征
- 主动学习

开放度有限

- 固定材料领域
- 固定仪器
- 强人工工程

VS



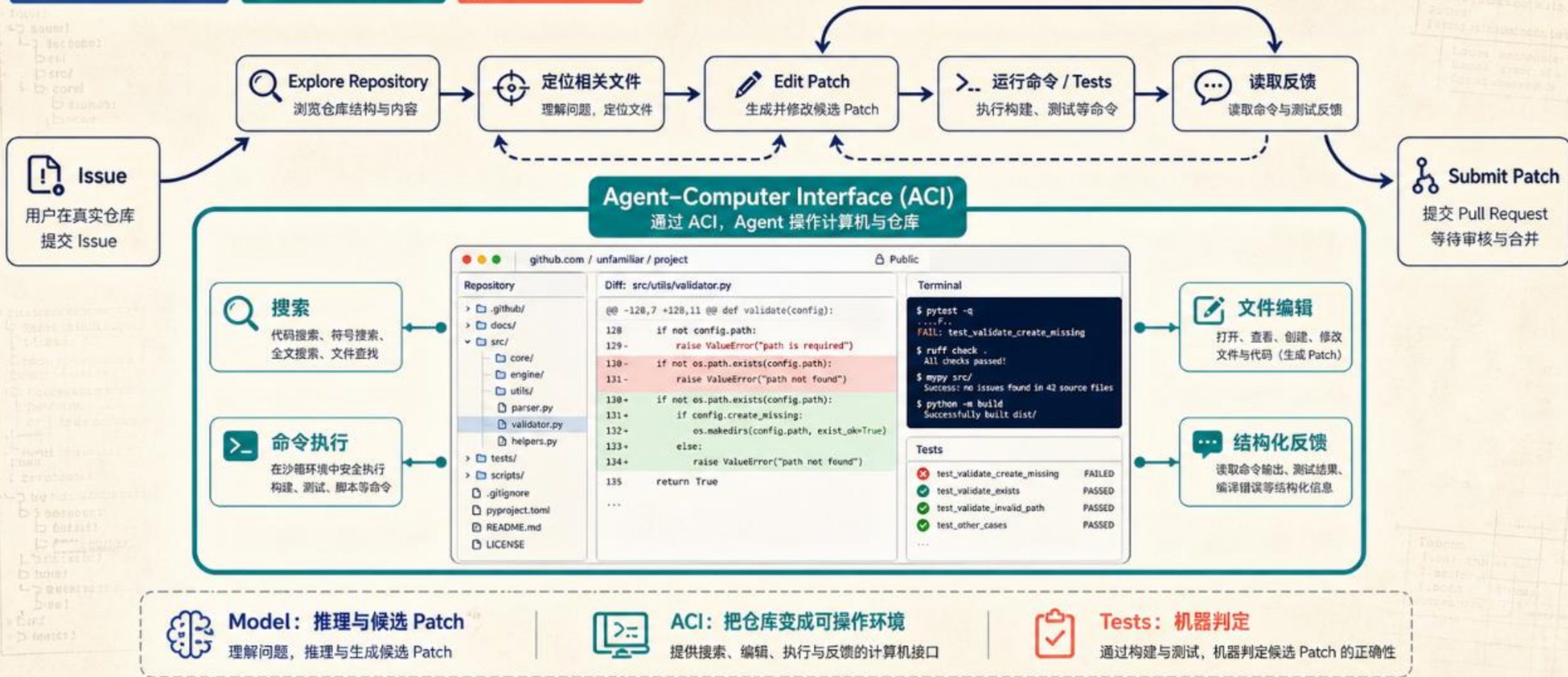
A-Lab 的关键不只是自动移液，而是让系统维护状态、解释证据，并据此选择下一步实验。

2024 | SWE-agent: 让通用模型进入开放代码仓库

</> Software Engineering

L2 代码 / Repository

真实 GitHub Issue



★ 同一个模型, 换一种面向 Agent 的计算机接口, 真实任务成功率就会改变。

SWE-agent 的效果：接口设计本身就是能力变量

SWE-bench Lite | Resolved

约 11%



约 18%

SWE-agent ACI

同一类模型，
更好的 ACI



范围说明：真实仓库更开放，
但绝大多数 Issue 仍未被解决。

证据：全量基准 (Full SWE-bench)

286 / 2294

12.47% pass@1

SWE-agent 的 ACI 组成 (示例)

搜索



文件编辑



命令执行



结构化反馈



```
@@ -128,7 +128,7 @@  
- return false;  
+ return true;
```

运行测试



从聊天到工作，不只是多一个工具按钮，而是重新设计模型观察、行动和接收反馈的方式。

2025 | KernelBench: Agent 开始搜索“正确而且更快”的 GPU Kernel

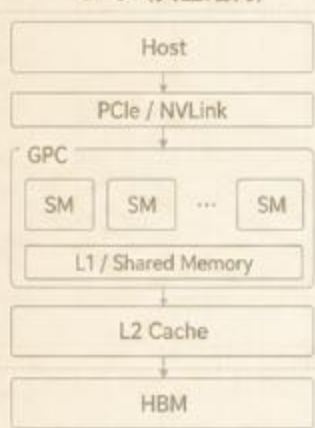
</> Kernel / Compiler

IR L2 代码 / IR

250 个 PyTorch Workloads

在真实 GPU 上的执行轨迹对比 (示例)

GPU (典型结构)



性能度量 (延迟)



1 PyTorch 参考实现



7 Refine or Select

- 生成更优变体
- 或选择当前最优
- 更新上下文与提示

6 Latency / Profile

- 延迟 (端到端)
- 内核耗时分解
- 访存 / 带宽 / 占用率等

机器可搜索的系统闭环

2 LLM 生成
CUDA / Triton Kernel



3 Compile

- NVCC / Clang
- Triton Compiler

4 Correctness Tests

- 数值一致性
- 形状 / 类型检查
- 边界与异常

5 GPU Runtime

- 真实设备执行
- 事件计时

Baseline
(PyTorch 原生 Kernel)



Kernel

SM Activity

Memory

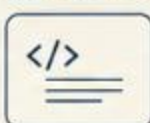
L2 Hit Rate

Occupancy

Time →



Candidate
(LLM 生成 Kernel)



Kernel

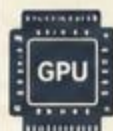
SM Activity

Memory

L2 Hit Rate

Occupancy

Time →



Correctness: 输出必须匹配



数值一致性
(多输入、多形状)



容差检查
(绝对 / 相对误差)



边界 / 异常 / NaN
行为一致性



通过



Performance: 真实 GPU
延迟必须更低



真实 GPU 延迟
(端到端)



统计显著性
(多次运行)



稳定性检查
(方差 / 抖动)



通过



Selector

仅保留同时通过
两条判定轨道的候选

Selected Kernel(s)



系统优化第一次被改写成大规模、可执行、可自动判定的搜索问题。

KernelBench 的结果：会写代码，还远远不等于会优化系统

```
__global__ void kernel(  
const float* __restrict__ x,  
float* __restrict__ y,  
int N) {  
    for (int i = blockIdx.x * blockDim.x + threadIdx.x;  
        i < N; i += blockDim.x * gridDim.x) {  
        y[i] = x[i] * x[i];  
    }  
}
```

250

PyTorch
ML Workloads

```
// grid-stride loop  
for (int i = blockIdx.x * blockDim.x + threadIdx.x;  
    i < N; i += blockDim.x * gridDim.x) {  
    y[i] = x[i] * x[i];  
}
```



fast_p = [**正确** (通过功能校验) $\wedge$ **性能达到阈值** (不慢于 PyTorch baseline)]

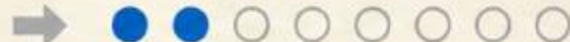
机器执行把性能工程从“写答案”变成“跑实验”。

策略

被接受的候选 (示意)



只生成



仅靠一次生成，能通过筛选的候选很少



+ 执行反馈



利用执行结果 (通过/失败、错误等)，能筛选出更多可行候选



+ Profiling 反馈



结合 Profiling 指标 (瓶颈、热点等)，引导更有针对性的迭代与探索



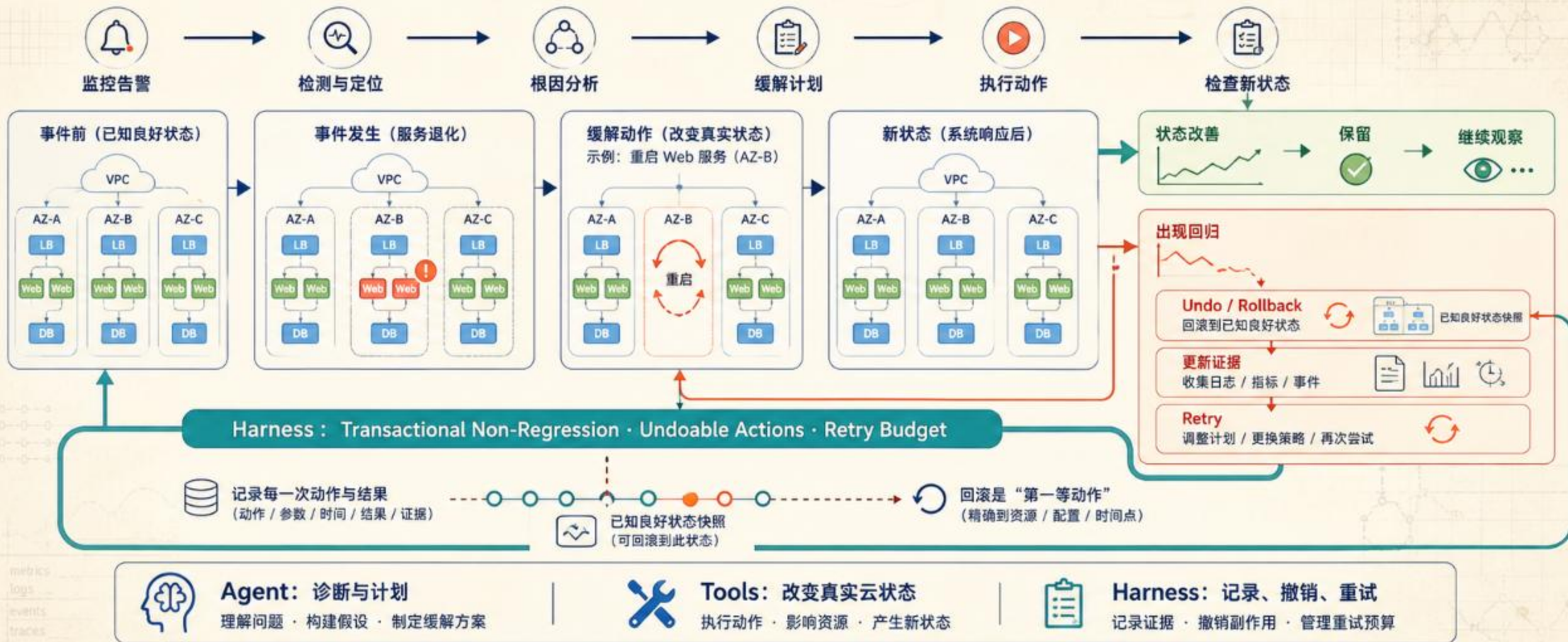
真正的突破不是一次生成写对，而是让 Agent 能低成本地提出、执行、淘汰并改进大量候选。

2025 | STRATUS: Agent 进入会被动作改变的在线系统

Cloud Operations / SRE

L3 工作流 / 工具

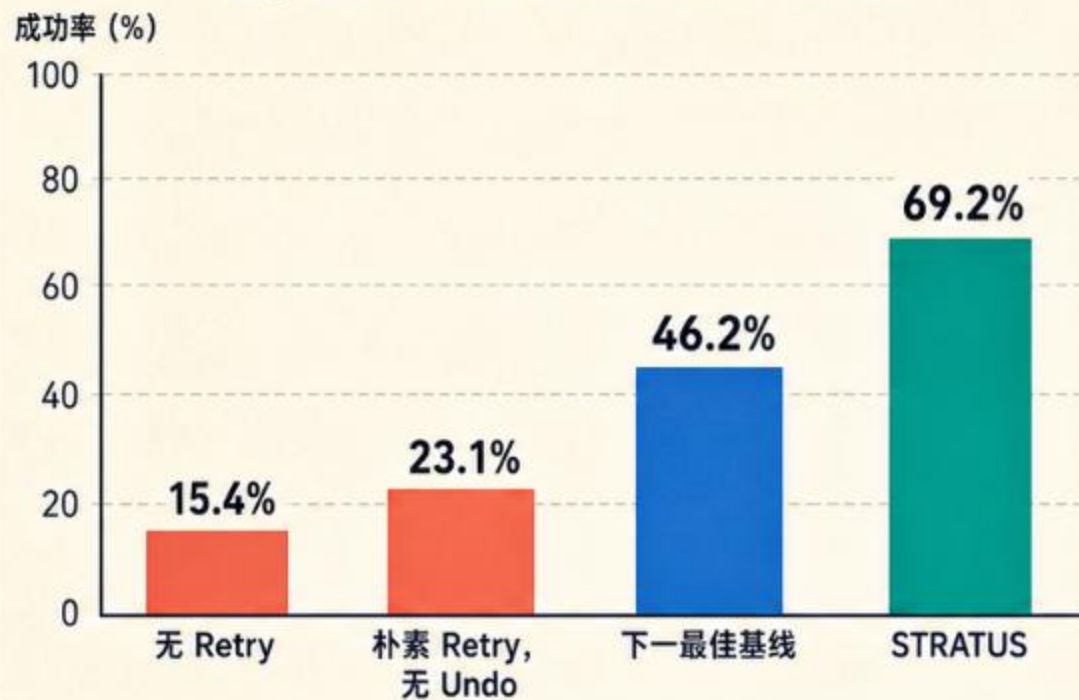
AI Ops Lab · ITBench



在有副作用的系统里，恢复不是失败后的附属功能，而是探索能力的一部分。

STRATUS 的效果：真正拉开差距的是 Undo-and-Retry

AIOpsLab | 13 个缓解问题 | 成功率



$69.2\% - 46.2\% = +23.0$ 个百分点

为什么 Undo-and-Retry 构成有效的因果闭环



为什么朴素 Retry 会不断叠加状态变化

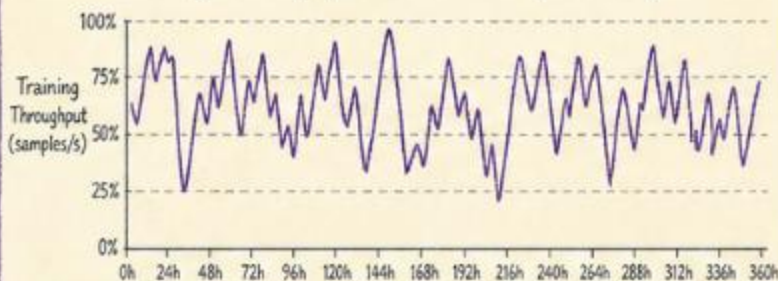


Retry 本身不是能力；可撤销动作、状态检查与证据更新共同组成了可持续探索。

2025 | 无问芯穹：运维 Agent 进入真实 GPU 集群

幽灵故障

training throughput oscillates unpredictably



7 名资深工程师

15 天

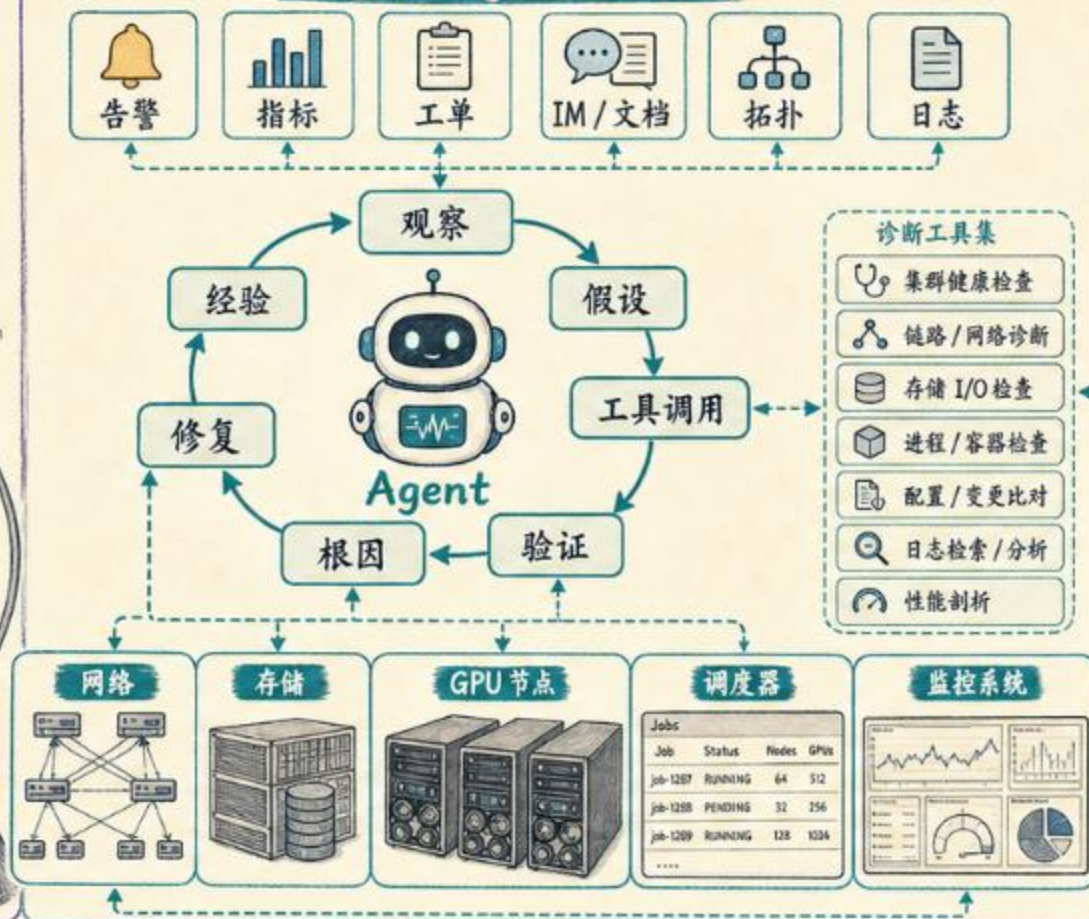
105 人天

256 台服务器闲置

存储缓冲机制偏差



运维 Agent 闭环



上线后的业务结果



工单平均处理时长
-50%



关键故障处理
≈ 6x



人效
> 5x



综合运维成本
≈ -30%

L3 工作流 / 工具

好用的运维 Agent，不是回答得漂亮，而是能在真实集群里把排障闭环跑通。

AISHPerf: 把真实运维经验压缩成可执行评测

真实生产数据

— 近百亿条真实运维记录 —



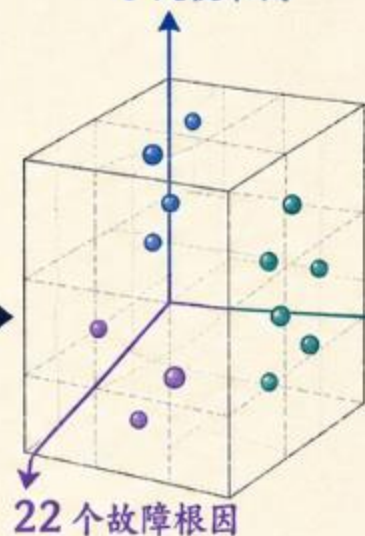
清洗·去重·
脱敏·专家复核

十万条
有效数据

103个
高保真评测案例

评测空间

5大技术栈



可执行 Episode

1



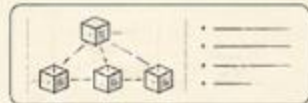
问题现象



2



探索



3



工具调用



4



故障模拟



5



根因定位



6



修复方案



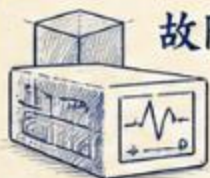
7



轨迹评分



发布组件 (开源中立制品)



故障模拟器

- 可注入多类故障
- 可控、可复现
- 支持复杂场景编排



运维数据集

- 十万条有效数据
- 高质量标注与结构化
- 隐私安全处理



评测框架

- Episode 定义与执行
- 工具与环境集成
- 轨迹采集与评分

评测不再问模型说得再好，而是问它能不能把事情做成。

2026 | PithTrain: 不只升级 Agent, 也重新设计它要修改的系统

 Distributed Training

 L5 为 Agent 设计系统

 约 11K 行 Python

这是一个人类设计的
System4Agent
而非让 Agent 自行重构系统

传统训练框架

-  大量隐式间接层
调用链长, 难以追踪
-  重度依赖原生扩展
C++/CUDA 扩展分散在多处
-  错误位置远离
从报错到编辑处隔层远, 定位成本高



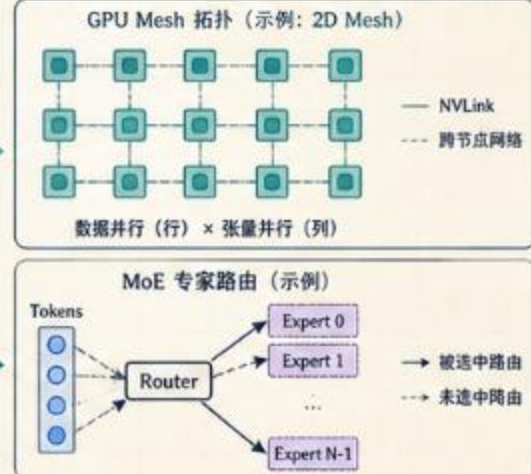
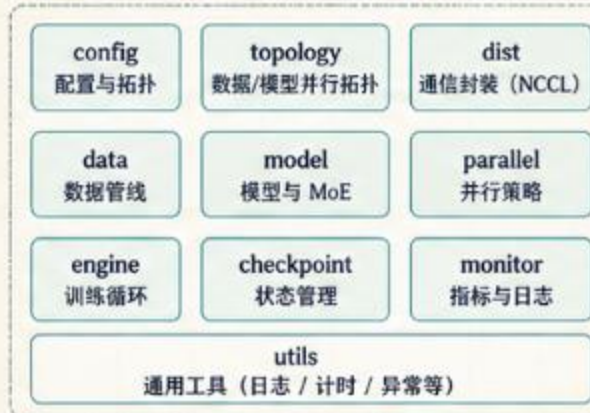
报错位置 (远离编辑处)

隐式调用链
(示意)

VS.

PithTrain

显式的数据 / 并行拓扑, 少而直接的依赖, 易读易改



✓ 错误靠近编辑位置, Traceback 直达源码行

Traceback (most recent call last):
.../pithtrain/engine/train_loop.py:217 in step
217 loss = self.model(batch)

Agent 工作循环



为 Agent 设计的 三大原则



少隐式间接层

直线型依赖与显式接口, 减少心智负担,
让 Agent 更快读懂与修改。



错误靠近编辑位置

Traceback 直达源码行, 日志与断言在身边,
调试成本显著降低。



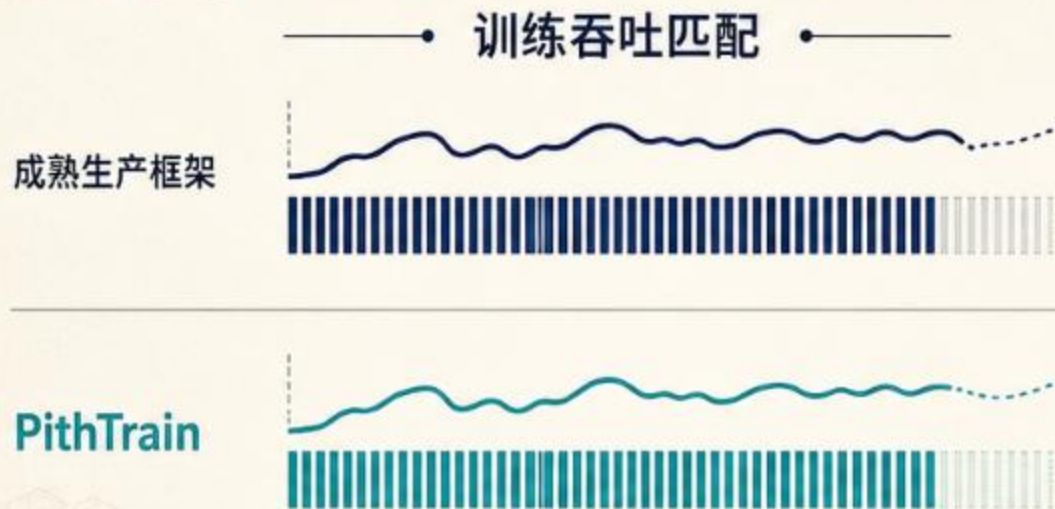
任务专用 Skills

围绕分布式训练的高频任务沉淀可复用能力,
提升实现效率与正确率。

当 Agent 成为开发者, 系统是否容易被理解和修改, 本身就是性能指标。

PithTrain 的效果：系统吞吐不降，Agent 完成任务的成本更低

系统侧



Agent 侧 | ATE-Bench | 最多减少

Agent Turns ↓

62% 最多减少

Before (成熟生产框架)



After (PithTrain)



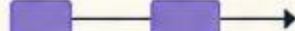
Active GPU Time ↓

64% 最多减少

Before (成熟生产框架)



After (PithTrain)



约 **11K** 行 Python



局部错误反馈



任务专用 Skills



这是人类为 Agent 设计的 System4Agent，不是 Agent 已经自主重写训练系统。



面向 Agent 的系统指标，不只看每秒训练多少 Token，也看完成一次修改需要多少交互与 GPU 时间。

2026 | VibeServe: Agent 从优化单点, 走向生成完整 Serving Stack

Inference / Serving

L5 系统规格 / 架构

Generation-time Specialization

冻结合同
(Frozen Contract)



Model



Hardware



Workload



Correctness



Performance



Serving DAG



Outer Loop (设计问题 · 长期记忆 · Checkpoint)

Inner Loop (快速迭代 · 局部记忆)

Implementer
(实现代理)



生成与修改系统组件
与实现方案

Accuracy Judge
(正确性检查代理)



独立于性能的正确性检查
(功能与数值一致性)

Performance Evaluator
(性能评估代理)



独立于正确性的性能测量
(延迟 / 吞吐 / 资源)

反馈 (改进建议 / 诊断)

生成的 Serving Stack (系统组件)



Scheduler
(调度器)



Batching
(批处理)



KV Cache
(缓存管理)



Kernels
(计算内核)



Memory
Management
(内存管理)



Runtime
(运行时)

专用 Serving Runtime
(最终输出)



GPU
(执行资源)



请求队列
(Request Queue)

长期记忆
(Long-term Memory)

GPU Memory Layout (示意)



冻结合同
(固定输入前提)



规划系统
(搜索与设计)



实现
(生成与集成)



正确性检查
(独立验证)



性能测量
(独立评估)



更新设计
(回到 Outer Loop)

Agent 不再只改一个 Kernel, 而是在固定场景合同下搜索整个系统设计空间。

VibeServe 的效果: 覆盖通用基线, 更在非标准组合上拉开差距

主流场景

Llama-3.1-8B · H100

VibeServe

vLLM

SGLang

保持竞争力

6 个非标准 Model × Hardware × Workload 场景

$1.69\times - 6.27\times$

场景 A
长上下文

场景 B
高吞吐

场景 C
突发流量

场景 D
长短混合

场景 E
多用户并发

场景 F
低延迟交互

生成的 Serving Stack (端到端联调后的产物)

Scheduler

队列

调度

KV Cache

分块

复用

Kernels

算子融合

多版本候选

Memory

分配计划

布局优化

Runtime

执行图

重叠与流水

固定合同后, 专用 Runtime 可以跨层联合搜索。

软件开始从“一套通用系统服务所有场景”, 转向按 Model × Hardware × Workload 生成专用 Artifact。

2026 | Prime nanoGPT : Agent 开始持续运行多日科研 Campaign

AI4AI / Autonomous Research

L3-4 研究工作流 / Harness

固定 124M nanoGPT

RESEARCH CAMPAIGN

- ☒ state
- ☒ evidence
- ☒ budget
- ☒ next step

模型 / 数据固定

目标 loss 固定

允许修改: Optimizer · Schedule · Initialization · 部分超参数

Agent 研究员
(自主循环)

读取人类
前沿与历史

提出实验假设

修改训练代码

提交 GPU Run

分析曲线与日志

更新 Scratchpad /
长期记忆

选择下一实验

Harness
(编排层)

实验队列

GPU Runner

Run Logs

统计噪声门

前沿知识板 (长期参考)

Scratchpad / 长期记忆 (持久化)

训练系统
(执行层)

固定模型与数据
(124M nanoGPT)

应用代码改动
与超参配置

分布式训练执行
(8 × GPU)

保存日志 / 权重 / 指标
供上层分析



多日自主科研的关键，不只是会写实验代码，而是能维护状态、证据、预算和下一步。

Prime nanoGPT 的效果：两周、万次实验，把人类前沿推进了 60 steps



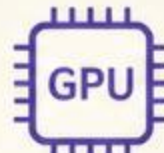
约 2 周

| 持续 Campaign



约 10,000 次

| Training Runs



约 14,000

H200 GPU 小时

EXPERIMENT LOG

RUN ID	CONFIG	VAL LOSS	STEPS
9921	A1-73	0.0321	2510
9922	A1-79	0.0298	2470
9923	A1-75	0.0276	2445
...	...	...	...
9976	C3-42	0.0152	2975
9977	C3-43	0.0148	2955
9978	C3-44	0.0140	2930
9979	C3-45	0.0141	2940
9980	C3-46	0.0143	2948
...	...	...	...

Lower is better (更低更好)

人类前沿

2990 steps



Agent 最佳

2930 steps

-60 steps | 约 2.0%



已经实现

- 1 长期实验状态
- 2 大规模候选搜索
- 3 组合已有方法并推进指标



仍未等于完整 RSI

- 1 依赖人类前沿记录
- 2 尚未证明能持续改进“产生改进的能力”

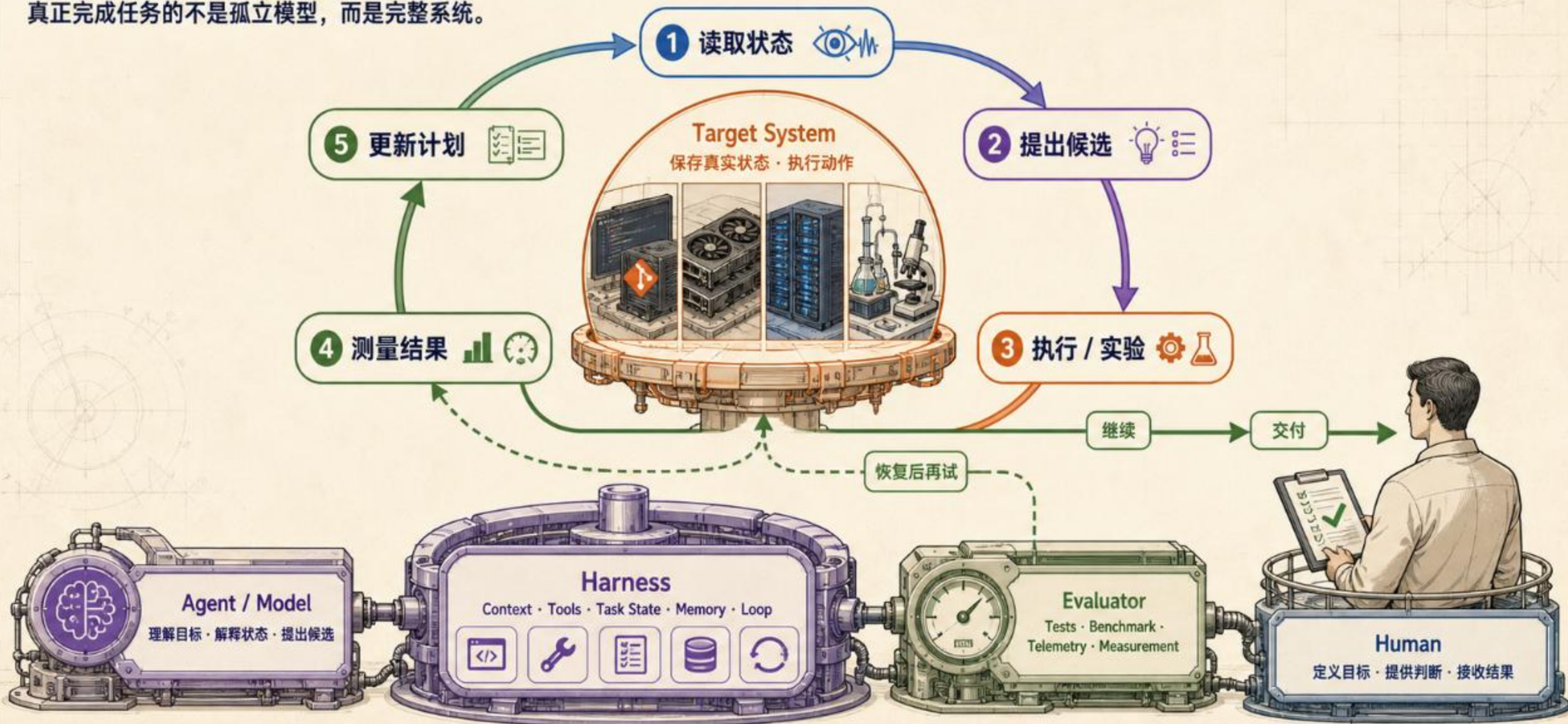


这是 AI4AI 的重要一步：Agent 已能持续改进 AI 研发对象，但还不是无限递归的自我提升。

八个方向，都在运行同一个状态—行动—反馈闭环

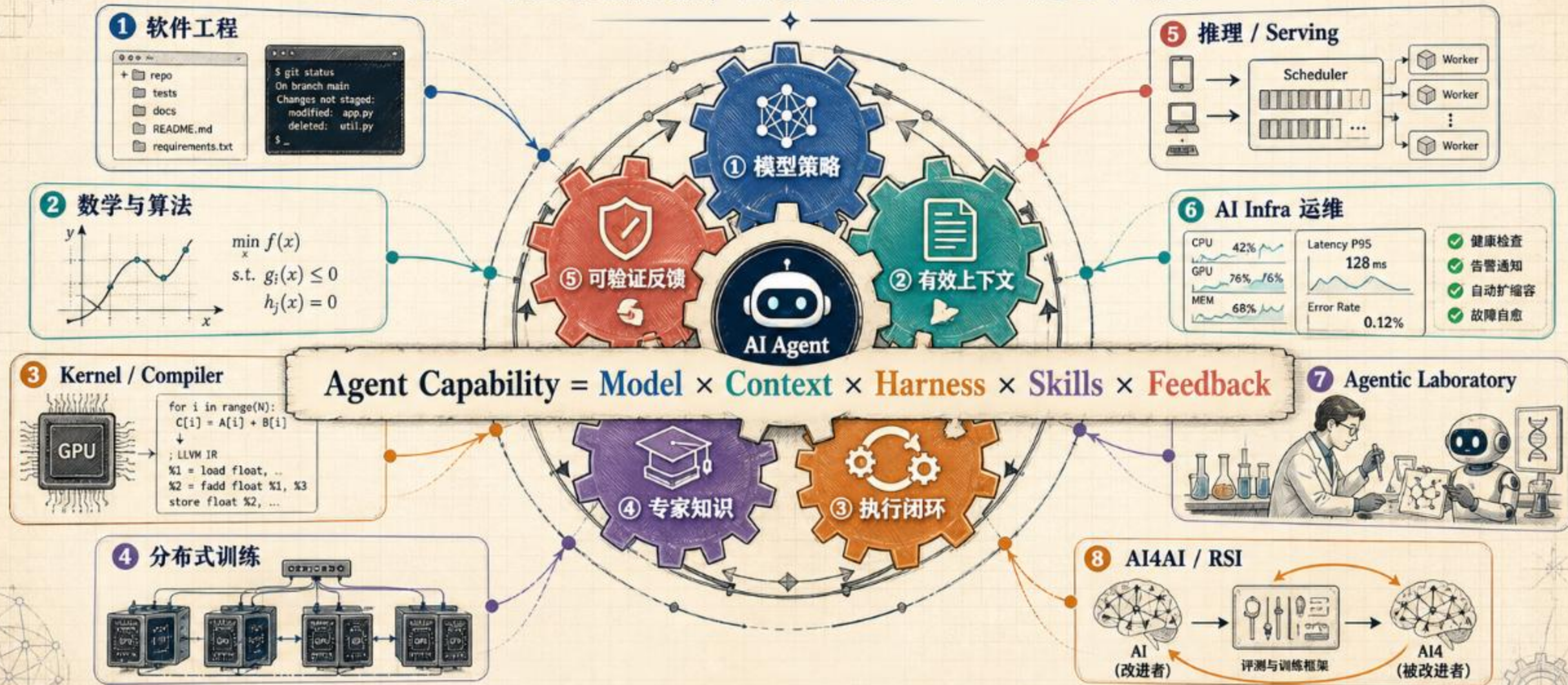
Agent4System = Agent Loop × Stateful Target × Measurable Feedback

真正完成任务的不是孤立模型，而是完整系统。



为什么这些方向几乎同时开始突破？

不是某一个应用突然成熟，而是五个能力环节同时越过了阈值。



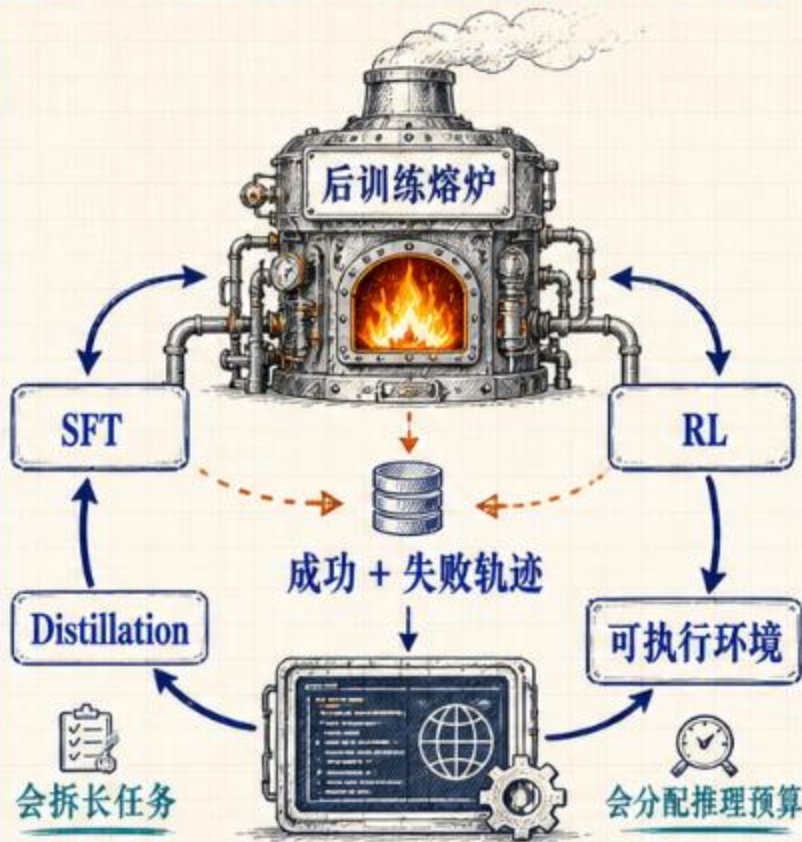
Agent 的突破不是单点突破，而是 Model 与 System Stack 共同跨过阈值。

模型没有明显变大，为什么 Agent 能力仍然大幅提升？

后训练正在把工具使用、长程决策和纠错策略内化进模型。

DeepSeek-V4-Flash | 结构与规模相同，仅重新后训练

● Preview ● 0731



GLM-5.2 → GLM-5.3 | 同一 Base Model, 增量来自 Post-training

● GLM-5.2 ● GLM-5.3



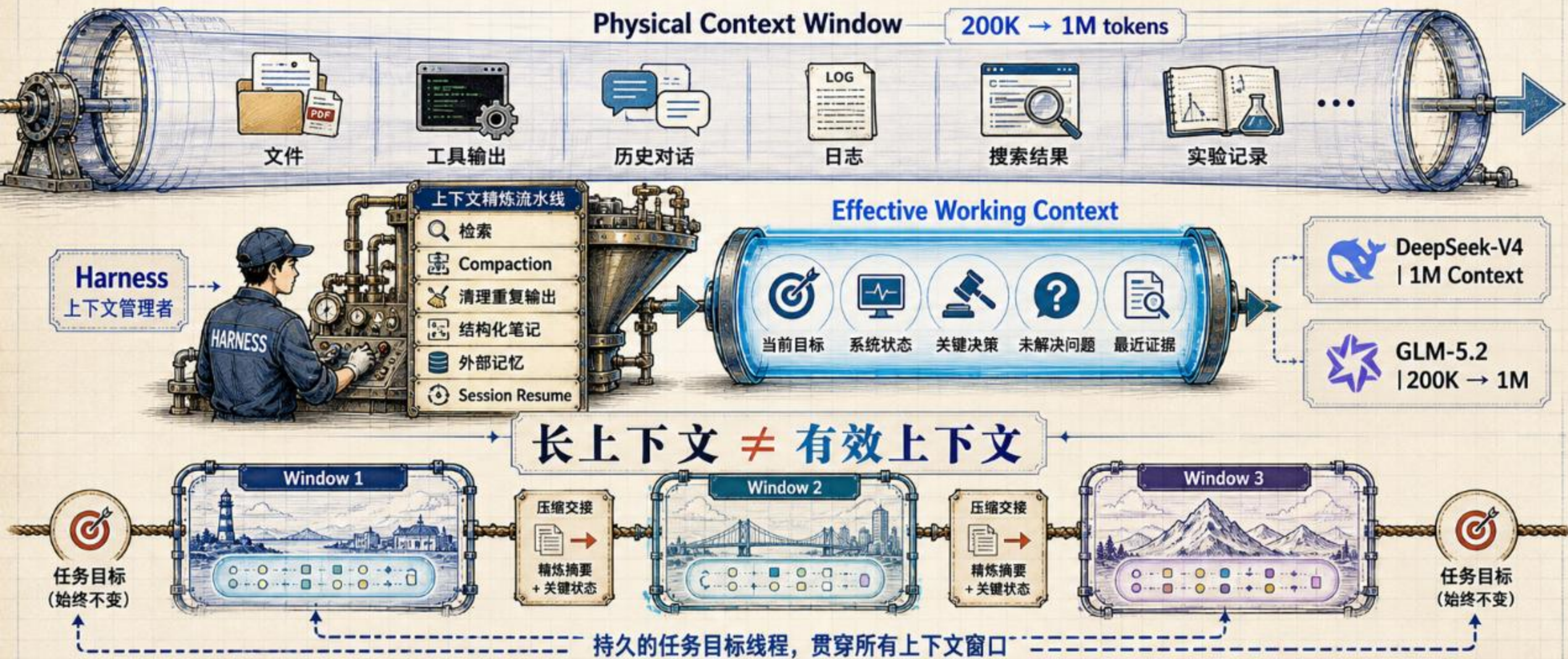
① 同一 Benchmark 内比较；不同 Harness / 预算不可横向拼接。



参数规模决定潜在容量；后训练决定这些容量能否组织成有效的行动策略。

上下文越长，Agent 就一定能完成越长的任务吗？

窗口长度提供容量，Context Engineering 提供连续性。



上下文长度决定能装下多少信息；**Harness** 决定有多少信息仍然对当前决策有效。

反馈接口既放大 Agent 能力，也改变我们在测什么

Anthropic VLIW Take-home: 从高信号招聘题，到被 Claude 逐代攻破



Cycle 越低越好



模型没有改变，Harness 改进后仍能继续逼近更优解。

同一个反馈接口，在两类系统中承担不同角色

生产系统追求把事情做好；评测系统追求分离能力来源。

生产系统 | 放大 Agent 能力

目标：更快、更稳定地完成真实任务



更短反馈



毫秒级响应
快速迭代循环

更强工具



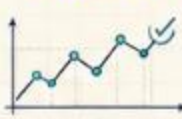
更多能力接入
提升执行上限

持久状态



记住上下文
跨会话累积价值

自动回归



持续验证行为
防止性能退化

接口越快、反馈越清楚，Agent 越能搜索复杂方案。

评测系统 | 保留区分信号

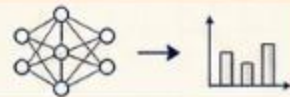
目标：知道能力来自模型、Harness，还是搜索预算



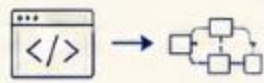
评测者



模型能力



Harness 能力



测试时计算

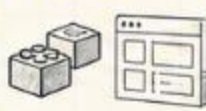


冻结工具



固定可用工具集
保证公平可比

控制 Scaffold



统一框架与提示
降低实现差异

限制搜索预算



限定步数/ token
控制计算资源

隐藏测试与变体



隐藏评测细节
防止过拟合

若把最强反馈全部开放，评测会被迅速攻破并失去区分度。

$$\text{能力} = \text{Model} \times \text{Harness} \times \text{Interface} \times \text{Test-time Compute}$$

VLIW 的启示：生产环境应主动放大能力；能力评测必须明确自己到底在测什么。

模型不会的专业经验，是否必须重新训练进权重？

Skill 提供了一条更快、更便宜、也更容易更新的路径。

Kernel Design Agents — GPU-Kernel 优化工作坊

1 Observation Skill | 告诉 Agent 怎么看

- NCU Metrics
- Stall Hotspot
- PM Sampling Timeline

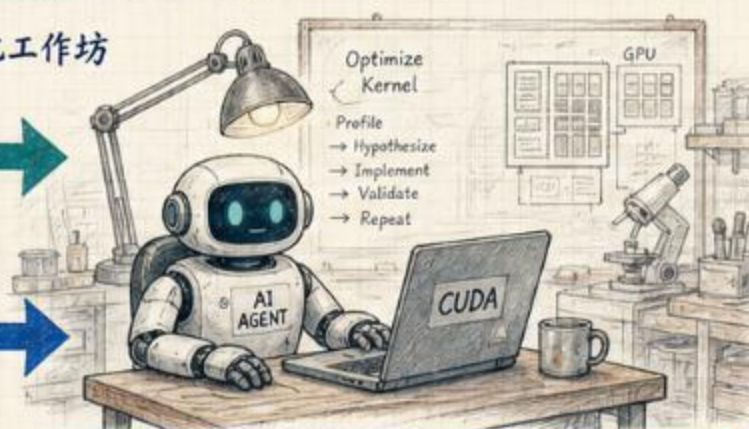


2 Domain Knowledge Skill | 告诉 Agent 尝试什么

- KernelWiki
- Blackwell / Hopper
- PR 与已知优化经验



3 Workflow Skill | 告诉 Agent 怎么形成闭环



迭代式 CUDA 优化 workflow



SkillsBench | 成对实验



86 tasks



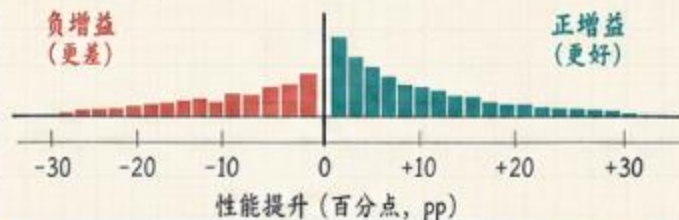
11 domains



7,308 trajectories

Curated Skills
+16.2 pp

16 / 84 tasks 出现负增益



Focused 2-3 modules > 全量文档

Self-generated Skills: 平均无增益

💡 Skill 不是让模型临时背更多文档，而是把专家的观察方法、工具和工作程序变成可调用能力。

为什么旧模型需要的 Skill，新模型用了反而可能变差？

模型在进化，Skill 也必须被版本化、评估和淘汰。

较弱模型

v1.0
过去



- 提供分解方法
- 避免常见失败
- 选择正确工具

更强模型

v2.0
现在



- 重复原生能力
- 锚定旧策略
- 占用上下文
- 版本冲突

薄 Skill，强工具，硬反馈。

保留目标、约束、专业工具与完成证据；
删除模型已经原生掌握的步骤教学。

SWE-Skills-Bench | 49 Skills



39 / 49 通过率无提升

7 个专业 Skill
有明显收益

3 个 Skill
造成下降

最高 +30%

Token 最高 +451%，通过率不变

最低 -10%

平均增益: +1.2%

$$\text{Skill Value} = \Delta\text{Success} - \text{Cost} - \text{Latency} - \text{Constraint}$$

No Skill



成功率 · Token · Wall time 工具调用 恢复次数

Full Skill



成功率 · Token · Wall time 工具调用 恢复次数

Thin Skill

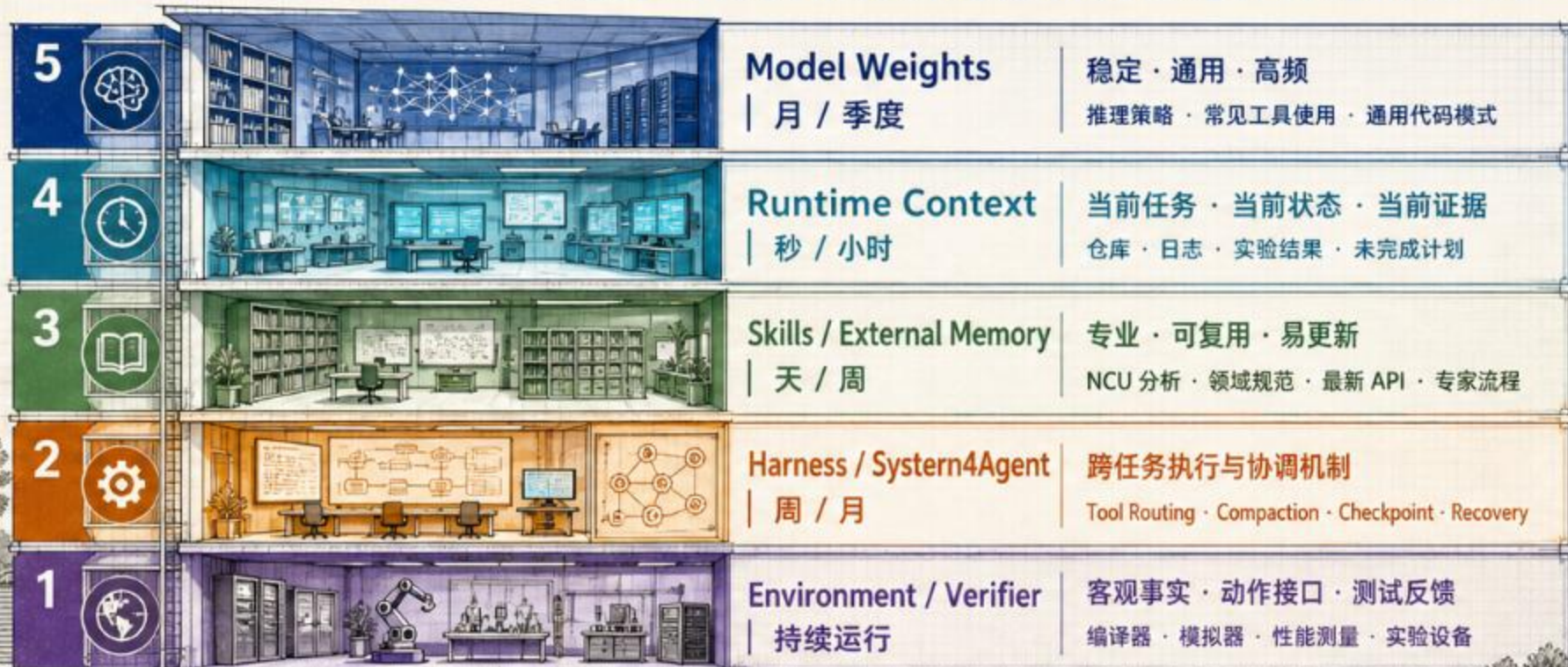


成功率 · Token · Wall time 工具调用 恢复次数

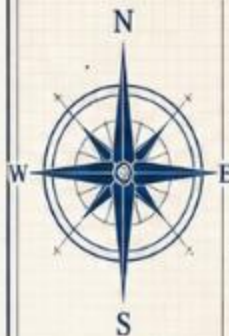
Skill 不是知识圣经，而是需要持续评测的软件组件。

什么应该被模型内化，什么应该留在系统外部？

能力应该放在更新速度和责任边界合适的位置。



放置位置的四个判断维度



是否稳定？

是否通用？

更新多快？

是否依赖
当前真实状态？



模型内化“如何判断”；系统保留“此刻发生了什么、能够做什么、是否真的做对”。



System4Agent
放大并稳定能力



Agent4System
修改真实目标系统



轨迹与经验



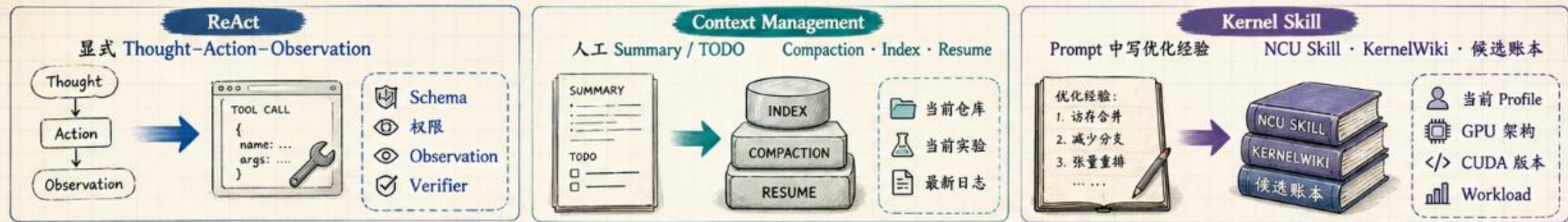
更新
Skill / Harness /
Post-training



能力不是越多塞进权重越好；稳定知识向内沉淀，动态状态与客观反馈留在外部。

今天写在 Skill 里的东西，明天还需要写吗？

能力会从外部脚手架逐渐沉淀到 Harness 与模型，但真实状态和反馈始终来自环境。



- 1 人工 Prompt 技巧
- 2 可复用 Skill
- 3 Harness / Plugin 原语
- 4 成功 + 失败轨迹
- 5 SFT / RL / Distillation
- 6 模型原生能力
- 7 旧 Skill 压缩 / 改写 / 退休



★ 自我改进首先发生在**外循环**；稳定、通用的经验再逐渐被**蒸馏**进模型**内循环**。

这不是所有能力必经的单向通道，而是一种常见的能力沉淀过程。

💡 “教模型怎么想”的脚手架可能变薄；让模型**看见**、**行动**和**验证**的系统不会消失。

参考文献与项目资料

按课程出现顺序列出主要论文、技术报告与公开项目

1. Yao et al. ReAct. ICLR 2023. arXiv:2210.03629

2. OpenAI. Function Calling API. 2023.

3. Anthropic. Model Context Protocol. 2024.

4. Google. Agent2Agent Protocol. 2025.

5. He et al. ChatEDA. arXiv:2308.10204

6. Szymanski et al. A-Lab. Nature 2023.

7. Yang et al. SWE-agent. arXiv:2405.15793

8. Ouyang et al. KernelBench. arXiv:2502.10517

9. KernelBench Verified. arXiv:2607.16241

10. STRATUS. NeurIPS 2025.

11. AISHPerf 智算运维智能体评测基准. 2026.

12. PithTrain. arXiv:2605.31463

13. VibeServe. arXiv:2605.06068

14. Prime Intellect. Autonomous nanoGPT Research.

15. Anthropic. AI-resistant Technical Evaluations.

16. Anthropic. Effective Context Engineering for AI Agents.

17. MIT HAN Lab. Kernel Design Agents.

18. SkillsBench. 2026.

19. DeepSeek. DeepSeek Harness.

完整链接见本页讲者备注与配套讲稿。

参考文献与项目资料

按课程出现顺序列出主要论文、技术报告与公开项目

1. Yao et al. ReAct. ICLR 2023. arXiv:2210.03629

2. OpenAI. Function Calling API. 2023.

3. Anthropic. Model Context Protocol. 2024.

4. Google. Agent2Agent Protocol. 2025.

5. He et al. ChatEDA. arXiv:2308.10204

6. Szymanski et al. A-Lab. Nature 2023.

7. Yang et al. SWE-agent. arXiv:2405.15793

8. Ouyang et al. KernelBench. arXiv:2502.10517

9. KernelBench Verified. arXiv:2607.16241

10. STRATUS. NeurIPS 2025.

11. AISHPerf 智算运维智能体评测基准. 2026.

12. PithTrain. arXiv:2605.31463

13. VibeServe. arXiv:2605.06068

14. Prime Intellect. Autonomous nanoGPT Research.

15. Anthropic. AI-resistant Technical Evaluations.

16. Anthropic. Effective Context Engineering for AI Agents.

17. MIT HAN Lab. Kernel Design Agents.

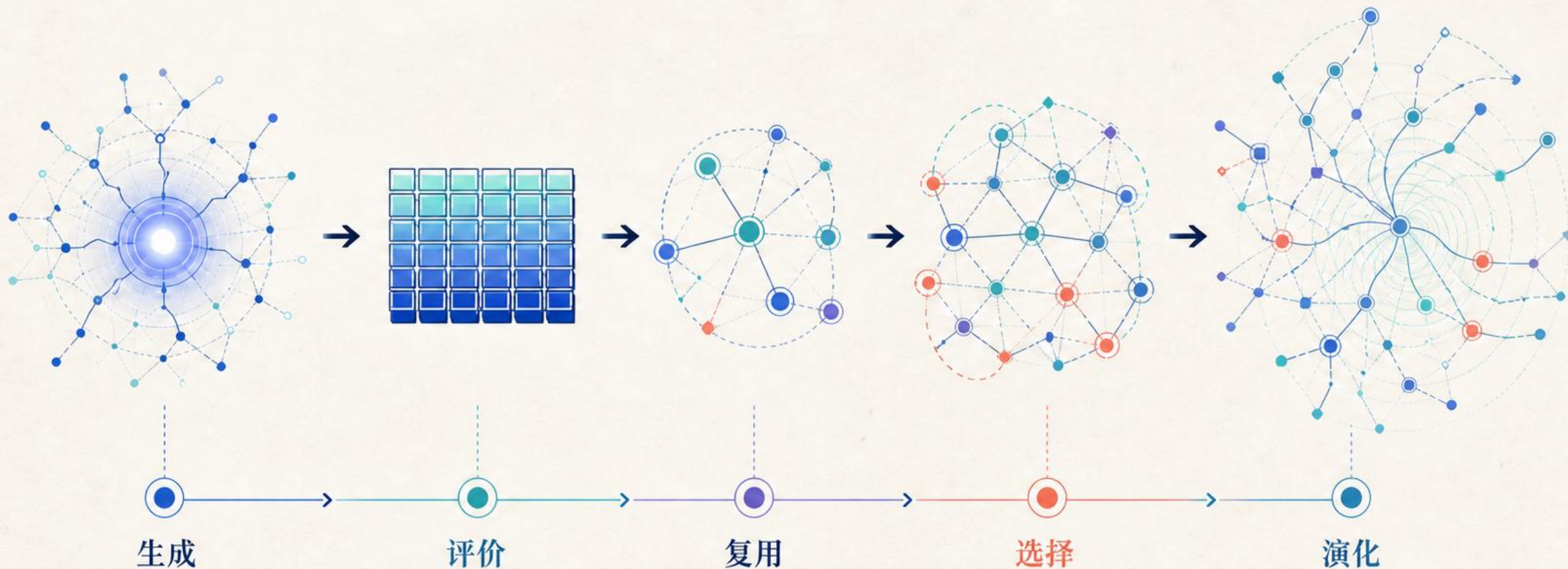
18. SkillsBench. 2026.

19. DeepSeek. DeepSeek Harness.

完整链接见本页讲者备注与配套讲稿。

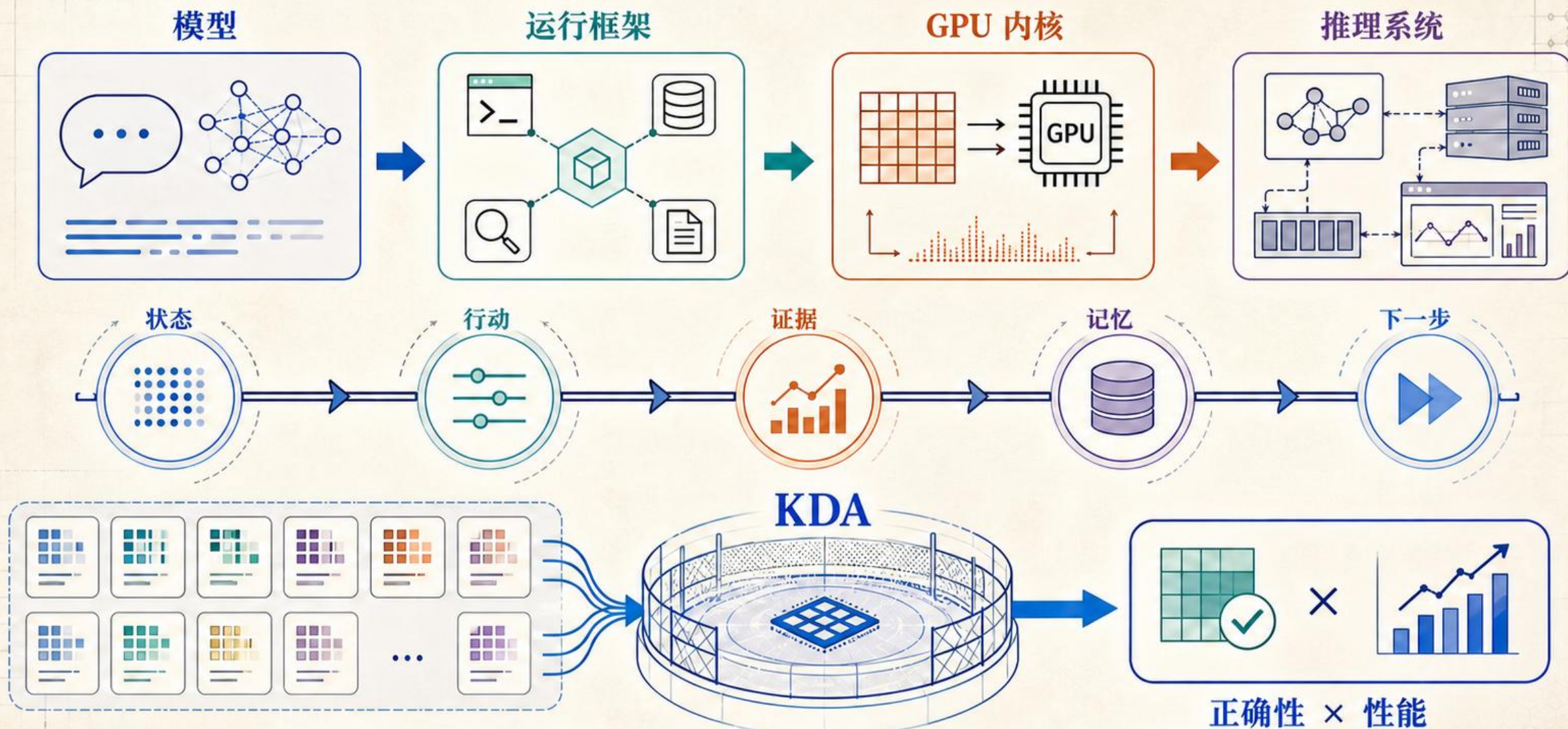
Agent4Kernel: 人工智能如何发现、验证并积累 GPU 优化

从单个内核生成，到持续演化的推理系统

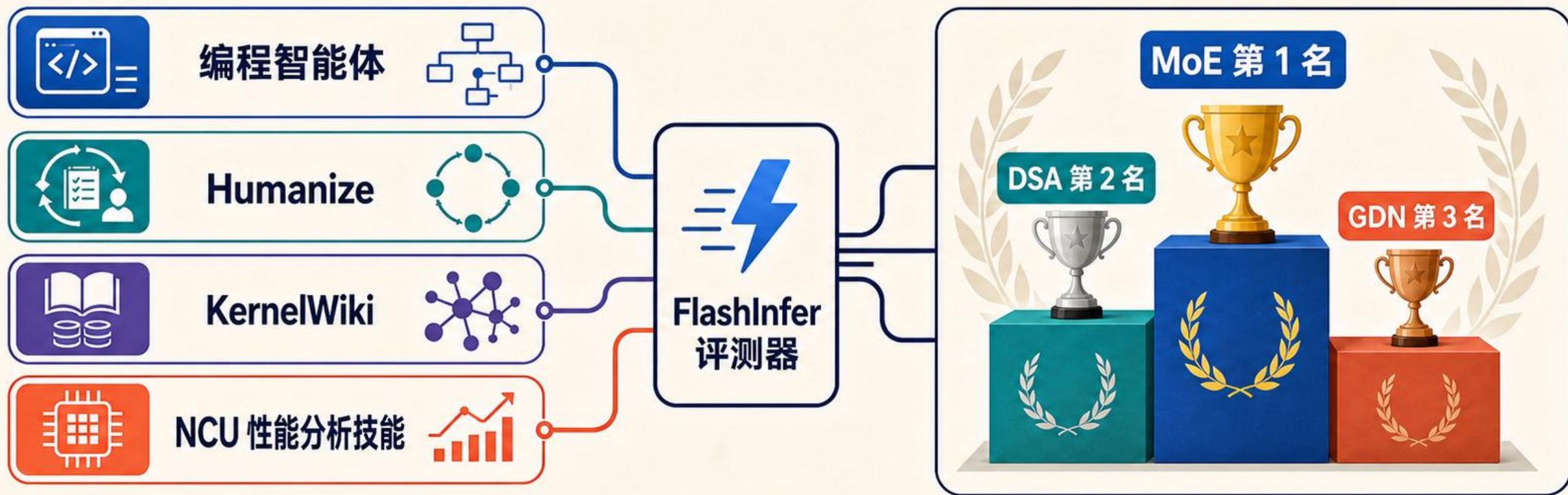


第二节课：把第一节课的状态—行动—反馈闭环，放进 GPU 内核与推理系统。

第一节课的闭环如何落到 GPU 优化



MLSys 内核优化挑战：KDA 在三条赛道进入前三



模型 × 运行框架 × 专业技能 × 快速反馈

比赛结果证明：模型能力只有经过运行框架、专业知识和评测反馈，才能变成真实优化能力。

比赛之后，智能体可以修改的范围继续扩大



行动范围扩大

反馈延迟与系统耦合增加

KDA

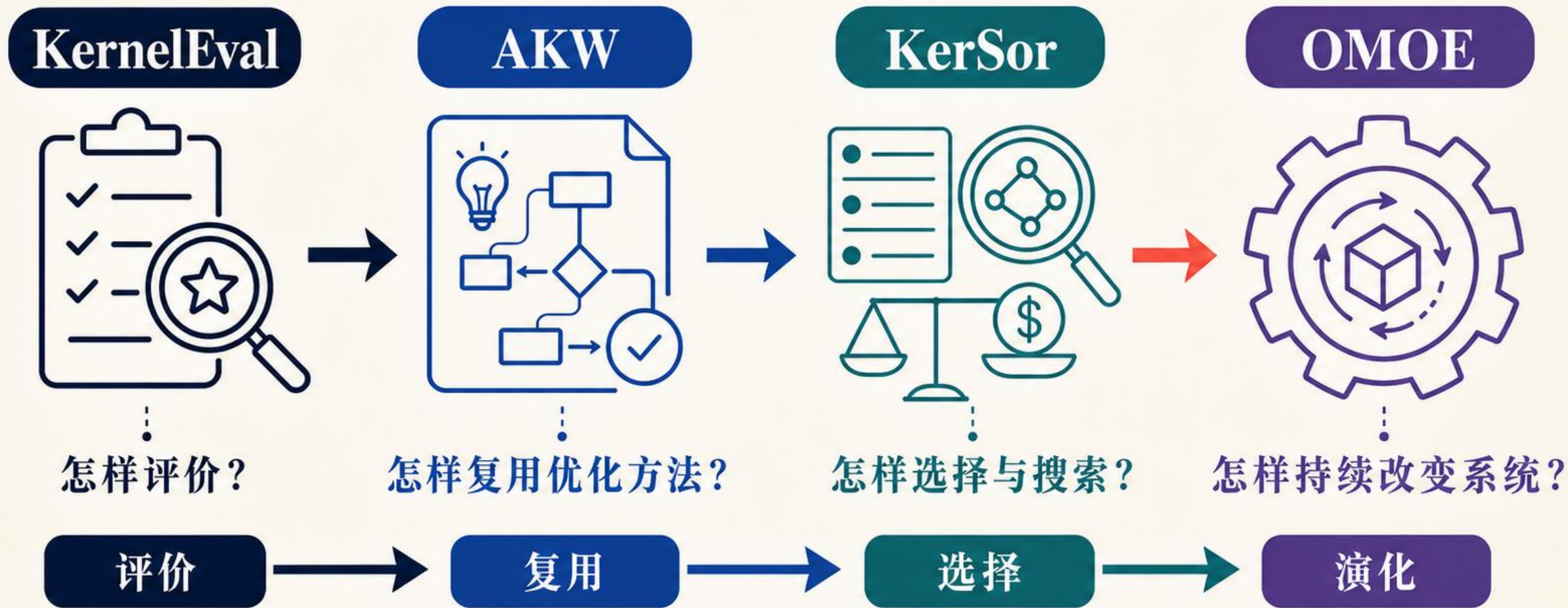
KernelEval

AKW · KerSor

OMOE

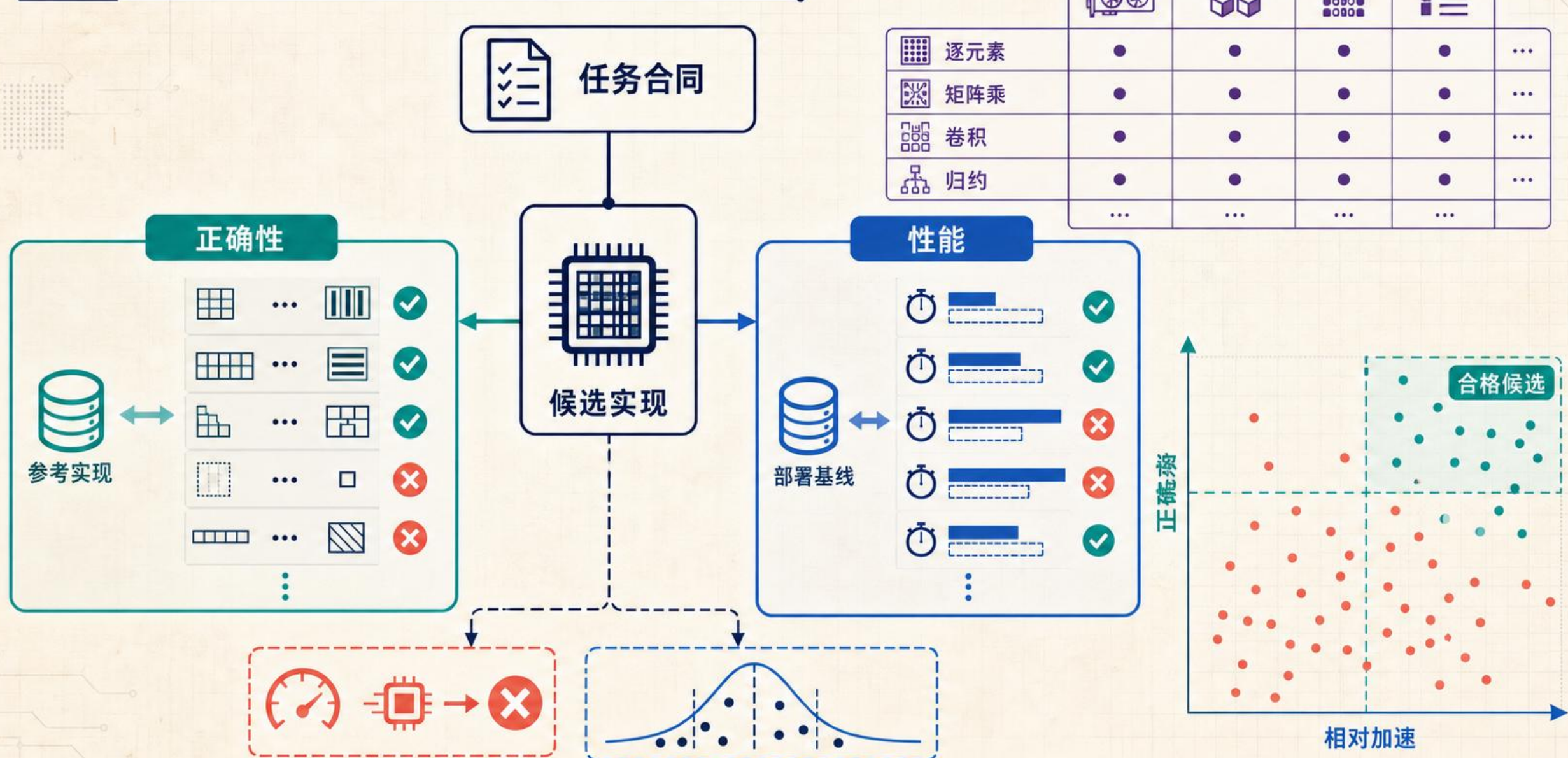
范围越大，潜在收益越大；但实验成本、反馈延迟和归因难度也随之增加。

四项工作解决的是同一条链上的四个问题

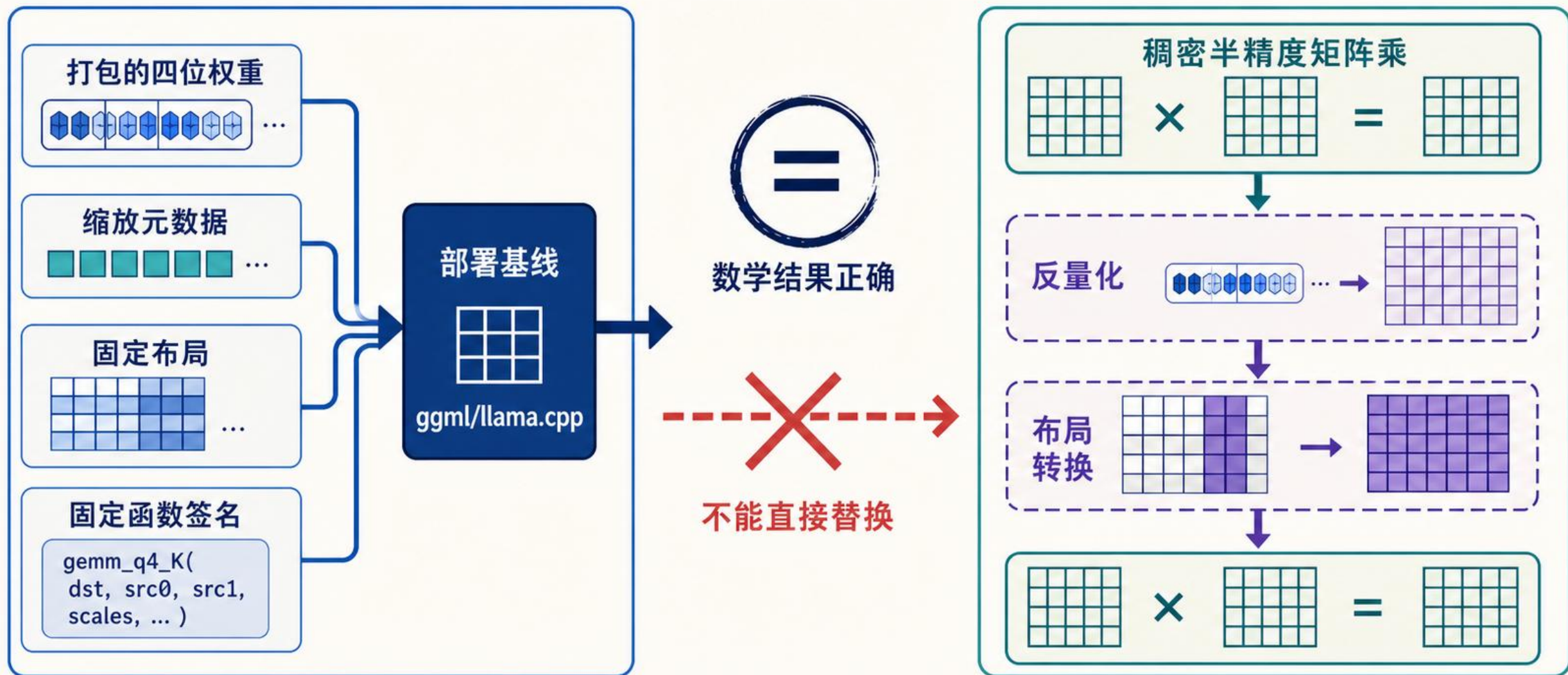


比赛展示智能体已经能做什么；四项工作继续回答怎样把一次胜利变成可靠、可复用、可持续的系统能力。

KernelEval: 先定义什么样的结果才有价值

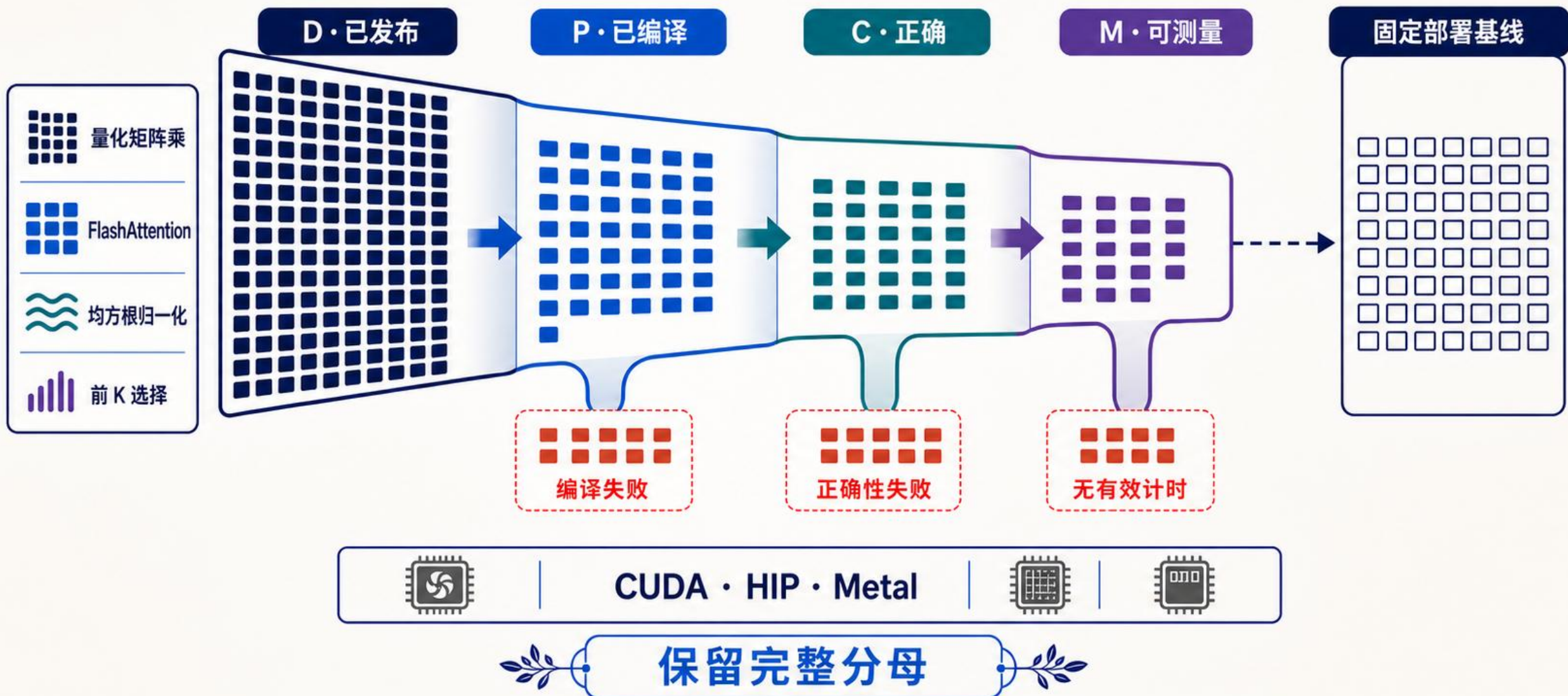


数学等价，不等于可以替换生产内核



生产调用合同也是问题的一部分：输入表示、布局 and 函数签名都必须保留。

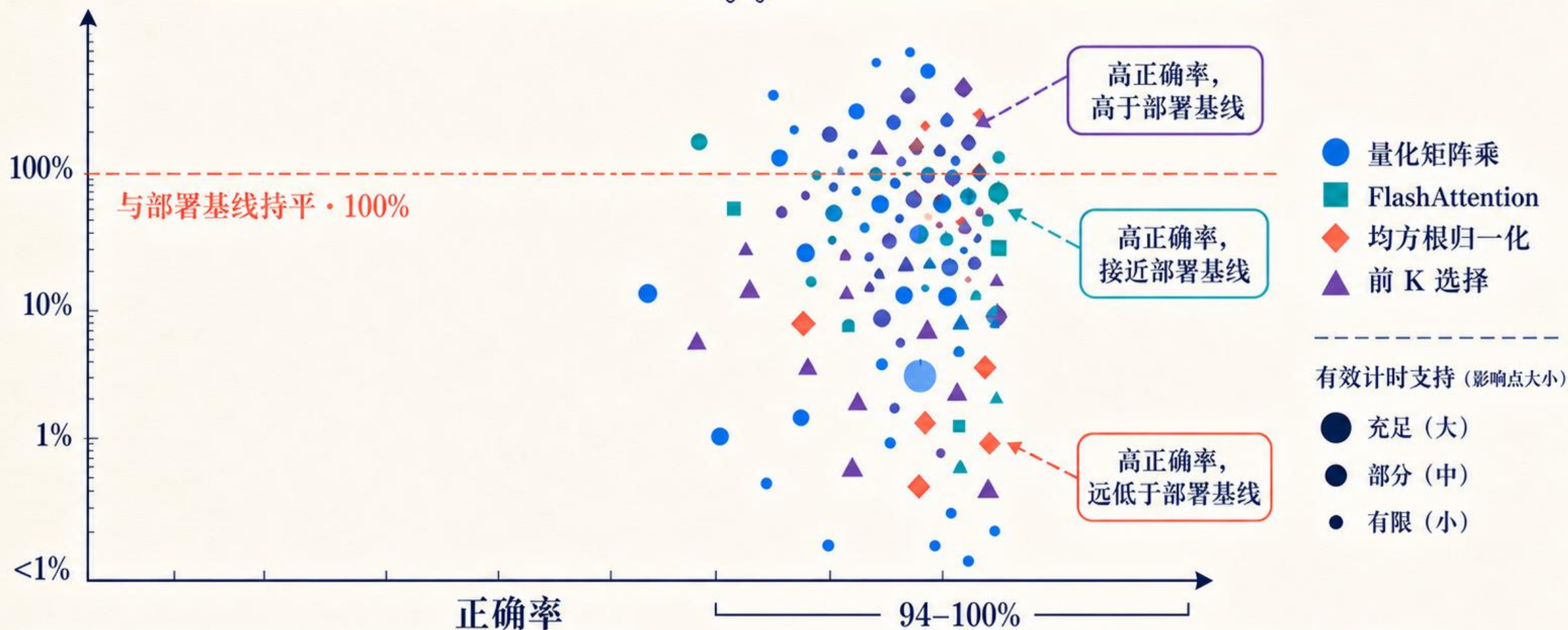
103 个任务把失败拆成四个阶段



系统不会因为大量失败而获得更漂亮的平均数; 每个失败都必须留在分母里。

高正确率，仍可能远离部署基线

相对部署基线比值



正确性只是门槛

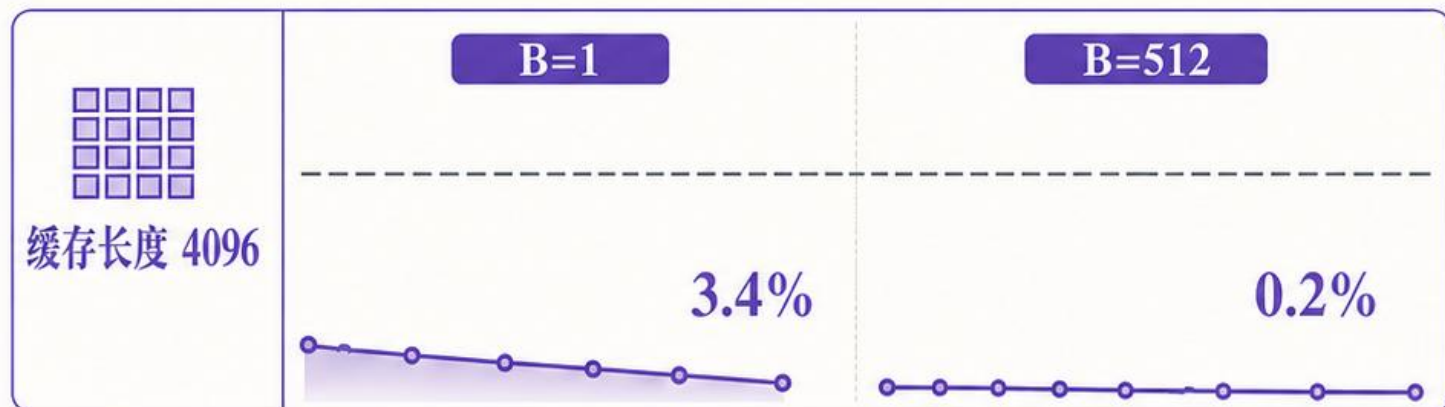
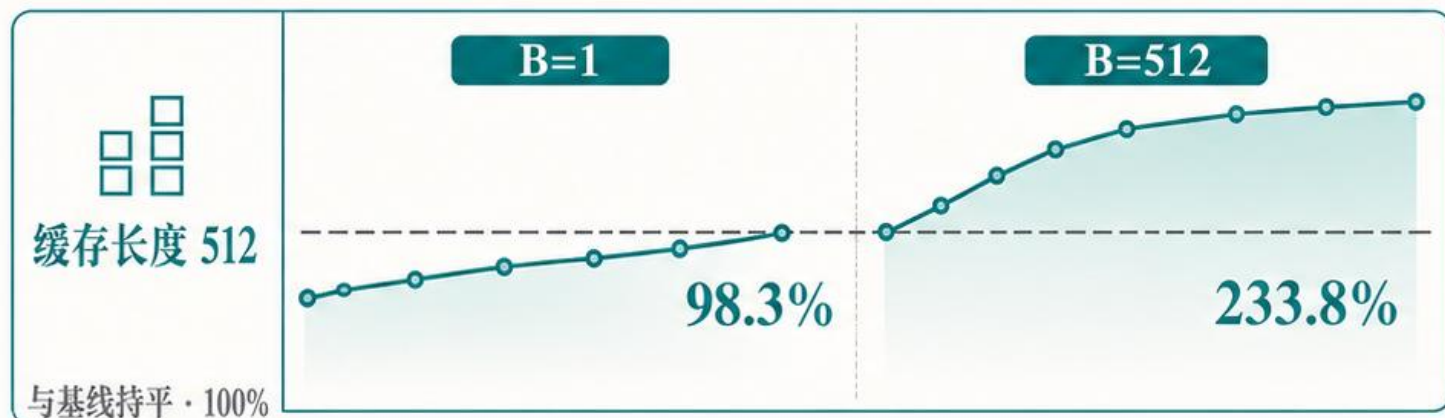


性能是另一条轴

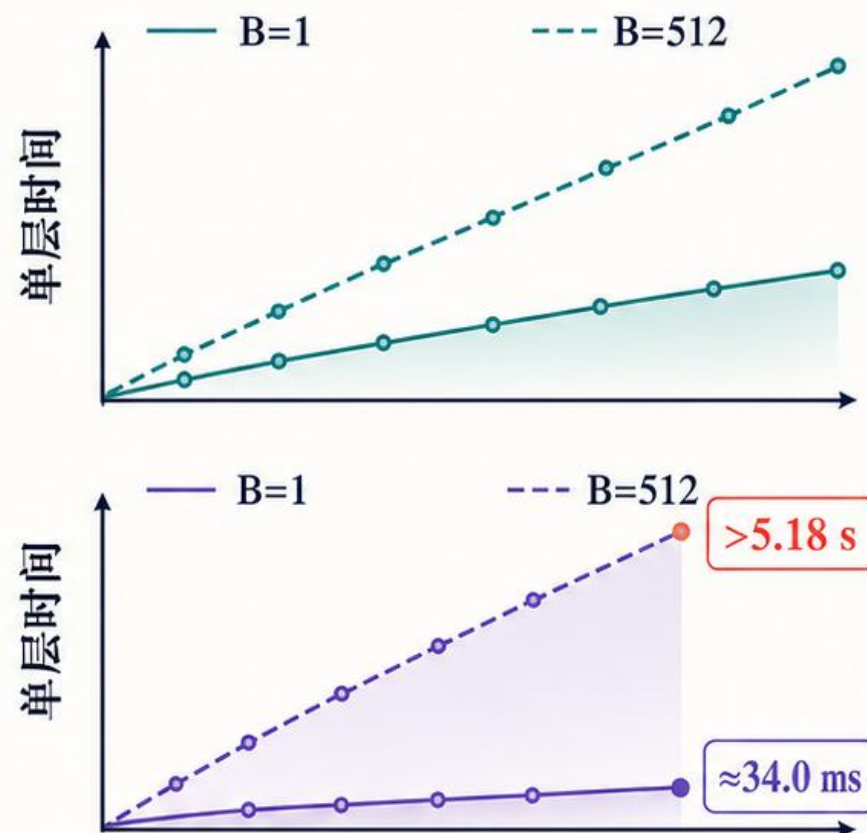
多个系统都能生成正确内核，但真正的部署价值仍可相差两个数量级。

算子类型和批量区间共同决定能力边界

A800 · FlashAttention



32 层累积



同一个算子，不同负载区间



能力边界必须写清算子、形状、批量、布局和硬件，不能只写一个任务名称。

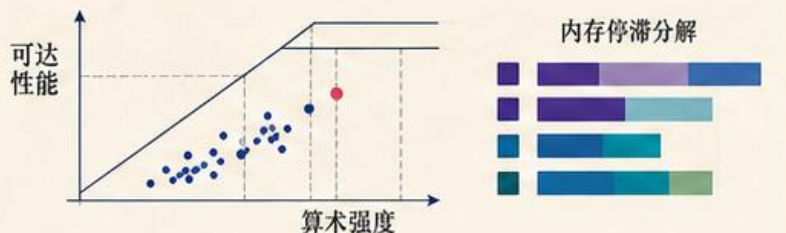
AKW: 把优化经验变成智能体可执行的方法

代码变更记录

```
__global__ void kernel(float* A, float* B, float* C, int N) { .... }
```

- before	+ after
<pre>__global__ void kernel(float* A, float* B, float* C, int N) { int i = blockIdx.x * blockDim.x + threadIdx.x; if (i < N) { C[i] = A[i] * B[i]; } }</pre>	<pre>__global__ void kernel(float* A, float* B, float* C, int N) { int i = blockIdx.x * blockDim.x + threadIdx.x; float4 a = *reinterpret_cast<float4*>(A+i*4); float4 b = *reinterpret_cast<float4*>(B+i*4); float4 c = a * b; *reinterpret_cast<float4*>(C+i) = c; }</pre>

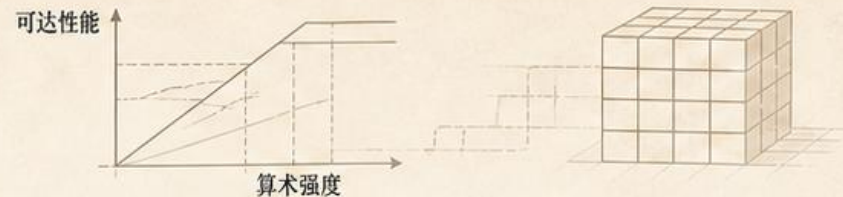
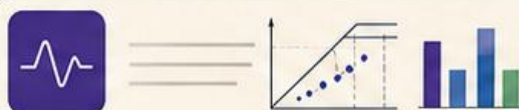
NCU



实验记录

	接受	拒绝

KernelWiki



技能模块



候选实现

```
__global__ void kernel(...){  
    int i = ...;  
    C[i] = A[i] * B[i];  
}
```

```
__global__ void kernel(...){  
    int i = ...;  
    float4 a = *reinterpret_cast<...>(A+i*4);  
    float4 b = *reinterpret_cast<...>(B+i*4);  
    float4 c = a * b;  
    *reinterpret_cast<...>(C+i) = c;  
}
```

```
__global__ void kernel(...){  
    int i = ...;  
    for (int k = 0; k < ...; ++k){  
        C[i] += A[i * k + lda] * B[k];  
    }  
}
```



知识可以缩小搜索空间，但不能替代新目标上的正确性与性能测量。

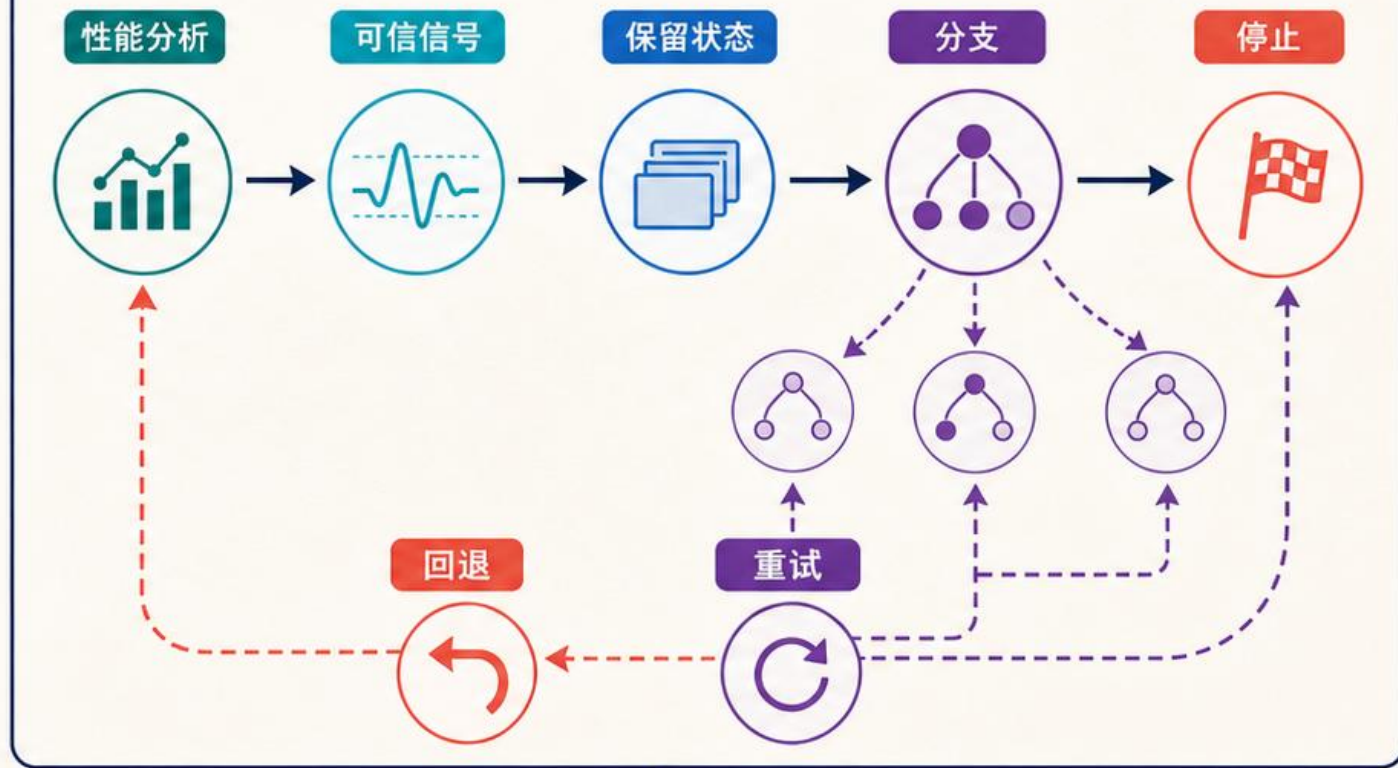
专家方法是决策过程，不是最终代码

最终内核

```
extern "C" __global__ void awk_final(  
    const float* __restrict__ in,  
    float* __restrict__ out,  
    const int n) {  
    extern __shared__ float sdata[];  
    const int tid = threadIdx.x;  
    const int i = blockIdx.x * blockDim.x + tid;  
  
    float sum = 0.0f;  
    for (int idx = i; idx < n; idx += blockDim.x * gridDim.x) {  
        sum += in[idx];  
    }  
    sdata[tid] = sum;  
    __syncthreads();  
  
    for (int stride = blockDim.x >> 1; stride > 0; stride >>= 1) {  
        if (tid < stride) sdata[tid] += sdata[tid + stride];  
        __syncthreads();  
    }  
  
    if (tid == 0) atomicAdd(out, sdata[0]);  
}
```

复制结果

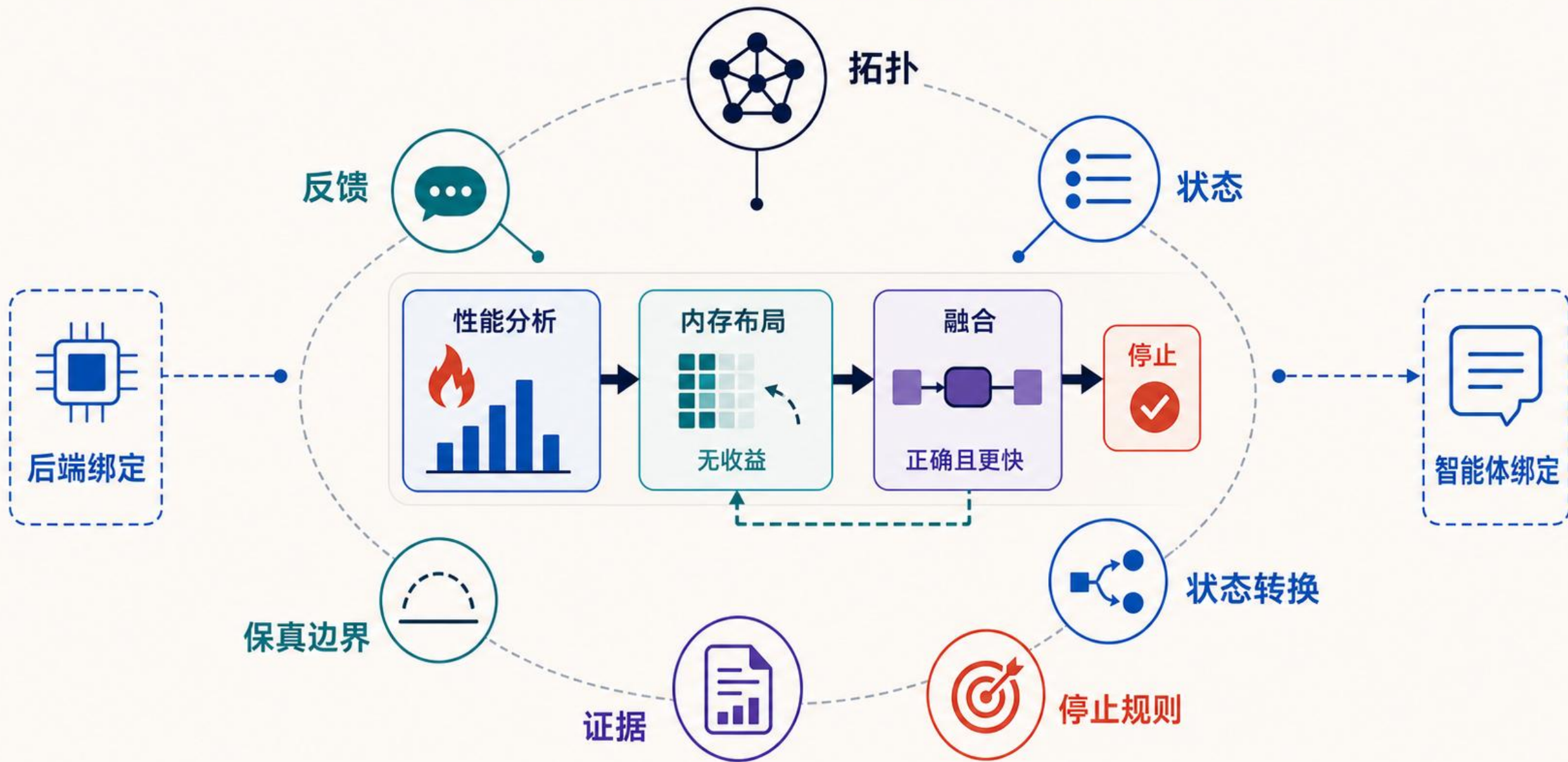
决策策略



复用方法

真正可以迁移的是观察、分支、回退和停止的决策过程，而不是原任务的代码快照。

AKW 把方法表示成可执行优化流程

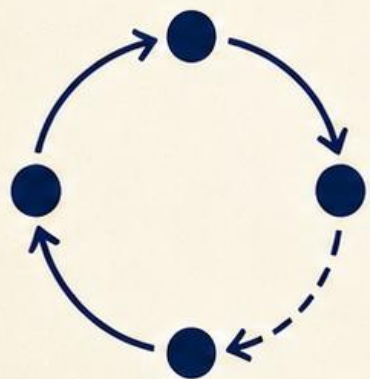


优化流程的身份由拓扑、状态、反馈、转换、停止、证据和保真边界共同定义。

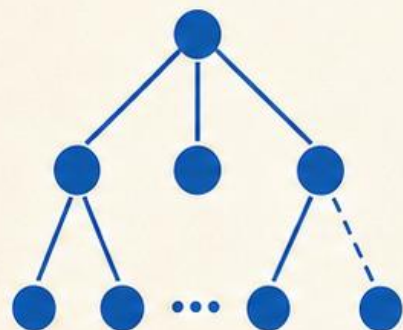
32 种优化方法，不是 32 条提示词



迭代



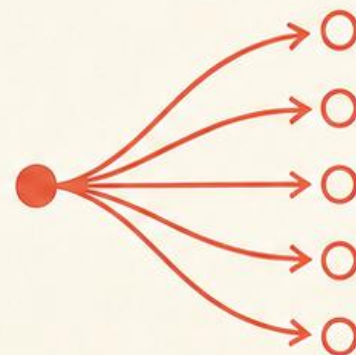
树 / 图搜索



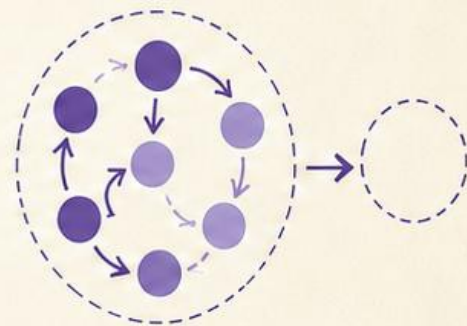
引导探索



并行生成



群体演化



方法之间真正不同的是搜索拓扑和状态演化方式，而不只是提示词写法。

五种优化方法，共用一个 B200 实验合同

26 个任务

																						Claude Code	Codex
🔍	K-Search	●	○	●	●	○	●	●	●	●	●	●	●	○	●	●	●	●	●	●	●	●	●
</>	CUDA-Agent	●	●	●	●	●	●	●	●	○	●	●	●	●	●	●	●	●	●	●	●	●	●
📌	AdaExplore	●	○	●	○	●	○	●	●	●	●	●	●	●	○	●	●	●	●	●	●	●	●
🏠	KernelAgent	●	○	●	●	●	●	○	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●
📁	KernelFoundry	●	●	●	●	●	●	●	●	●	●	●	●	●	●	○	●	●	●	●	●	●	●



3 个独立进程



10 次预热



50 次正式计时



所有工作负载通过



182 个
评测组合



179 个
有效结果



127 / 130 个
优化方法结果有效



52 / 52 个
默认智能体结果有效



13,719 / 13,731 个
工作负载通过

同一硬件、同一任务、同一正确性门槛和同一测量流程，让不同方法可以公平比较。

统一实验条件后，不同优化方法仍各有所长

		归一化	矩阵乘	分组查询注意力	多头潜在注意力	混合专家	
	K-Search	●		●		●	● 延迟比
	CUDA-Agent		●				● 延迟比
	AdaExplore	●		●	●		● 延迟比
	KernelAgent		●	●			● 延迟比
	KernelFoundry	●	●	●	●	●	● 延迟比



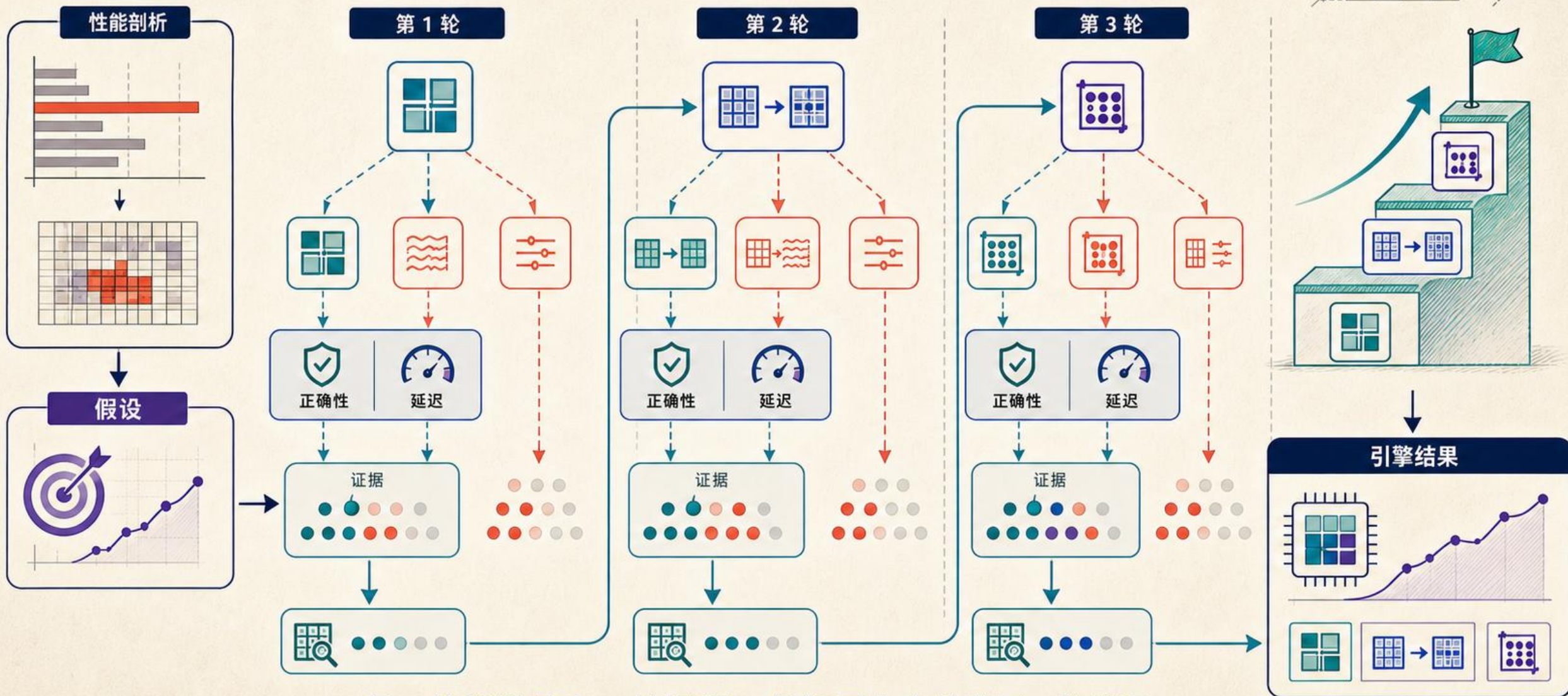
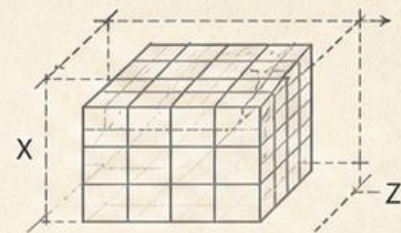
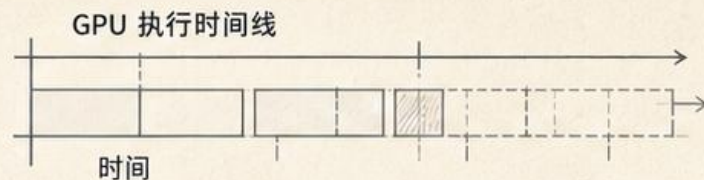
25 个有效任务中，10 个不差于更强默认智能体
几何平均延迟为 1.019 倍



每种优化方法都在部分任务上领先
不存在普遍最优的方法

统一硬件、任务和评测后，差异仍然存在，说明互补性来自方法本身。

KerSor: 在候选空间中持续搜索



每一轮都继承上一轮证据；负结果也会改变下一轮搜索。

没有任何一种优化流程能够普遍获胜

26 个任务 · B200



K-Search · 2 次获胜

CUDA-Agent · 3 次获胜

AdaExplore · 6 次获胜

KernelAgent · 2 次获胜

KernelFoundry · 13 次获胜

最佳固定方法 · 1.3718×

五种方法事后上界 · 0.8758×

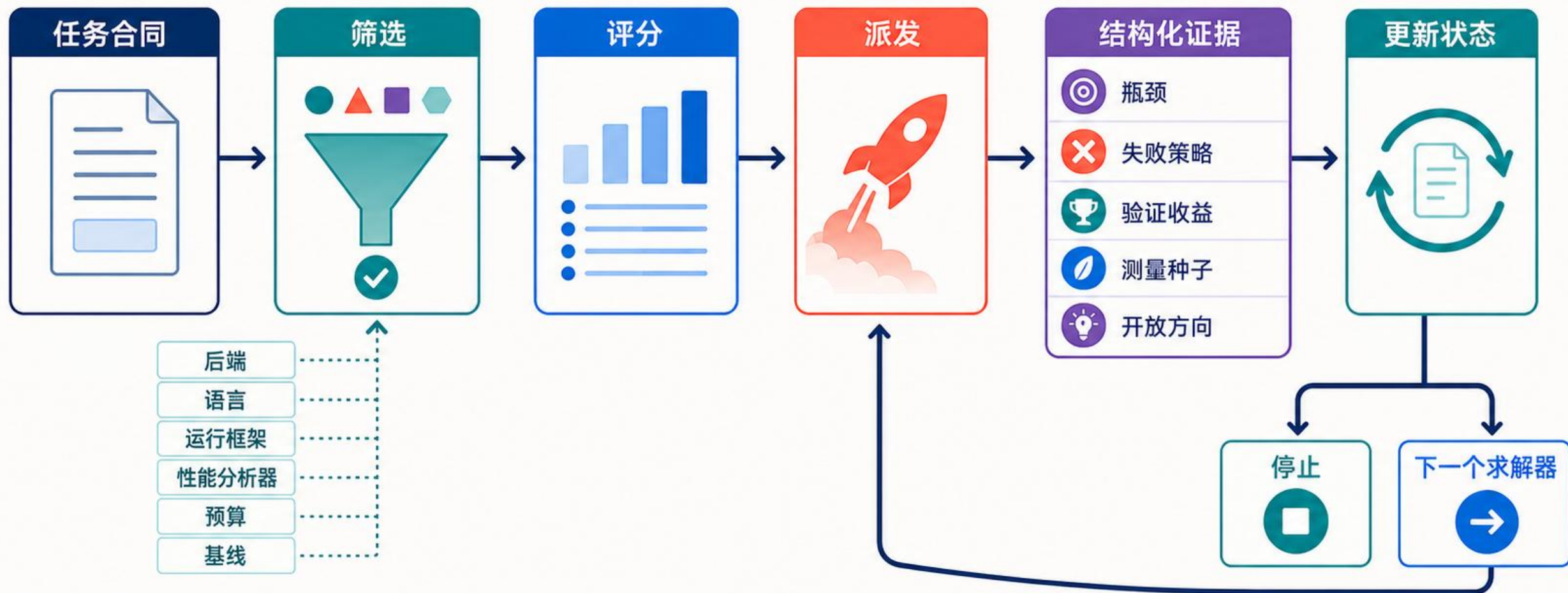
选择器可利用空间

越低越好

固定方法与事后上界之间，就是选择策略可以利用的空间。

事后上界 = 上界

KerSor 把每个优化流程当作求解器

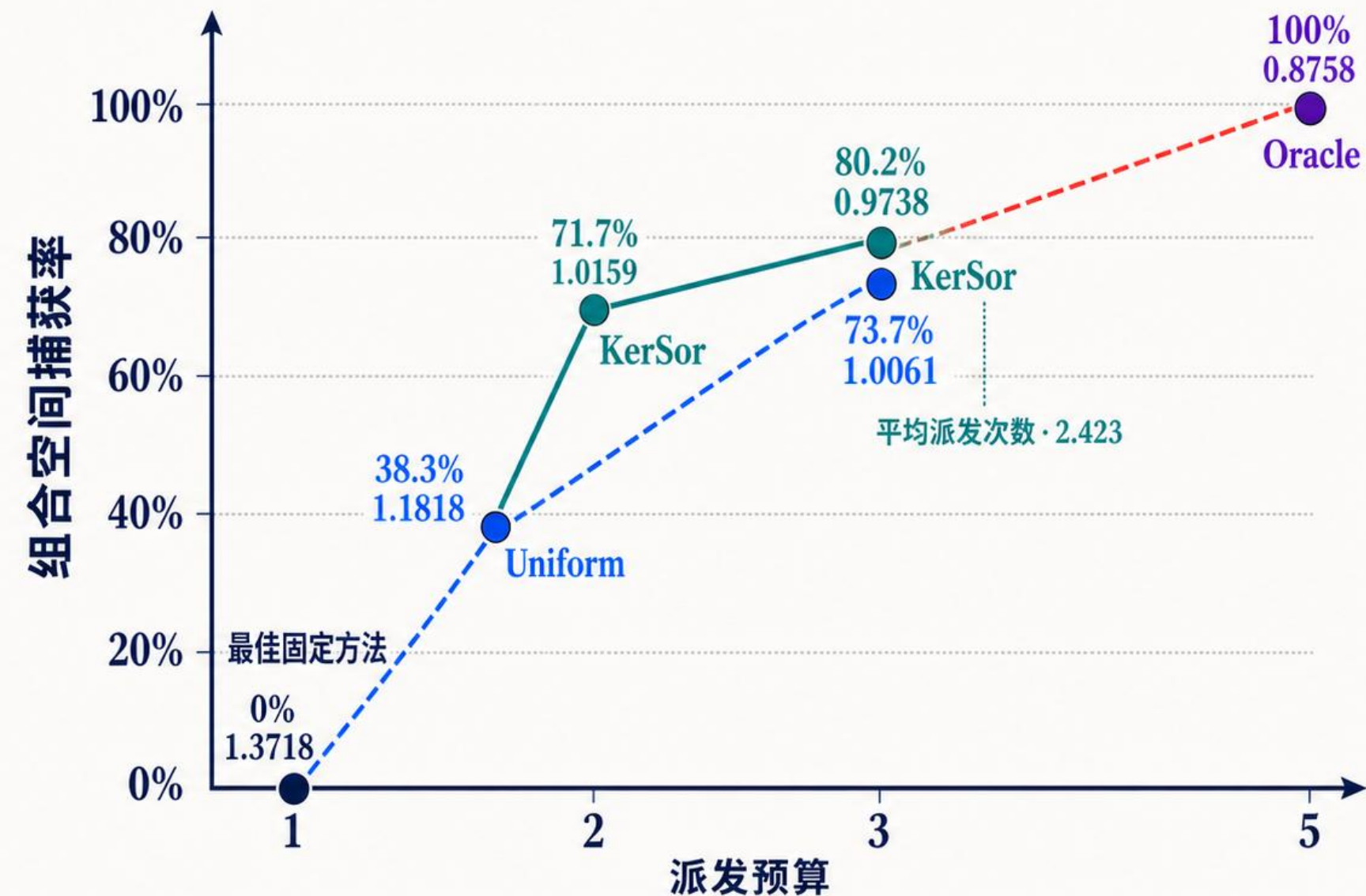


WSR · 未来先验



控制器不创造万能方法；它让上一轮证据决定下一轮运行谁。

两个派发已获得大部分方法组合空间



方法	预算	捕获率	几何平均延迟比
最佳固定方法	1	0%	1.3718
Uniform	2	38.3%	1.1818
KerSor	2	71.7%	1.0159
Uniform	3	73.7%	1.0061
KerSor	3	80.2%	0.9738
Oracle	5	100%	0.8758

延迟比越低越好

预算为 2 时，KerSor 已捕获 71.7% 的方法互补空间。

传递实验依据，可以改变下一轮结果

任务 017 · GQA 分段预填充

同一路由 · 预算 2 · 同一正确性门



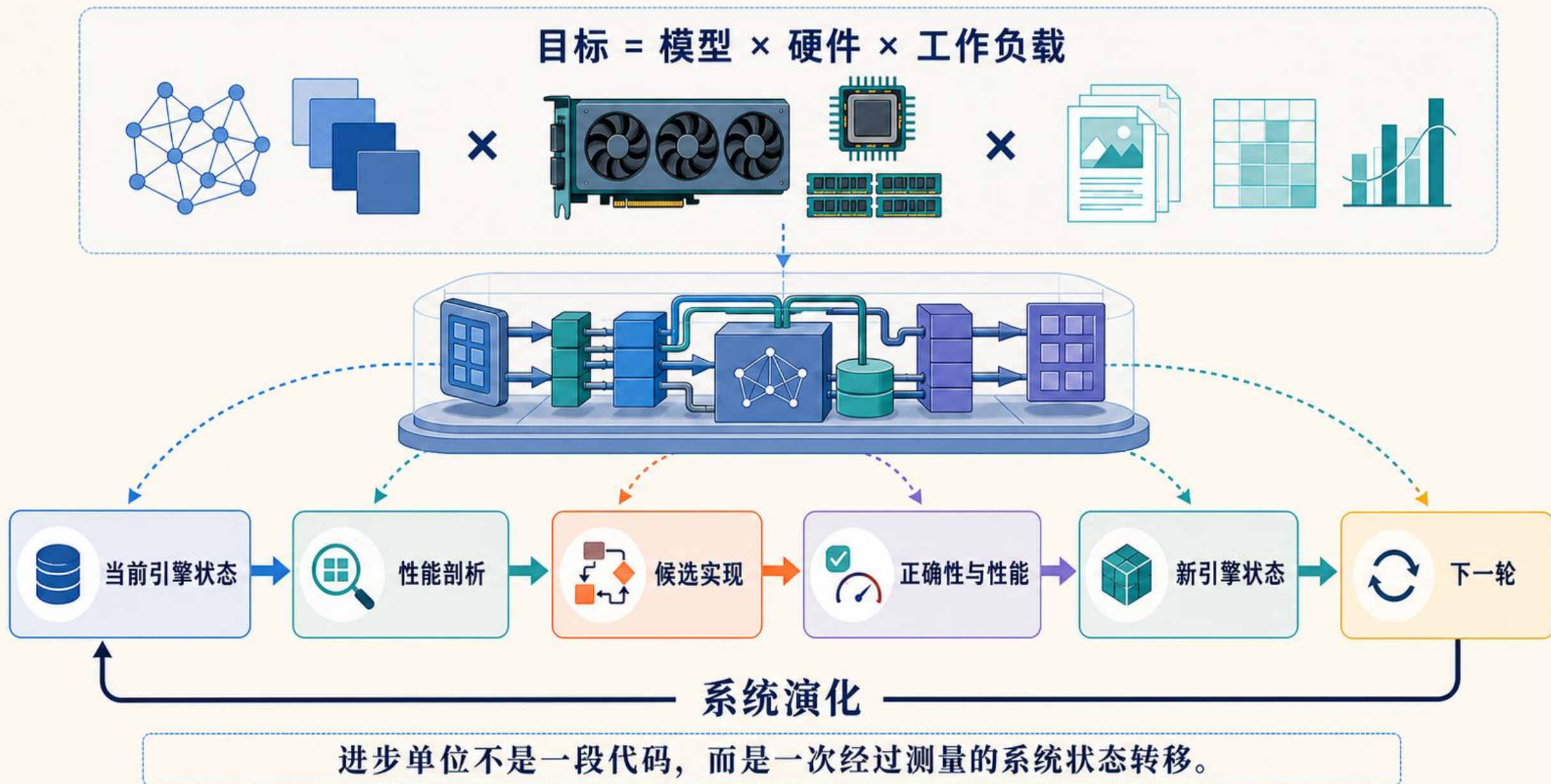
不传递证据

完整传递证据

-17.2% 延迟

只改变证据传递，最终延迟再降低 17.2%。

OMOE: 让智能体持续改变一个真实推理系统



OMOE: 同一个智能体，两种系统表面

同一智能体 · 同一任务 · 同一预算



直接优化 SGLang



约 59 万行代码



约 30 个
注意力后端



注册表、模型执行器、
CUDA Graph、请求批次、
内存池、配置



编辑(s)



构建



部署



端到端
运行



收集
指标



分析与
解读

反馈路径长 · 依赖多 · 等待久

OMOE: 局部优化单元

Op = 参考实现 + 候选版本 + 输入 + 检查 + 基准测试

参考实现

候选版本

输入

检查

基准测试



约 11 分钟



21.25 →
约 18 ms/token



约 1.19x



修改
候选版本



检查



基准测试



测量

反馈路径短 · 本地可解释 · 测量快

有效实验吞吐 (随迭代轮次)

第 1 轮

第 2 轮

第 3 轮

一次昂贵的端到端周期

等待与开销

第 1 轮

第 2 轮

第 3 轮

第 5 轮

第 6 轮

第 7 轮

第 8 轮

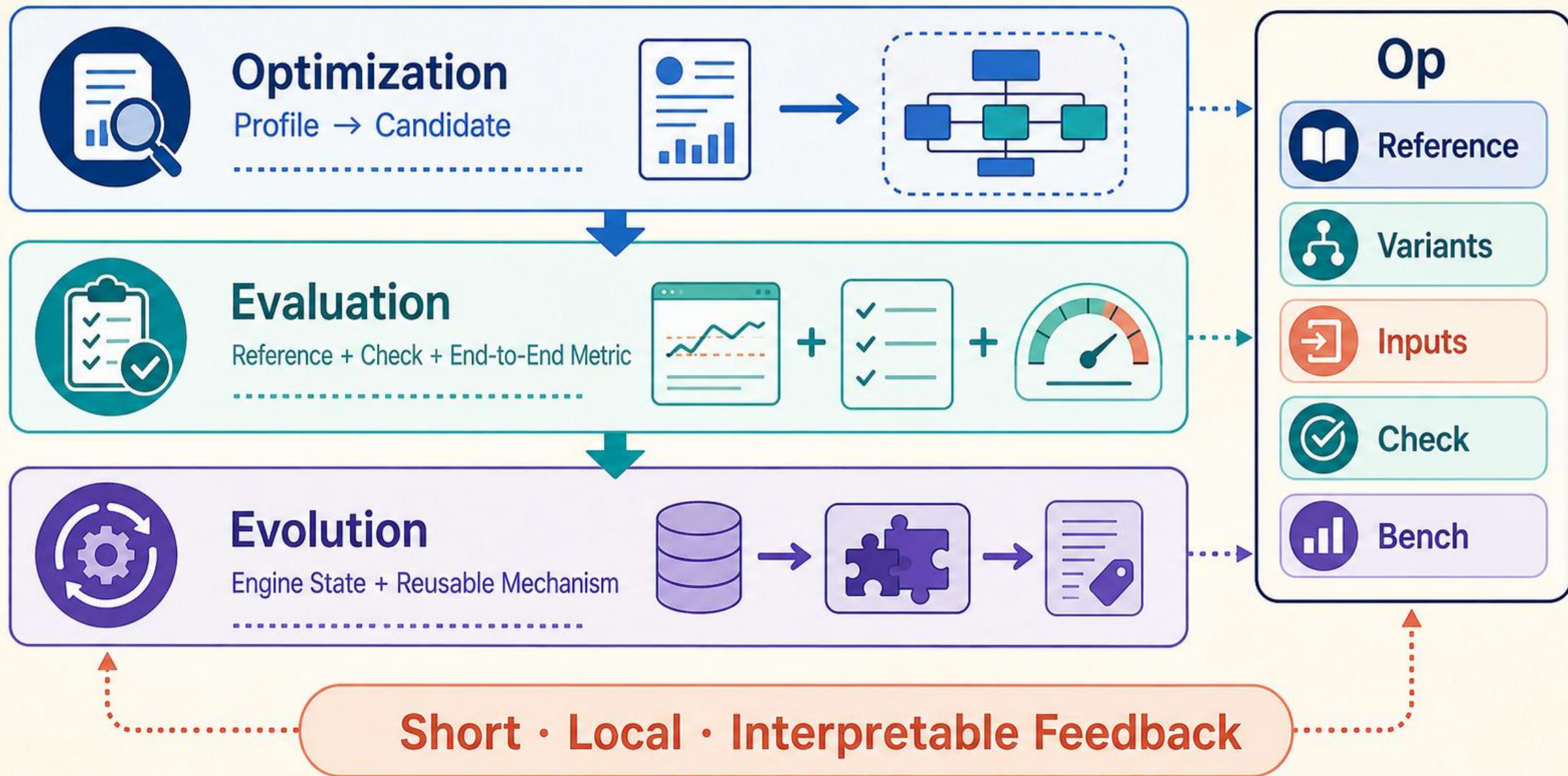
第 9 轮...

一次短的本地周期

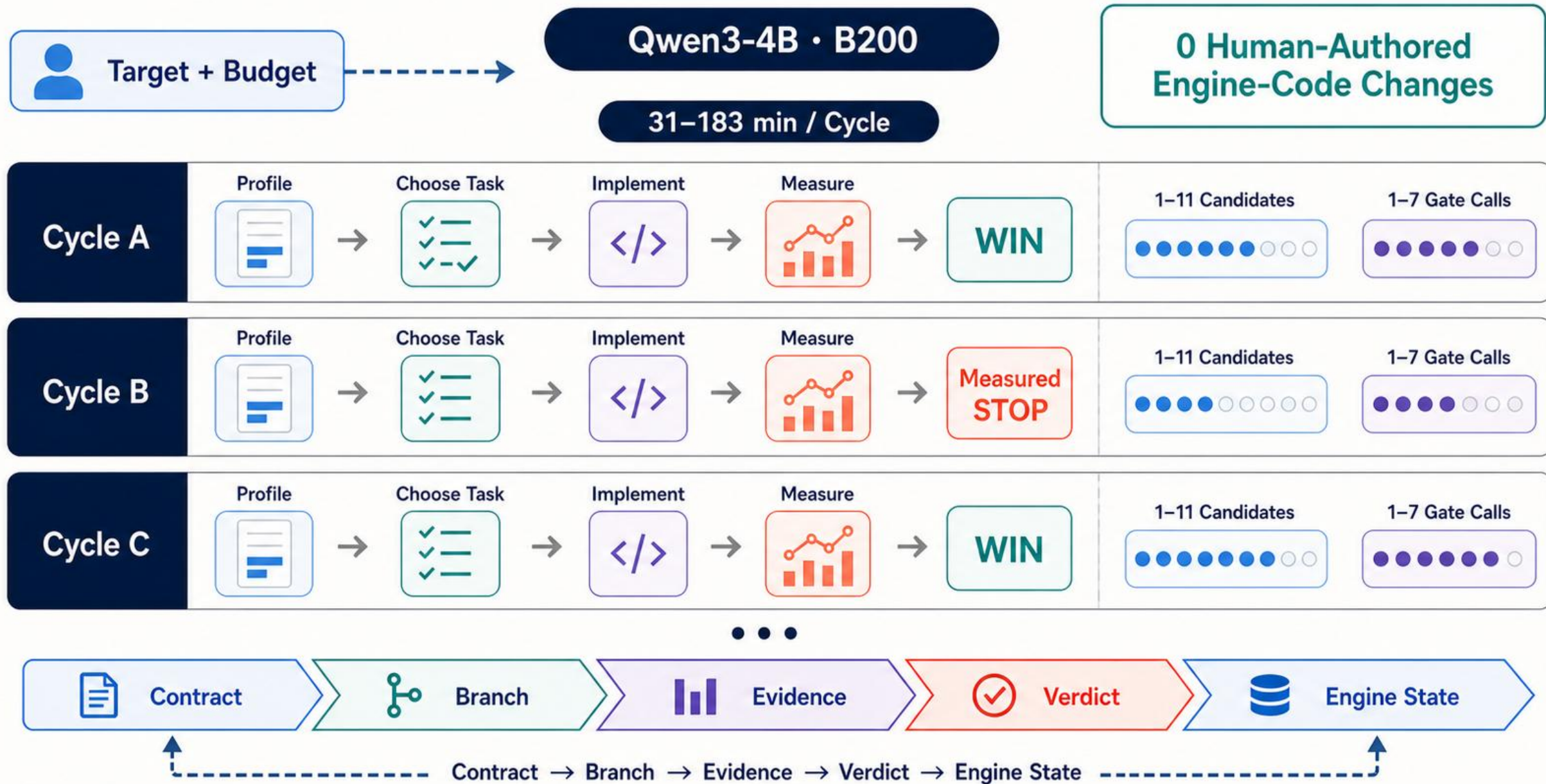
开销极小

系统表面的复杂度，决定同一智能体每轮能完成多少有效实验。

OMOE 把系统重组为 Agent 可以优化的对象



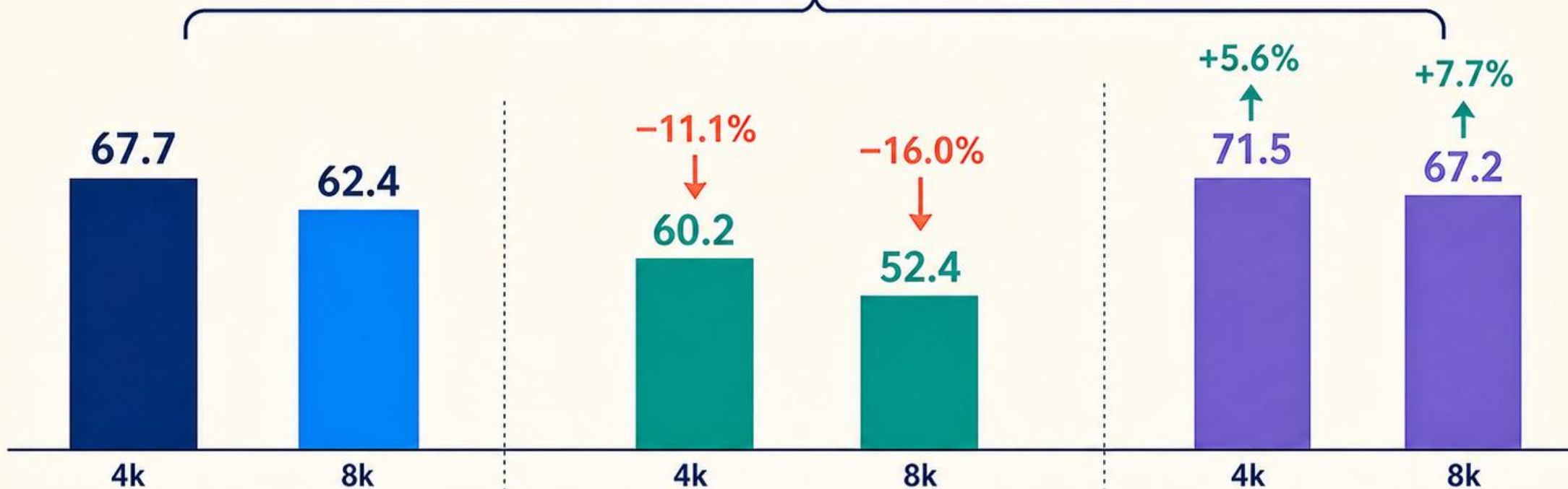
一次优化不是故事：Campaign 由真实尝试组成



两个候选同样正确，性能结果可以完全相反

RTX 5070

126 / 128 Correct



Verified

Single-Pass

Split-K

STOP

WIN

Correctness → Eligible

Performance → Persist

OMOE 的结果是一组 Target Specialist

$$\text{Target} = \text{Model} \times \text{Hardware} \times \text{Workload} \times \text{Precision}$$

Model	Hardware	Workload	Precision	Peer	相对结果 (Relative Result)
 Qwen3-4B	 B200	 TG@4k	BF16	 SGLang	↓ -24.4% latency
 Qwen3-4B	 B200	 TG@4k	BF16	 VibeSys	↓ -61.8% latency
 Llama-3.1-8B	 B200	 c16	BF16	 VibeSys	↑ +16.4% throughput
 Qwen3.5-2B	 RTX 5070	 TG@4k	BF16	 SGLang	↑ +5.9% throughput
 Pi0.5	 Jetson Orin	 action	BF16	 PhyAI	↓ -6.2% latency
 Qwen3-0.6B	 M1 Pro	 wall-token	W8	 Ollama Q8_0	↓ -27.4% latency

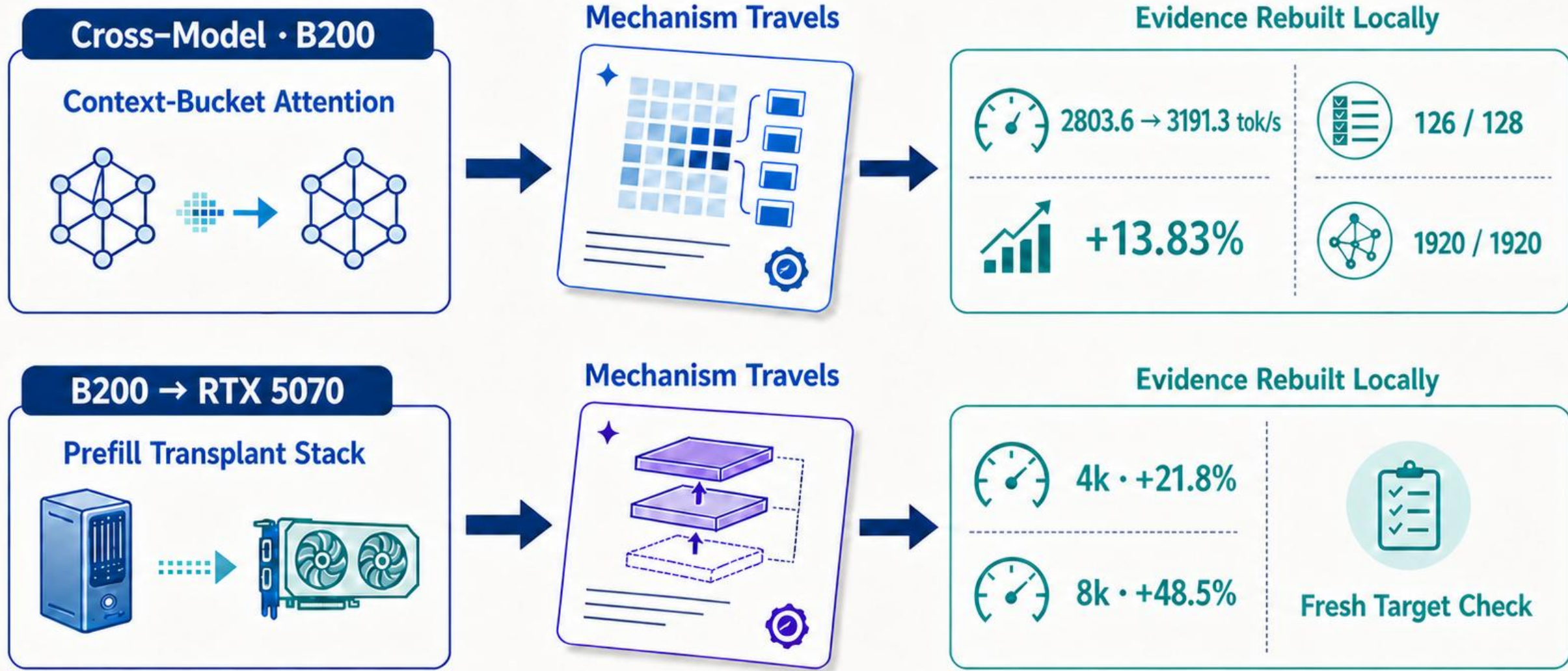
Open Gap

 Llama (混合精度) c16	OMOE 3103.12 tok/s	SGLang 4156.70 tok/s	ratio 0.7465×
--	-----------------------	-------------------------	------------------



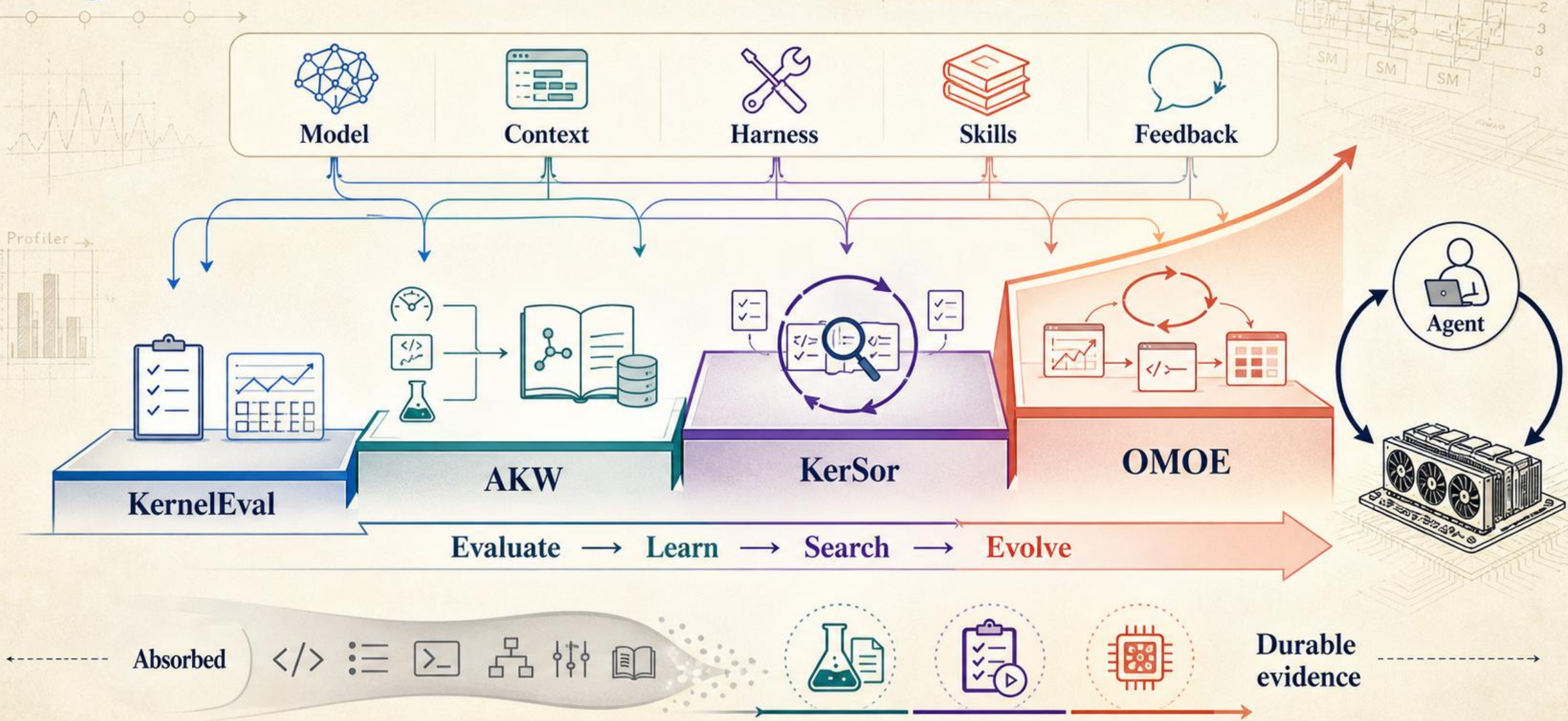
不同 Cell, 数字不可相加

经验可以迁移，结果必须在新 Target 上重新产生

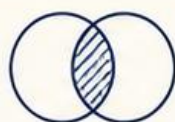
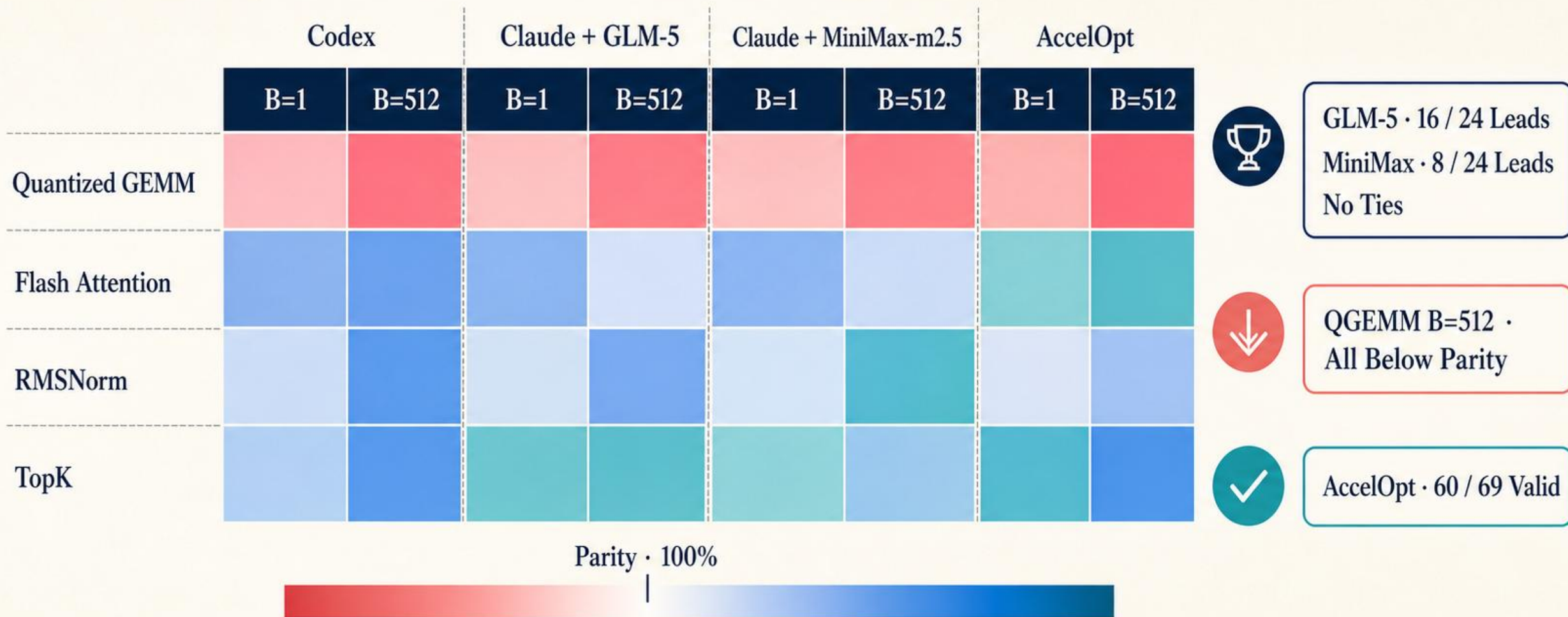


✦ 迁移机制，不迁移结论 ✦

Agent4Kernel: 能力如何逐级扩展



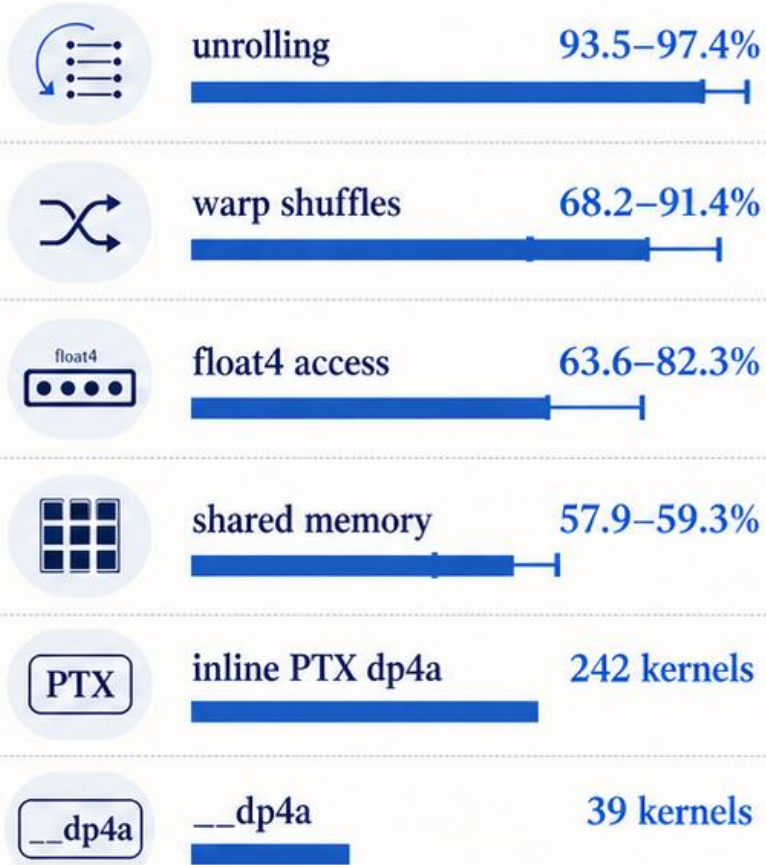
附录 | 完整系统差异仍然依赖 Operator 与 Regime



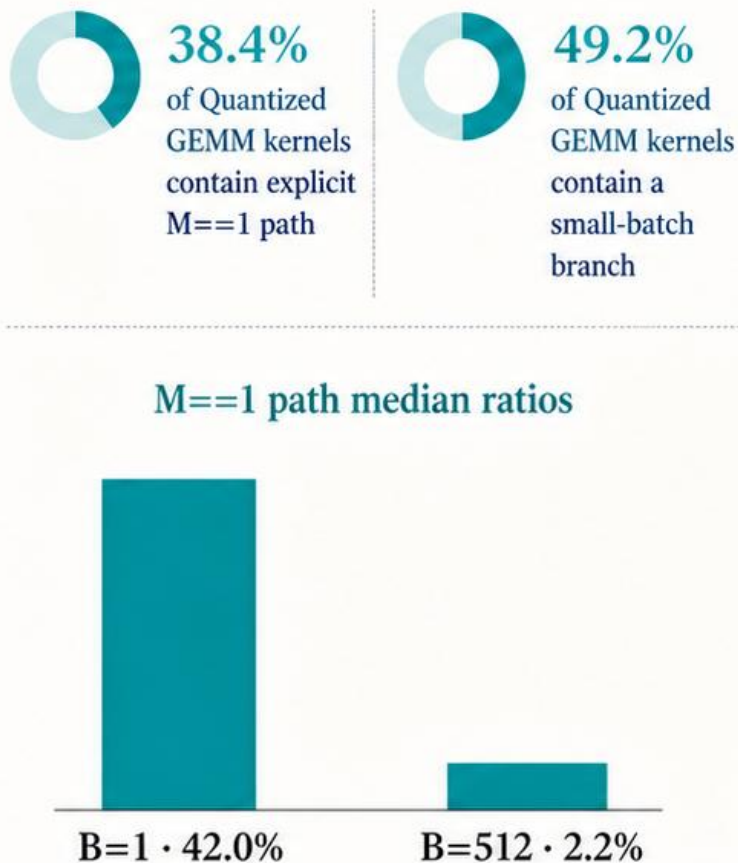
Matched Measurable Tasks

附录 | 645 个 Artifact 暴露当前 Agent 的优化偏好

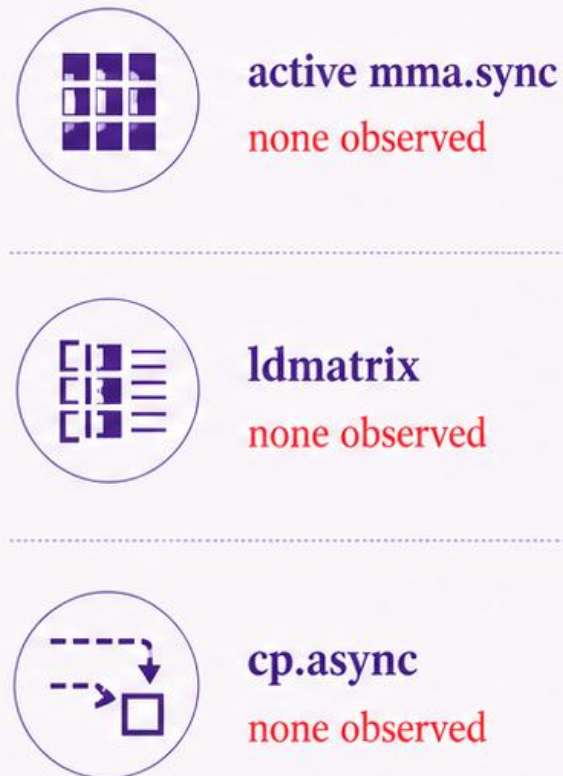
Common Repertoire



Regime Specialization



Rare Complete Tensor-Core Pipelines



Lexical Observation • Not Causal Proof

附录 | AKW Catalog: 方法库与执行证据分层



32 Entries



16 Academic



13 Industrial



3 AKW-native

Source Links



Topology



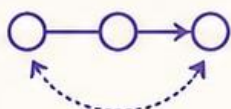
State



Feedback



Transition



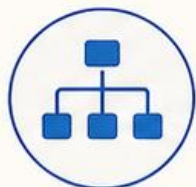
Stop



Evidence



Fidelity



27

Workflow Families



112

NVIDIA / CUDA
Sessions



68

Complete Quantitative
Chains



H800



RTX 4090



MetaX C550



MultiKernelBench

Binding Breadth $\neq$ Performance Claim

附录 | 五种 Workflow 的 Endpoint Signature

Workflow	Valid	Match / Beat	Geo Ratio <i>Lower Is Better</i>	Kernel-Family Lead-Set Footprint				
				Norm	GEMM	GQA	MLA	MoE
 K-Search	25	3	1.583×					
 CUDA-Agent	26	2	2.210×					
 AdaExplore	25	7	1.316×					
 KernelAgent	26	6	1.922×					
 KernelFoundry	25	10	1.019×					

◆ Every Workflow Leads Somewhere ◆

附录 | KerSor: 公开结果与跨后端接口证据

Official B200 Score

	SOL bound	1.0000
	public-best envelope	0.7682
	top submission	0.7614
	KerSor	0.6443
	scoring baseline	0.5000
	reference implementation	0.2105

Interface Compatibility

	NVIDIA B200	18 sessions
	NVIDIA 4090/A800/H800	90+ solutions
	Huawei 910B	30 sessions
	MetaX C550	77 sessions
	AMD gfx1151	8 sessions



Stop



Trust Gate



Multi-Round



Contract Reuse

Compatibility $\neq$ Performance Ranking

附录 | OMOE Full Cross-System Board

A

Qwen3-4B/B200

Target Cell	Precision / Setup	Ours	Peer (SGLang)	Ratio
TG BF16 4k	BF16	2.669 ms/tok	3.533 ms/tok	1.324×
TG BF16 8k	BF16	2.804 ms/tok	3.645 ms/tok	1.300×
PP BF16 4k	BF16	27.89 ms	29.73 ms	1.066×
PP BF16 8k	BF16	61.41 ms	64.68 ms	1.053×
PP FP8 B16×512	FP8	≈33.7 ms	≈39.5 ms	≈1.17×

B

Llama-3.1-8B/B200

Target Cell	Precision / Setup	Ours	Peer (SGLang)	Ratio
TG BF16 4k	BF16	4.4018 ms/tok	4.3853 ms/tok	0.996×
PP BF16 8k	BF16	94.47 ms	100.85 ms	1.068×
Mixed (c16) Throughput	Mixed (c16)	3103.12 tok/s	4156.70 tok/s	0.7465×

C

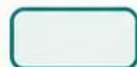
Qwen3.5-2B/RTX 5070

Target Cell	Precision / Setup	Ours	Peer (SGLang)	Ratio
BF16 TG 4k (Throughput)	BF16	71.1 tok/s	67.14 tok/s	1.059×
Mixed TG 4k (Throughput)	Mixed	76.9 tok/s	SGLang FP8 80.58 tok/s	0.954×
FP8 PP 4k (Throughput)	FP8	11025 tok/s	16134 tok/s	0.683×

D

Edge Peers · Target-Specific Latency

Target Cell	Model / Setup	Ours	Peer	Ratio
Pi0.5 / Orin	BF16	313.27 ms	PhyAI 333.88 ms	1.066×
Qwen3-0.6B / M1 W8	W8	11.117 ms/tok	Ollama Q8_0 15.317 ms/tok	1.378×



Ahead

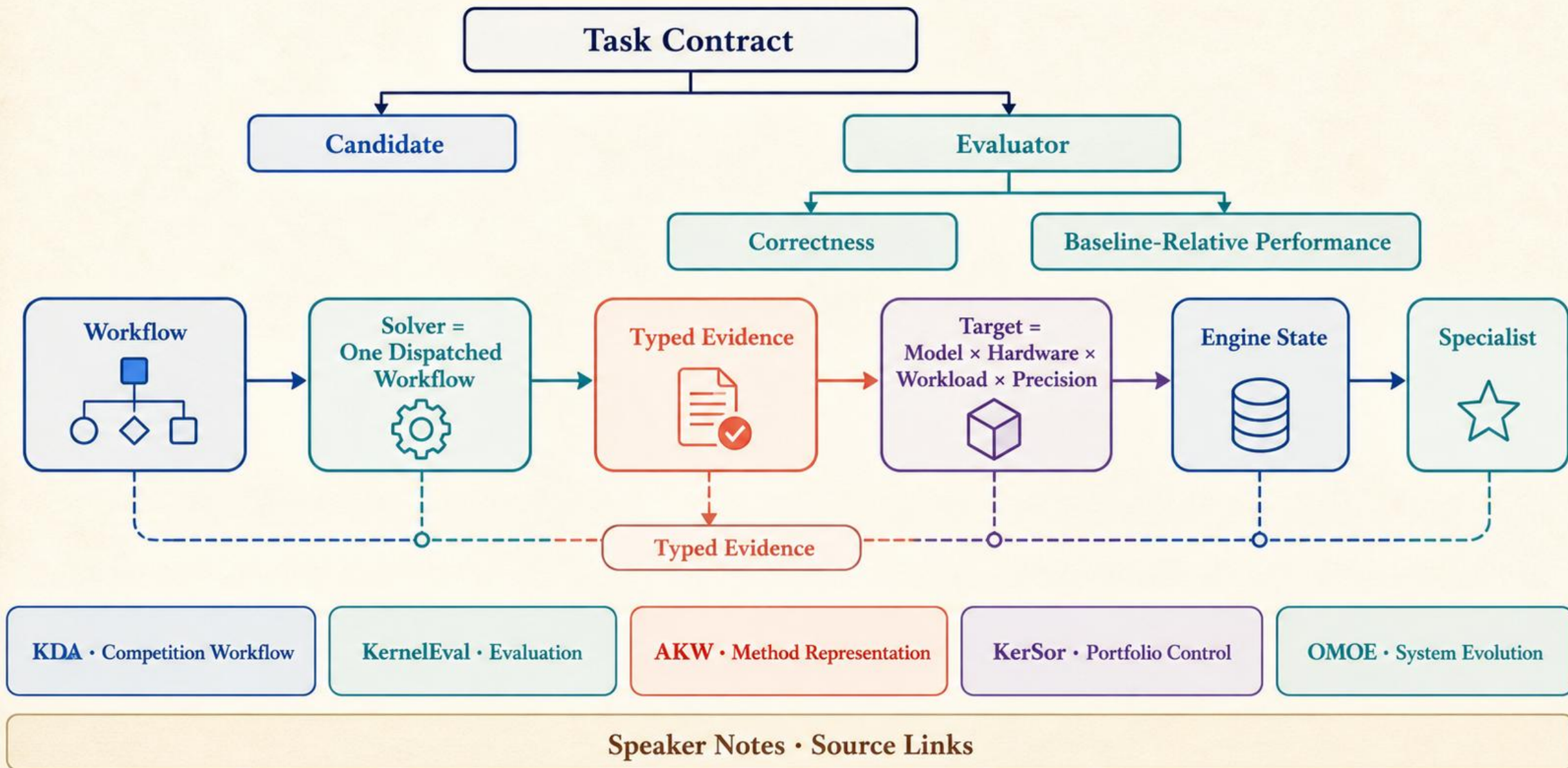


Open Gap



Each Row = Separate Target Cell

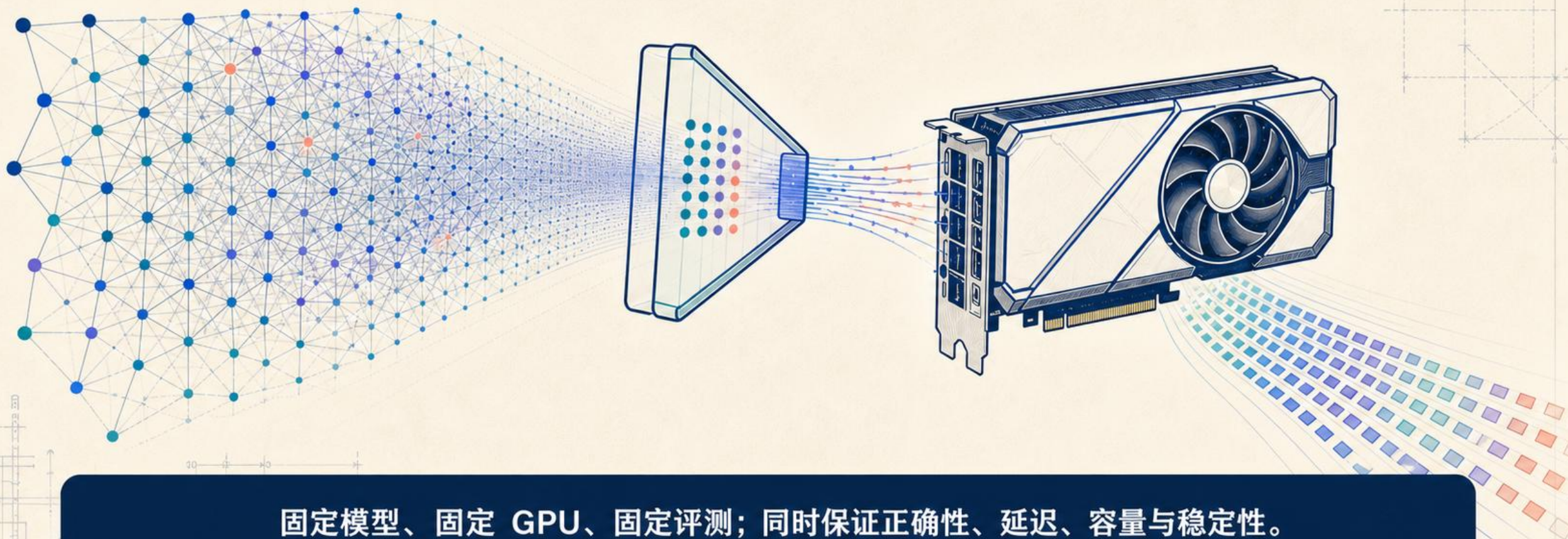
附录 | 课程术语与来源地图



APXInf 推理优化实验

一张 RTX 4090，能把 27B 模型做到什么程度？

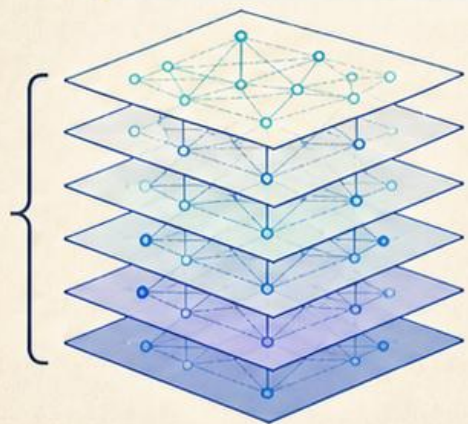
Qwen3.8-27B AWQ INT4 · 7 天系统实验



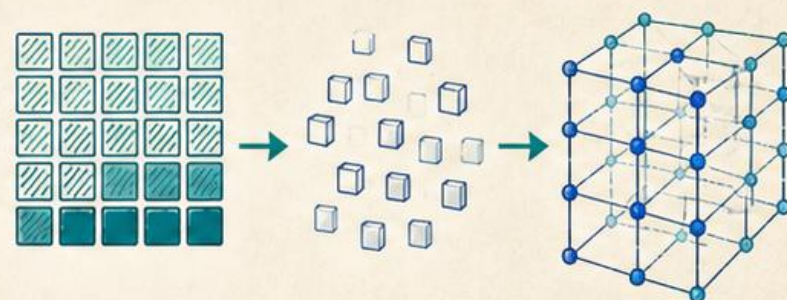
固定模型、固定 GPU、固定评测；同时保证正确性、延迟、容量与稳定性。

为什么这个题有挑战？

模型规模
27B $\leftrightarrow$ 24GB



量化路径
INT4 $\leftrightarrow$ 语义正确



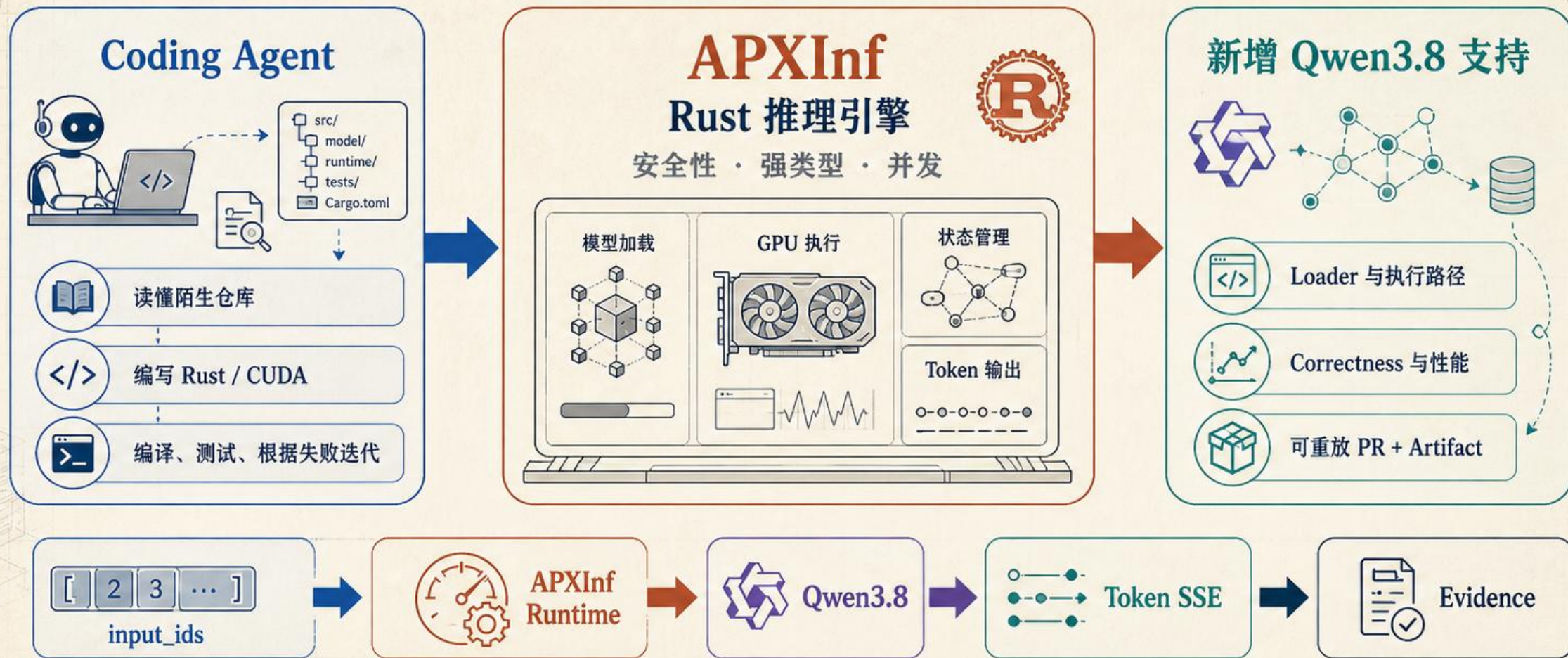
服务瓶颈
Prefill $\leftrightarrow$ Decode



✦ 能启动 $\neq$ 能正确推理；Kernel 快 $\neq$ 服务端到端快。 ✦

APXInf: 用 Coding Agent 扩展一个 Rust 推理引擎

课程不是考 Rust 语法，而是让 Coding Agent 进入一个真实、陌生的系统工程。



✦ Rust 的陌生感不是额外门槛，而是观察 Coding Agent 如何放大开发能力的实验条件。 ✦

什么固定，什么可以优化？

冻结合同



Qwen3.8-27B AWQ INT4 与固定 revision



单张 RTX 4090; TP=PP=DP=1



greedy、temperature=0、thinking 关闭



教师预分词 input_ids; 完整 128 token 输出



客户端计时; 公开/隐藏 Evaluator 与评分合同



优化空间



Loader、packing、W4A16 GEMM / GEMV



GDN、Attention、KV / recurrent state



Prefill tile、CUDA Graph、Host / Device 边界



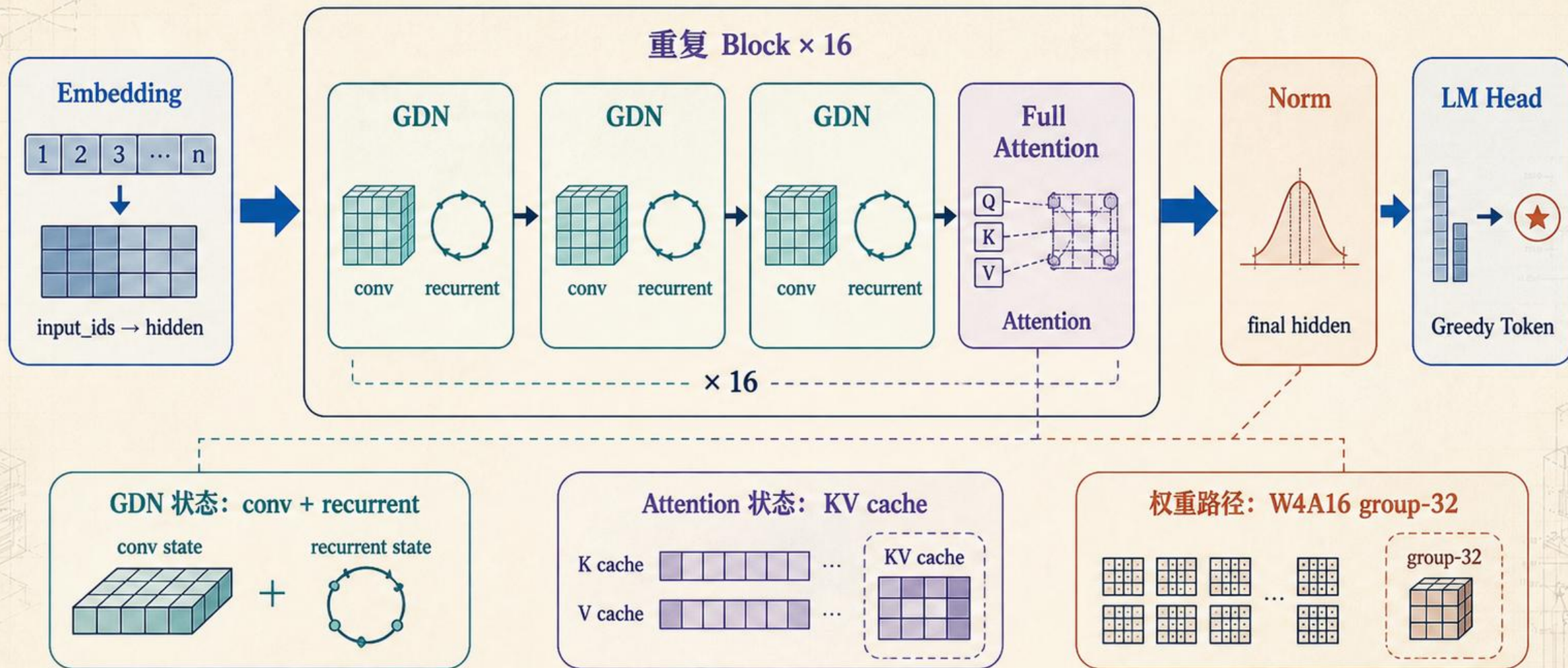
长上下文、调度、多请求与 Tail latency



可选真实图片路径: Vision、Merger 与 mRoPE

★ 优化自由来自合同冻结：不能靠换模型、改输入、缩短输出或静默 fallback 获得成绩。 ★

Qwen3.8: GDN 与 Full Attention 交替的混合主干

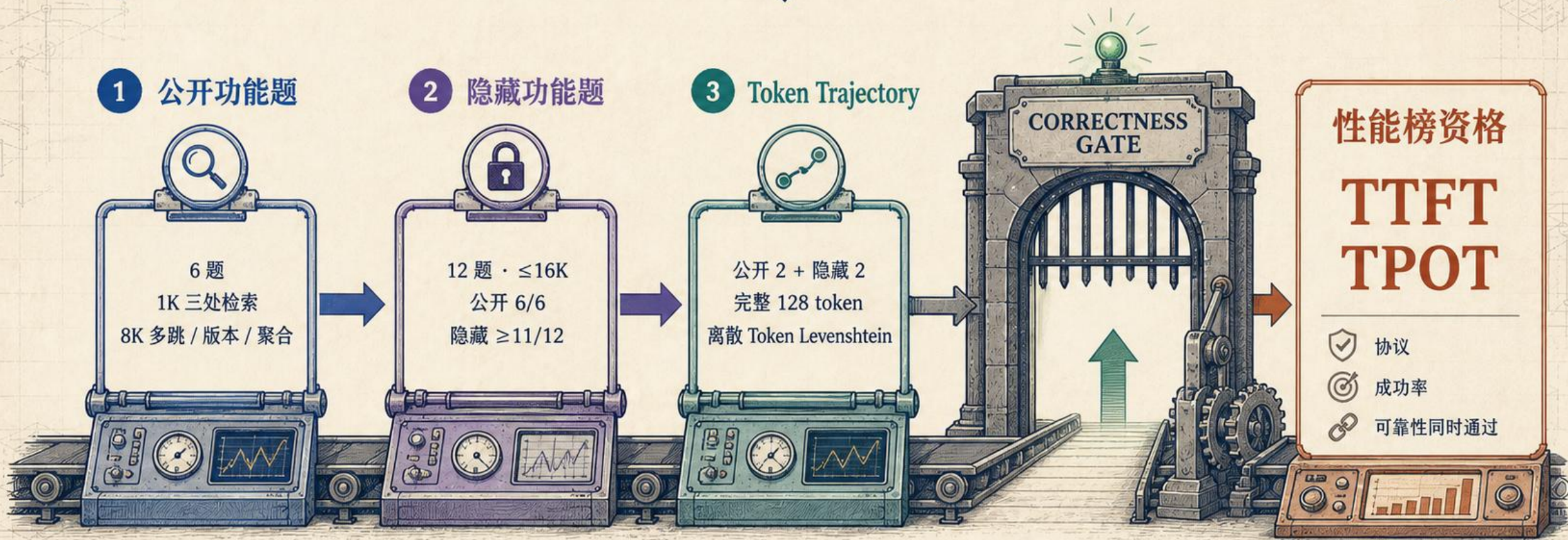


Request reset、cache ownership、position 与 state 更新, 都是 Correctness.

实验地图：100 分文字主榜 + 30 分扩展



先过 Correctness Gate, 性能才有资格排名



功能题使用固定 exact validator; Trajectory 比较离散 Token ID, 不使用 LLM Judge。

流畅文本不等于模型语义正确; 错误路由和静默 fallback 都会让证据失效。

TTFT 与 TPOT: Prefill 和 Decode 是两个问题



TTFT

Client Send → First Token

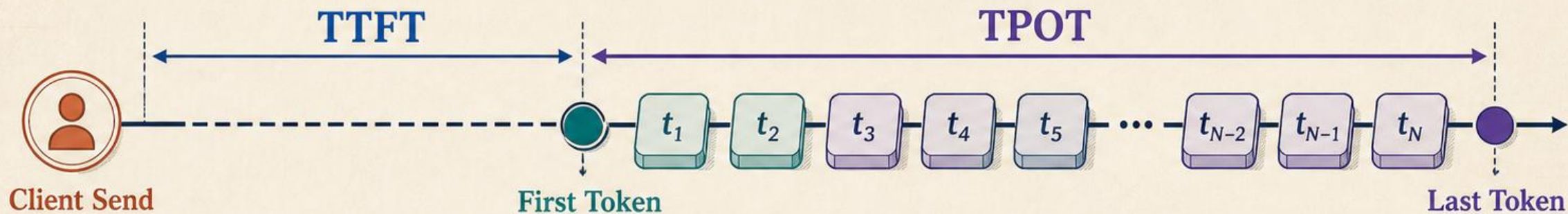
1K · 2K · 4K · 8K · 16K



TPOT

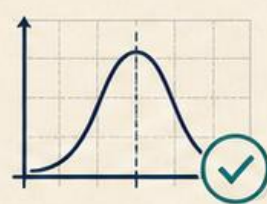
(Last - First) / (N - 1)

1K · 8K



每个 Cell:

1 warm-up
+ 5 measured



正式统计:

Median ·
CV ≤ 10%



输出预算:

完整
128 token



客户端计时是正式成绩; Server 自报时间只作诊断。



vLLM 是同机 Control, 不是固定满分答案

$$\text{cell_points} = \text{weight} \times \min(1, \text{best_valid_median} / \text{observed_median})$$

该 Cell 的最好有效中位数
成为满分参考



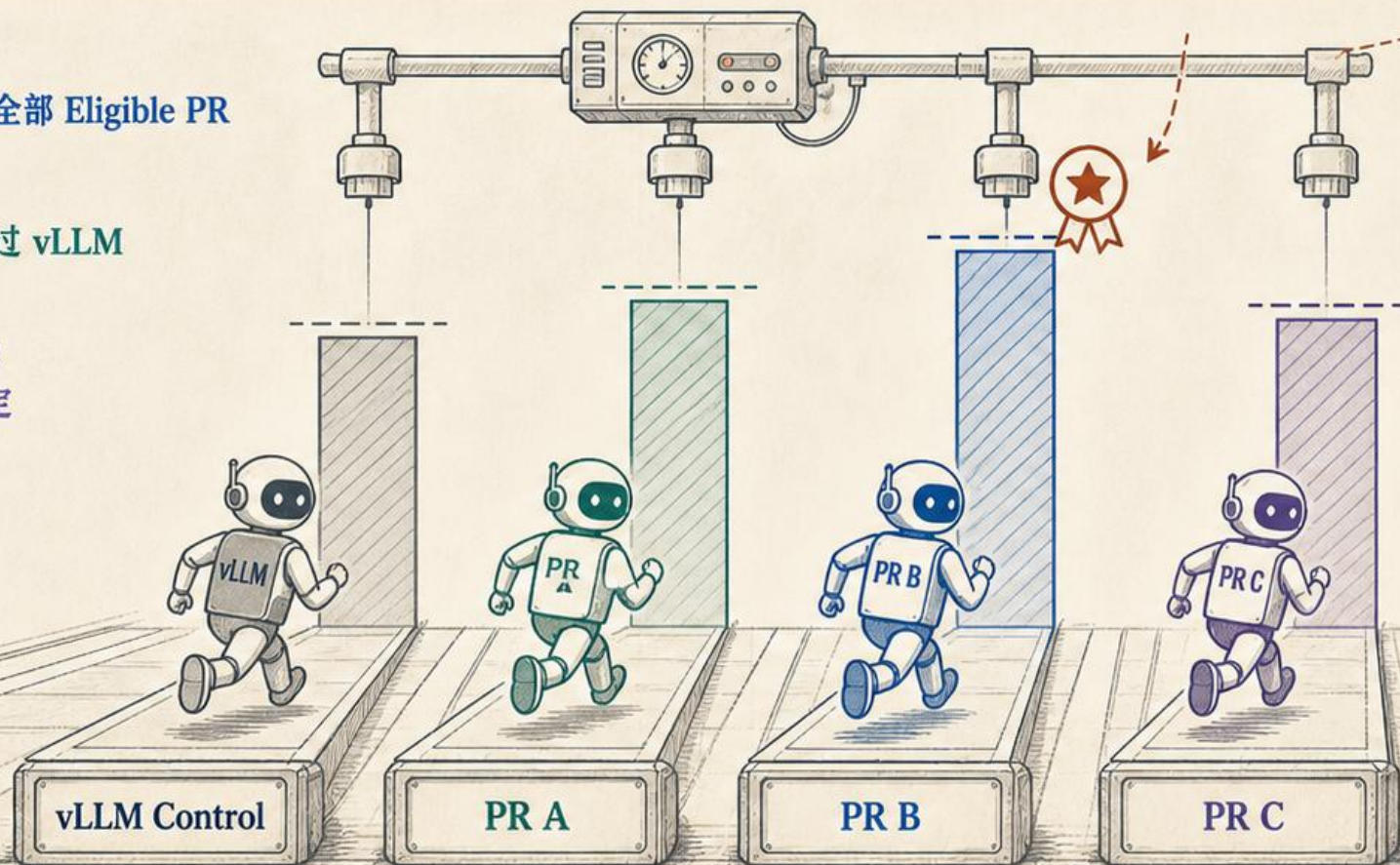
每轮统一运行 vLLM + 全部 Eligible PR



学生可以在某个 Cell 超过 vLLM



中期榜与结课榜都在冻结
Cohort Snapshot 后确定



长上下文：32K 只是 0 分起点



容量必须由六类任务验证

early · middle · late retrieval | 多跳 | 版本覆盖 | 聚合

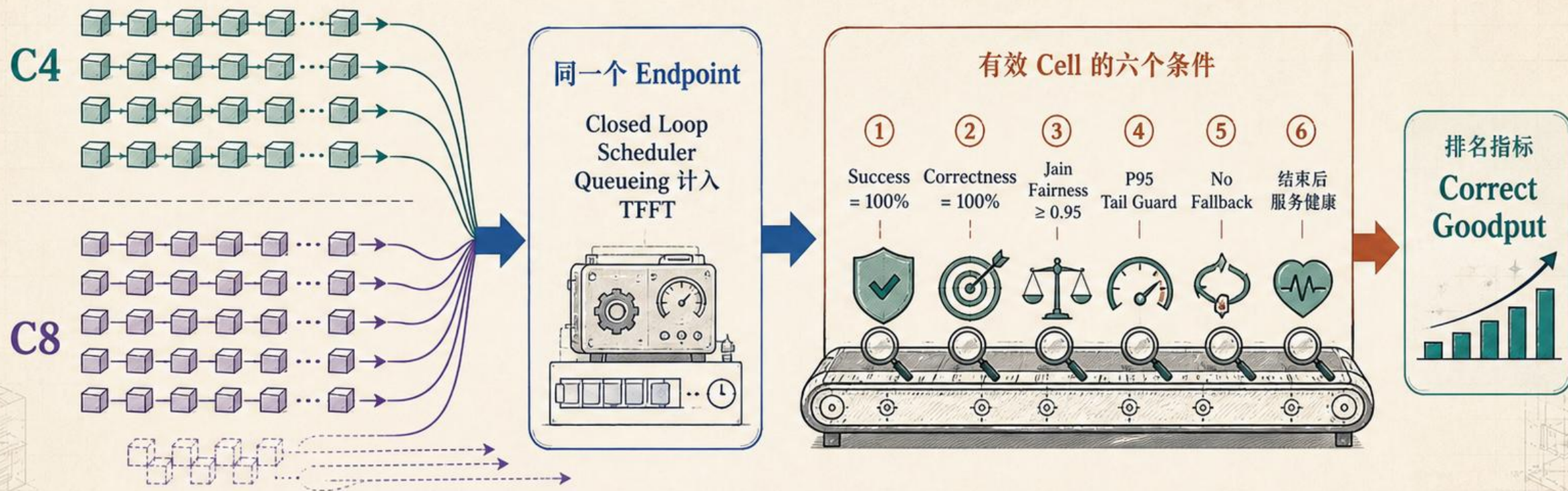


失败边界后：

/health 恢复 + 小请求恢复



多请求：吞吐不能掩盖尾延迟



每个 Cell:
 $32 \times 1K/128$ 请求

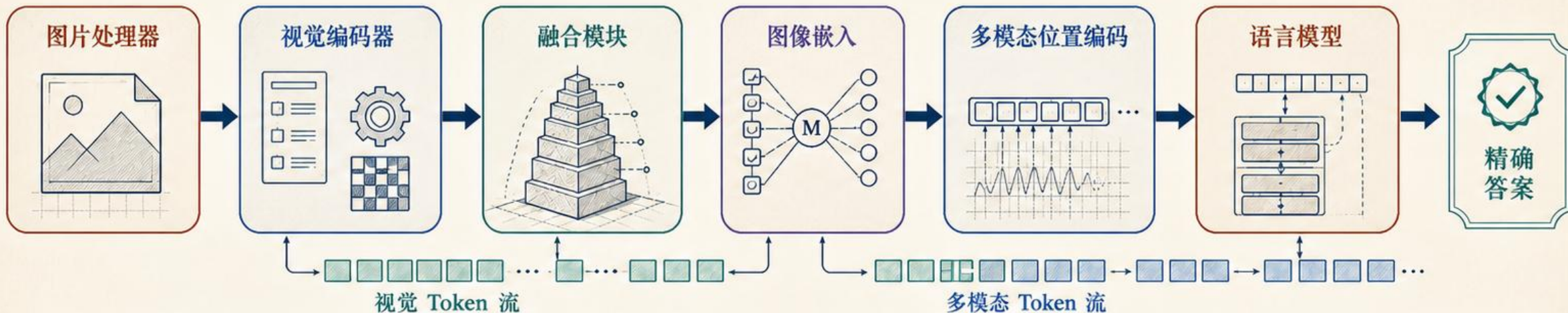


排名指标:
Correct Goodput



Bonus:
C4 5分 + C8 5分

真实图片能力：正式进入 10 分多模态加分



正式 10 分多模态评分（满分 10 分）

- 1 公开正确性： $2 \times$ 公开通过率（4 题）
- 2 隐藏正确性： $6 \times$ 隐藏通过率（8 题）
- 3 公开集成可靠性： 1 分
- 4 隐藏集成可靠性： 1 分

总分 =
 $1 + 2 + 3 + 4$
(满分 10 分)

集成可靠性（第 3、4 项加分条件）

对应题目全过 + 请求成功率 100% + 无回退 + 结束后健康

得分示例

- ✓ 公开 4/4, 隐藏未运行 → 3/10
- ✓ 公开 4/4, 隐藏 7/8 → 8.25/10
- ✓ 公开 4/4, 隐藏 8/8 → 10/10

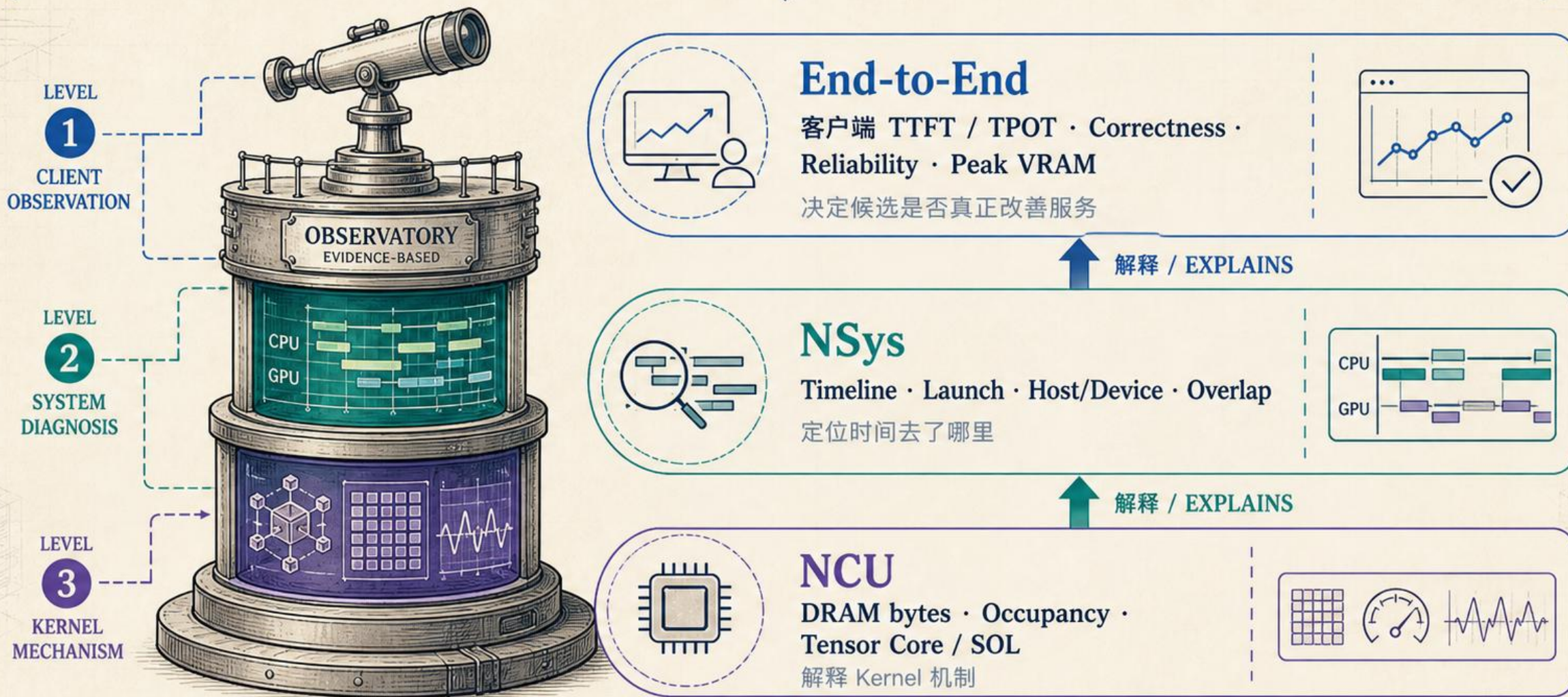


不支持图片仍可参加文字主榜，但多模态为 0 分，必须安全拒绝。



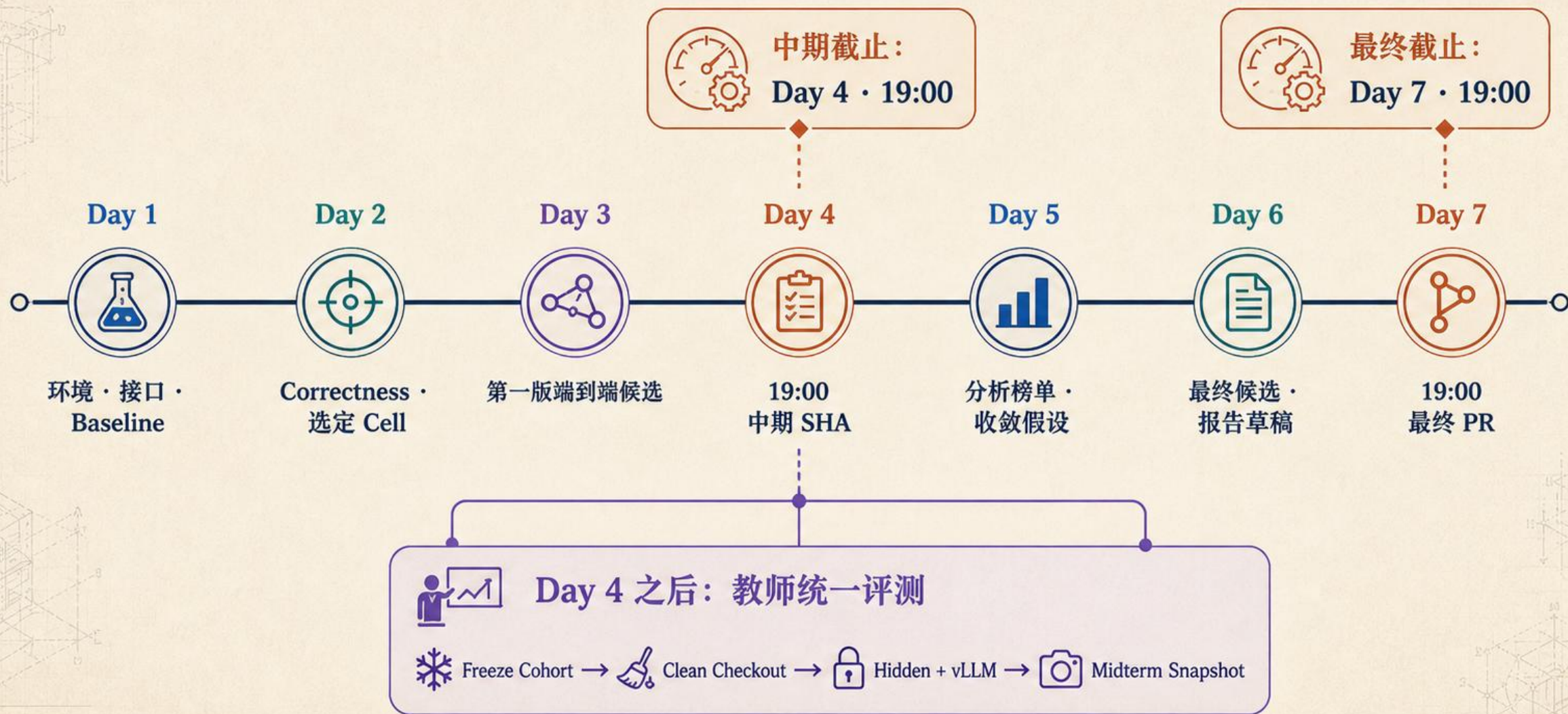
图片延迟与显存只做诊断，不做动态排名。

MFU、BWU 与 Profiler：用来解释原因，不替代端到端结果



GPU-Util $\neq$ MFU · Memory Utilization $\neq$ HBM BWU · Kernel Speedup $\neq$ Serving Speedup

一周任务周期：两个不可错过的冻结点



中期 Leaderboard: 只有统一 Cohort Snapshot 才能横向比较

学生提交三件东西

1 完整 PR SHA:

截止前可获取、不可变



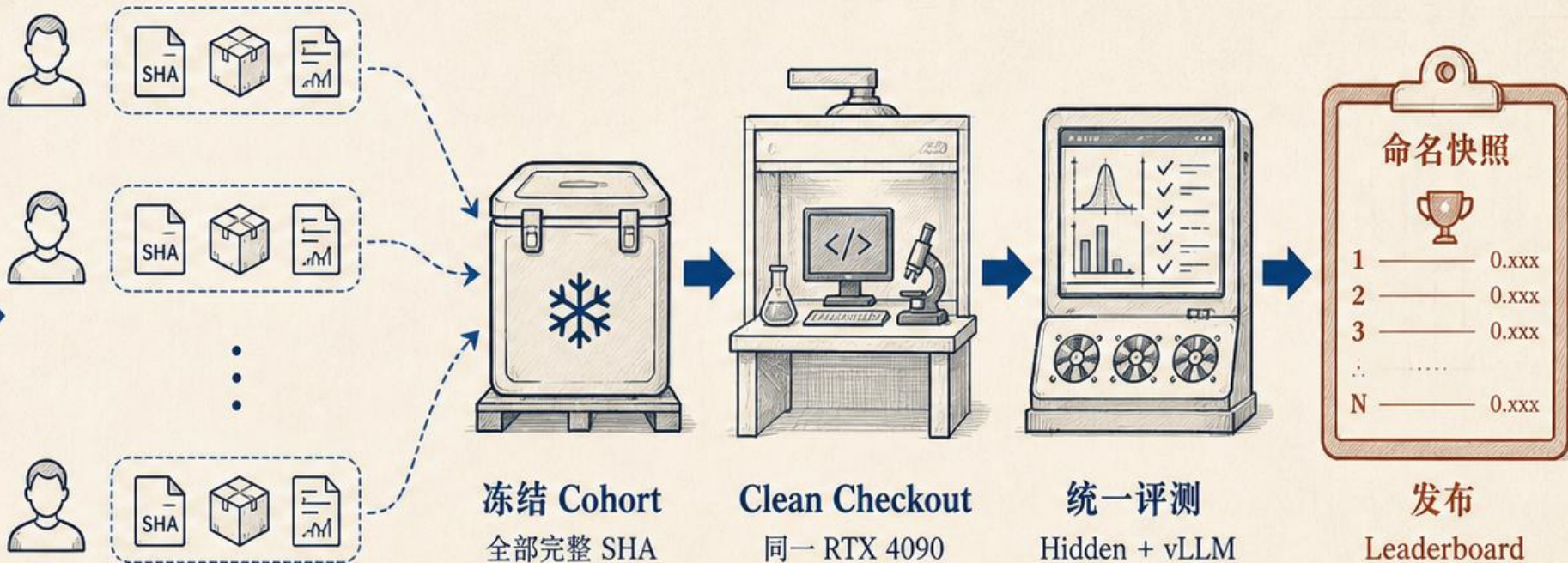
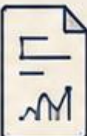
2 公开 Artifact:

必须由该 SHA 的 Runner 生成



3 中期简报:

Baseline · Candidate ·
Target Cell · 结果 ·
失败分支 · 限制



不发布只包含部分同学的滚动成绩；截止后的修改不回填中期 Cohort。

Workflow 创新与最终报告：让实验过程本身变得更可靠

Workflow 创新 · 实验 workflow 创新最多 5 分



新颖性与
系统相关性

2



可测量证据
价值

1



可复用性 /
文档 /
测试

1



Correctness /
SSOT

1

不改变 130 分自动排行榜；
课程总评仍封顶 100 分。



AUTOMATION / WORKFLOW BENCH

最终 REPORT.md · 建议 6-10 页



Baseline、执行路径与瓶颈



可证伪假设、实现与影响边界



Correctness、TTFT、TPOT、显存、效率



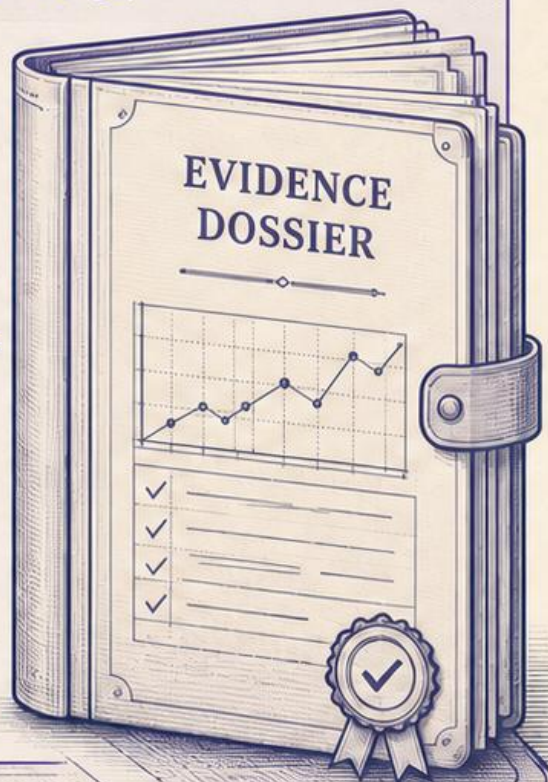
接受 / 拒绝 / 负结果 / 未决实验



中期到最终变化、限制与回滚



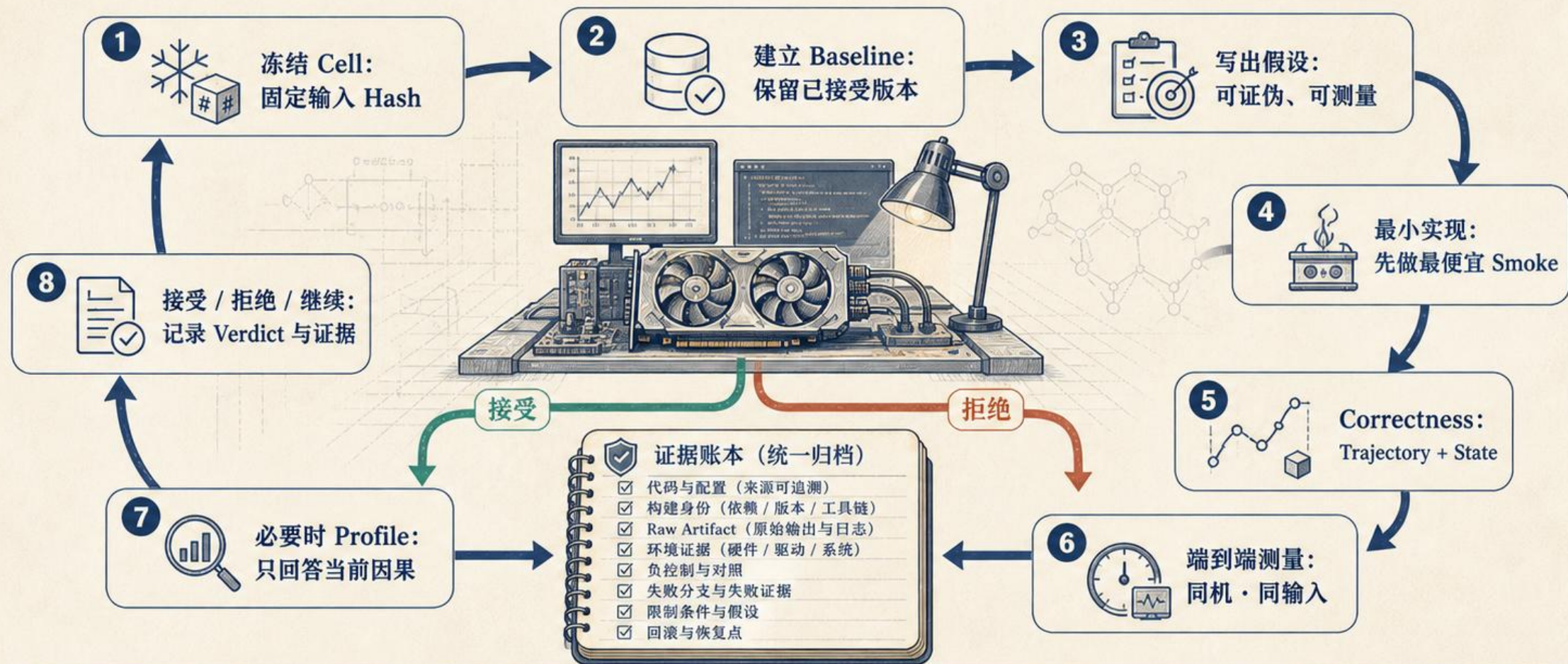
完整命令、PR SHA 与 Artifact Hash



TRUSTWORTHY EVIDENCE / REPORT

创新不是“脚本多”，报告也不是流水账：两者都要帮助别人复现并判断最终 Claim。

怎么做实验：让每个候选都留下可复核的决定



✧ 一个被证据拒绝的方向也是有效实验；没有 Correctness 和端到端结果的 Kernel Speedup 不是最终答案。✧

PR 需要同时留下代码、构建身份、Raw Artifact、环境证据、负控制、失败分支、限制与回滚。

开始你的第一个假设



01 Prefill:

为什么当前实现没有充分利用 RTX 4090?



02 Decode:

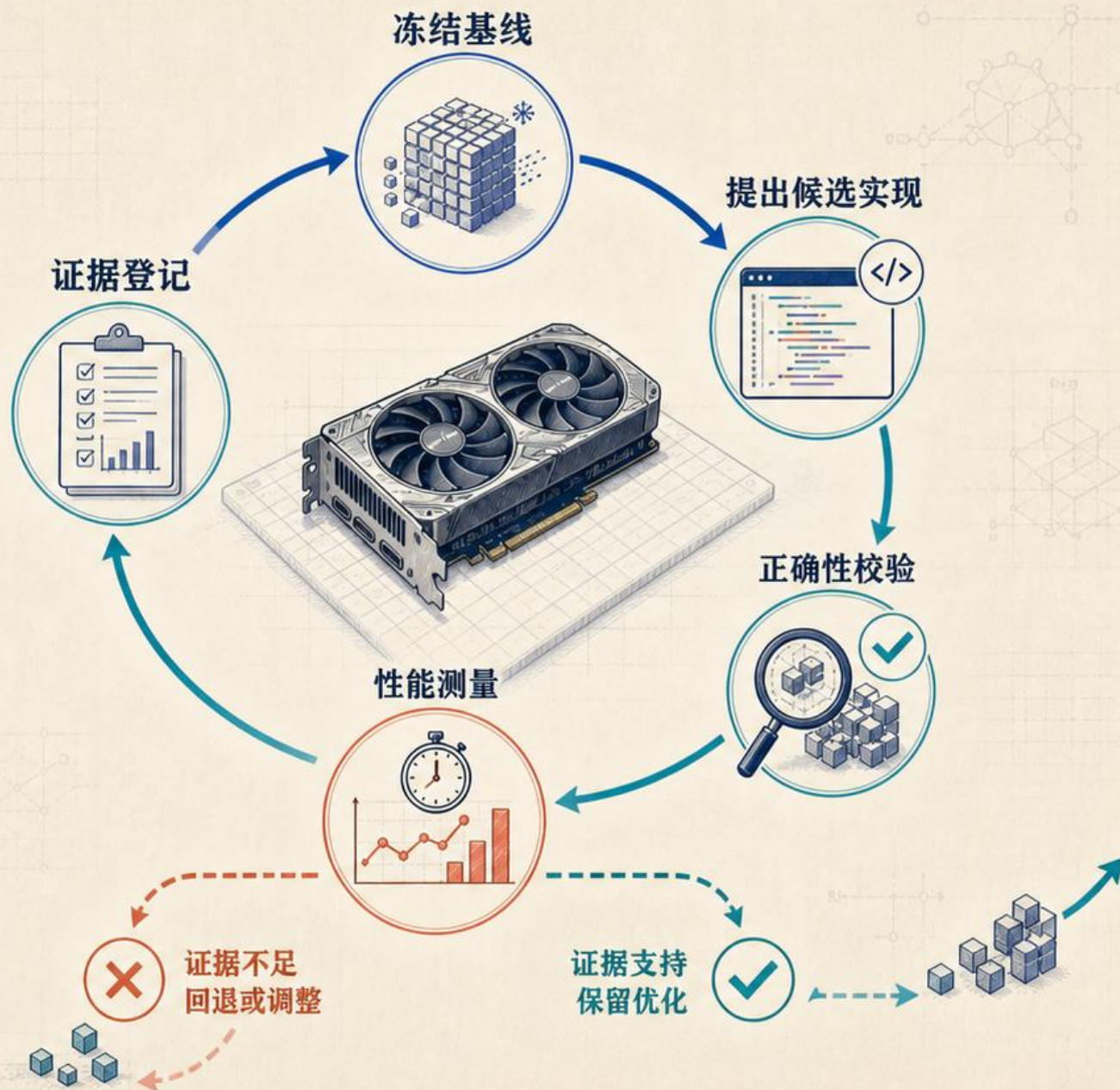
为什么后续 Token 更接近带宽与 Launch 受限?



03 容量 / 并发:

怎样扩大上下文或并发，
而不破坏 Correctness 与恢复?

先建立 Baseline，再让证据决定是否保留优化。



附录 | 公开接口速查

GET /health



- status: ok
- evaluation_contract
- model_revision
- max_model_len
- parallel_requests
- fallback_active: false
- capabilities



身份、容量和能力声明必须真实

POST /v1/evaluations/generate



input_ids: non-empty uint32[]



max_new_tokens: 128



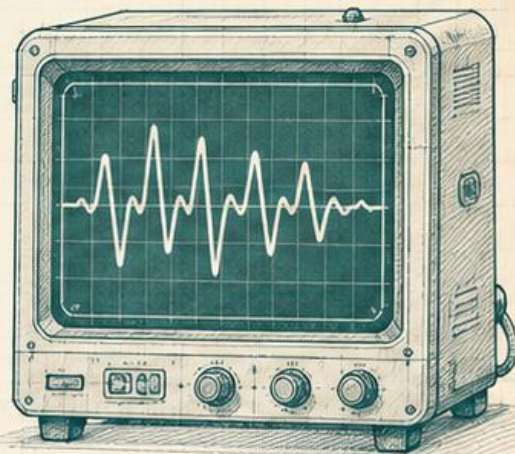
temperature: 0



ignore_eos: true



stream: true



SSE:
token event



contiguous index

0 1 2 3 ... n

done
event



[DONE]



非法输入、超容量与不支持能力必须
Fail Closed，失败后服务仍健康。

附录 | 评分与课程成绩速查

自动排行榜

文字主榜

100

长上下文

10

多请求

10

多模态

10

自动排行榜上限

130

100 分主榜评估维度

正确性

30

首词元延迟

35

逐词元生成间隔

25

可靠性

10



测量规则：1次预热 + 5次正式测量 + 中位数 + 变异系数 $\leq 10\%$

课程成绩



自动部分最多

80 分



代码评审

20 分



实验 workflow 创新最多

5 分



课程总评封顶

100 分



自动课程分 = $\min(80, 0.8 \times \text{合格排行榜分})$

附录 | 六个最容易误解的地方

误区 1 vLLM 是同机对比, 不是固定满分

同机 Control;
当轮最好有效实现才是 Cell 满分



误区 3 Token ID 可以算 MAE

Trajectory 使用离散
Token Levenshtein

3294	102	77	50256	...
↓	↓	↓	↓	
3294	102	71	50256	...

误区 5 Kernel 更快, 服务就更快

最终由无 Profiler 的客户端
TTFT/TPOT 决定



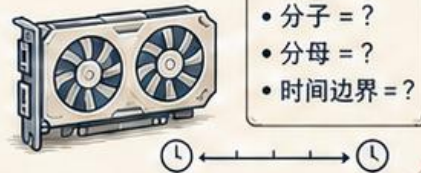
误区 2 达到 32K 就拿满分

32K 是零点起跑线,
不是终点



误区 4 GPU-Util 不是 MFU

MFU/BWU 必须说明
Proxy、分子、分母和
时间边界



误区 6 不做图片仍可参加文字主榜; 多模态得分为 0, 必须声明 不支持并安全拒绝图片请求。



新总榜: 100 分文字主榜 + 30 分扩展 = 130 分。

加入 APXInf 实验交流群

实验环境、错误排查、性能结果和优化思路，都可以在这里交流。



🔧 课程仓库 🔧

仓库地址: <https://github.com/infinigence/ApxInf>

👥 欢迎提交问题、分享实验，也欢迎为项目点亮 **Star**。

❓ 提问时请附：代码版本、运行命令、错误信息和已尝试的方法。



扫描二维码加入群聊

✦ 扫描二维码加入群聊 ✦